1.015 CÂU HỎI & TRẢ LỜI PHỎNG VẤN FULL-STACK — THE MANY COMPANY

> **Phạm vi:** Trả lời đầy đủ 1.015 câu trong file nguồn, theo nguyên thứ tự; không loại câu. Các đáp án kỹ thuật được viết bằng tiếng Anh theo phong cách phỏng vấn, đi kèm phụ lục thực hành SQL/TypeScript ở cuối. **Lưu ý về tính trung thực:** Những câu về năm kinh nghiệm, CV, dự án, kết quả cá nhân, khả năng pháp lý và mức lương không thể xác nhận từ file nguồn. Chúng được giải đáp bằng nội dung/cách trình bày cần có, không bịa chuyện, thành tích hay thông tin cá nhân. Các câu 996–1015 là câu *ứng viên hỏi nhà tuyển dụng*: phần trả lời giải thích mục đích và dữ liệu nên hỏi; không giả mạo câu trả lời từ công ty. **Phân biệt nguồn:** Câu hỏi, tên phần, vị trí và công nghệ được trích từ tài liệu bạn cung cấp. Các câu trả lời và đoạn mã trong tài liệu này là phần giải thích kỹ thuật được biên soạn thêm, không phải phát ngôn chính thức từ The Many Company.

Mục lục: Các câu hỏi & đáp án · Phụ lục A – PostgreSQL · Phụ lục B – TypeScript · Phụ lục C – giao dịch stock · Phụ lục D – kiểm thử · Phụ lục E – runbook

Câu hỏi và đáp án

PHẦN 1. INTRODUCTION — GIỚI THIỆU BẢN THÂN VÀ KINH NGHIỆM

Câu 1. Tell me about yourself and your experience as a full-stack developer.

Answer (English): I focus on building end-to-end web software: understanding the workflow, designing data models and APIs, implementing the interface, and supporting it after release. For this role, I would emphasize TypeScript, relational data integrity, and operational reliability; I would describe only projects I actually completed.

Câu 2. Can you walk me through your professional background?

Answer (English): I would explain my career chronologically: the products I contributed to, the responsibilities I personally owned, and how those experiences prepared me to build operational systems. I would connect each transition to a concrete skill rather than recite a list of tools.

Câu 3. How many years of professional experience do you have with React, Node.js, and TypeScript?

Answer (English): I would state my actual years of professional use separately for React, Node.js, and TypeScript, distinguishing paid production experience from personal projects. Accuracy matters more than inflating a number, and I would support it with a project example.

Câu 4. What types of web and mobile applications have you built?

Answer (English): I would identify the real categories I have shipped—such as dashboards, API-backed websites, or mobile interfaces—and explain their users, major workflows, and deployment status. I would distinguish prototypes from production applications.

Câu 5. What is the most technically challenging project you have worked on?

Answer (English): I would choose a genuine hard problem, explain the original symptoms, relevant constraints, options considered, implementation, verification, and outcome. A concurrency or synchronization bug is particularly relevant to this job, but I would not claim one unless I solved it.

Câu 6. Which project best demonstrates your ability to work across the entire stack?

Answer (English): The strongest example spans UI, API, relational schema, authorization, testing, and deployment. I would trace one user action through each layer and identify precisely which parts I implemented myself.

Câu 7. What was your specific contribution to your most recent project?

Answer (English): I would separate my own deliverables from the team's: the database changes, API endpoints, UI flows, tests, or operational fixes I authored. I would mention a code review or deployment responsibility if it actually happened.

Câu 8. Have you built an application from scratch, starting with requirements and database design?

Answer (English): I would answer yes only if I owned requirements discovery and initial data modeling; otherwise I would explain which phases I participated in. I would walk through entities, constraints, endpoints, UI, release, and what I learned.

Câu 9. Have you maintained an existing production application?

Answer (English): Maintaining production means diagnosing incidents, making backward-compatible changes, adding regression tests, and monitoring the release. I would provide a real example, or clearly say I have not yet held production on-call responsibility.

Câu 10. Which project are you most proud of, and why?

Answer (English): I would select a project with demonstrable user value and discuss the quality improvements I can substantiate. Pride should reflect ownership and learning, not unsupported claims about revenue or scale.

Câu 11. What was the business problem that your most successful project solved?

Answer (English): I would identify a specific user pain point, why existing steps failed, and the measurable workflow improvement. The answer should be about business outcomes, not merely which framework I used.

Câu 12. How did you measure whether your implementation was successful?

Answer (English): I would compare before-and-after metrics such as task completion time, error rate, latency, adoption, or support tickets. I would state which numbers were observed and which were only estimates.

Câu 13. What is the largest or most complex database you have worked with?

Answer (English): I would give the real order of magnitude of tables, rows, queries, and transactions, then describe indexes, integrity constraints, or migrations I owned. I would not guess at production scale.

Câu 14. Have you worked with inventory management, e-commerce, logistics, or operational software?

Answer (English): I would disclose the exact level of inventory, e-commerce, logistics, or operations experience I have. Even without warehouse experience, transferable skills include transactional modeling, workflow design, audit logs, and API integrations.

Câu 15. Have you integrated a third-party API into a production system?

Answer (English): I would describe one genuine integration: authentication, data mapping, pagination, retries, idempotency, rate limiting, observability, and failure recovery. I would distinguish reading an API from operating a reliable integration in production.

Câu 16. What is the most difficult production bug you have fixed?

Answer (English): I would describe impact and detection, narrow the failure using logs and reproducible inputs, verify the root cause, ship a safe fix, and add a regression test. The key is demonstrating evidence, not guessing.

Câu 17. Tell me about a technical decision you made that significantly improved a product.

Answer (English): I would explain the alternatives, constraints, why I chose the design, and the measured outcome; examples might include adding a database constraint or moving slow work to a background job. Any claimed improvement must be verifiable.

Câu 18. Have you ever disagreed with a technical decision made by a teammate or manager?

Answer (English): I would explain the disagreement respectfully, present data and trade-offs, seek a decision, and support the team once the decision was made. The point is collaborative judgment rather than winning an argument.

Câu 19. What are your strongest technical skills?

Answer (English): My strongest skills should be supported by evidence from real code and decisions, ideally TypeScript, database-backed API design, debugging, or reliable integrations. I would identify depth, not just enumerate libraries.

Câu 20. Which parts of our technology stack are unfamiliar to you?

Answer (English): I would candidly identify tools I have not used deeply, such as Retool or Deno if applicable, and explain the fundamentals that transfer. I would propose learning them through docs, a small vertical slice, tests, and review.

Câu 21. How do you learn a new framework or tool when you need to use it in production?

Answer (English): I start with official documentation and runtime constraints, build a small end-to-end feature, write tests for failure cases, and compare local behavior with production requirements. I learn APIs without abandoning established reliability practices.

Câu 22. Why are you interested in The Many Company?

Answer (English): This role combines product engineering with real warehouse workflows where correctness and traceability matter. I am interested in owning the full path from an operator's action to a durable, auditable database change.

Câu 23. Why does building software for warehouse operations interest you?

Answer (English): Warehouse software connects physical events with digital records; small errors cause lost time, stock discrepancies, and fulfillment problems. I find it meaningful to build interfaces and invariants that reduce those mistakes.

Câu 24. What interests you about our inventory and product management platform?

Answer (English): The interesting challenge is maintaining one dependable internal product/inventory model while integrating several external commerce systems. It requires thoughtful schema design, event processing, reconciliation, and usable workflows.

Câu 25. Why are you looking for a new project or position?

Answer (English): I would give my actual reason—new technical scope, better ownership, or a role more aligned with my skills—without making claims about prior employers. I would connect the transition to this position's responsibilities.

Câu 26. What would you like to achieve in your next role?

Answer (English): I want ownership of a meaningful system, exposure to difficult data-integrity problems, and a team that reviews architectural trade-offs carefully. I would aim to leave the codebase and operating procedures more reliable.

Câu 27. What does end-to-end ownership mean to you?

Answer (English): End-to-end ownership means clarifying requirements, defining invariants, implementing and testing all affected layers, deploying safely, observing results, and handling failures. It is responsibility for outcomes, not merely delivering a pull request.

Câu 28. What kind of engineering environment helps you perform your best?

Answer (English): I work best with clear priorities, documented decisions, small reviewed changes, reliable staging environments, and direct communication with users. Autonomy should come with observability and escalation paths.

Câu 29. Why should we hire you over a developer with more experience in PostgreSQL?

Answer (English): I would not pretend to outperform someone in an area where they have deeper experience. I would explain my strengths in product delivery and TypeScript while demonstrating a concrete plan to learn the specific PostgreSQL functions and locking patterns used here.

Câu 30. What would you need to learn during your first month to become effective in this role?

Answer (English): I would map the current schema and inventory invariants, trace receiving and picking flows, inspect migrations and recent incidents, understand synchronization ownership, and ship one low-risk change with tests. I would validate assumptions with warehouse operators and the architect.

PHẦN 2. PROJECT DEEP DIVE — ĐÀO SÂU DỰ ÁN ĐÃ LÀM

Câu 31. Can you draw the architecture of one of your recent projects?

Answer (English): I would draw client → API/router → service → repository/database plus background workers and external integrations. I would mark authentication boundaries, transaction boundaries, source-of-truth records, and failure/retry paths.

Câu 32. How does a request travel from the frontend to the database and back?

Answer (English): A request is validated in the browser, authenticated and validated again by the API, authorized in the service/database, executed inside any needed transaction, and returned as a typed DTO. Errors are mapped into stable responses and correlated with logs.

Câu 33. Why did you choose your frontend framework?

Answer (English): I choose React when component reuse, stateful workflows, and a strong TypeScript ecosystem matter. I would consider maintenance costs, accessibility, rendering performance, and whether a low-code UI is sufficient.

Câu 34. Why did you choose your backend framework?

Answer (English): I choose a backend framework for its routing model, validation hooks, runtime compatibility, ecosystem, and operational simplicity. The business logic stays in services so switching Express, Hono, or another router is less risky.

Câu 35. Why did you choose your database?

Answer (English): PostgreSQL offers relational constraints, transactions, mature indexing, JSONB for bounded metadata, and strong analytical SQL. Inventory correctness benefits more from ACID and clear relationships than from storing arbitrary documents.

Câu 36. How did you design the database schema?

Answer (English): I start with business entities and invariants, define keys and foreign keys, normalize stable relationships, and then add indexes driven by queries. Migrations and realistic test data validate the design before production.

Câu 37. How did you decide which tables and relationships you needed?

Answer (English): Each table represents a distinct entity or lifecycle with its own constraints. One-to-many and many-to-many relationships come from real operational rules, not arbitrary object nesting.

Câu 38. How did you structure your API endpoints?

Answer (English): I group endpoints by business resources and commands: products, orders, bins, receipts, reservations, transfers, and adjustments. Mutating operations accept idempotency keys, validate payloads, and return meaningful conflict errors.

Câu 39. How did you handle authentication and authorization?

Answer (English): Authentication proves identity; authorization checks permission for each action and resource. I use session/JWT validation plus backend RBAC or database RLS, and test access across tenants and warehouses.

Câu 40. How did you validate user input?

Answer (English): I validate untrusted data at runtime with schemas, enforce domain invariants in the service, and use database constraints as the final defense. Client validation improves UX but never establishes trust.

Câu 41. How did you handle errors across the frontend and backend?

Answer (English): I use structured API errors with codes, correlation IDs, and retry hints; the UI renders actionable messages. Sensitive internal details go to logs, while expected validation and conflict errors are not treated as system crashes.

Câu 42. How did you manage application configuration and environment variables?

Answer (English): Typed configuration is loaded and validated at service startup, with secrets from protected environment variables or a secret manager. I use per-environment settings and never commit credentials or expose server secrets in frontend bundles.

Câu 43. How did you deploy the application?

Answer (English): I build and test immutable artifacts, run reviewed migrations using a safe deployment order, deploy with health checks, and monitor key metrics. I prepare rollback or forward-fix options before the release.

Câu 44. How did you manage database migrations?

Answer (English): I version migrations in Git and apply them once in a controlled pipeline, not implicitly from every app instance. I review lock impact, compatibility, backfills, and recovery strategy.

Câu 45. How did you test the application before releasing it?

Answer (English): I run type checking, linting, unit tests, integration tests against a real test database, and smoke tests on critical user workflows. High-risk changes need concurrency and

negative-path tests.

Câu 46. How did you monitor the application after deployment?

Answer (English): I monitor error rates, latency, logs, traces, database health, queue backlog, and business invariants such as negative stock or failed reconciliations. Alerts require an owner and a response procedure.

Câu 47. What would happen if your database became temporarily unavailable?

Answer (English): Read-only pages may show a clearly labeled unavailable or cached state; writes fail closed rather than pretending to succeed. Short transient failures may be retried safely with idempotency, while workers resume after recovery.

Câu 48. What happens if a request is submitted twice?

Answer (English): Without safeguards, duplicate submissions can double-adjust inventory or create duplicate shipments. I assign a stable idempotency key, persist it under a uniqueness constraint, and return the previous result for retries.

Câu 49. What would you change if you had to redesign the project today?

Answer (English): I would first inspect observed problems and usage metrics; likely targets include clearer domain boundaries, stronger constraints, smaller transactions, better observability, or safer migration procedures. Redesign should be justified by evidence.

Câu 50. What technical debt did you knowingly accept, and why?

Answer (English): I accept bounded debt when the cost is known, risk is low, and repayment is tracked—for example a basic internal screen before generalizing a design system. I avoid debt that compromises authorization, data integrity, or recoverability.

Câu 51. How did you decide which parts needed abstraction and which did not?

Answer (English): I abstract logic used by multiple workflows when it expresses a stable invariant; I keep one-off transformations local. Premature abstractions can hide transaction boundaries and make debugging harder.

Câu 52. What was the hardest bug in the project, and how did you find its root cause?

Answer (English): I would demonstrate symptom → instrumentation → smallest reproduction → root cause → regression test → fix. For data bugs I would inspect affected rows/events and transaction history before changing anything.

Câu 53. How would you scale the application if usage increased tenfold?

Answer (English): First measure bottlenecks and business invariants. Optimize queries and indexes, cap concurrency, queue expensive work, and horizontally scale stateless services where useful; never trade away inventory consistency for throughput.

Câu 54. How would another developer understand and maintain your code?

Answer (English): Clear domain names, small services, explicit database constraints, tests, ADRs, examples, and runbooks make code understandable. I document why invariants exist and how to recover from failures.

Câu 55. Can you walk me through a pull request you are particularly proud of?

Answer (English): I would walk through the actual diff, requirement, main design choice, tests, review feedback, and deployment outcome. I would be transparent about which commits and decisions were mine.

PHẦN 3. TYPESCRIPT — RẤT QUAN TRỌNG

Câu 56. [ƯU TIÊN CAO] What is the difference between `type` and `interface` in TypeScript?

Answer (English): `interface` mainly declares object shapes and supports declaration merging; `type` aliases can express unions, tuples, primitives, and mapped/conditional compositions. Both can describe object types; choose consistent conventions rather than claiming one is universally faster.

Câu 57. What is the difference between `any`, `unknown`, and `never`?

Answer (English): `any` disables most type checking; `unknown` requires narrowing before use; `never` represents an impossible value or an exhaustive branch that should be unreachable. Use `unknown` for untrusted input, not `any`.

Câu 58. What is type inference?

Answer (English): Type inference lets TypeScript derive types from initializers, return expressions, and contextual usage. Explicit annotations remain valuable at public API boundaries and where inferred types would be too broad.

Câu 59. What are union types and intersection types?

Answer (English): A union `A | B` permits either shape; an intersection `A & B` requires both sets of requirements. For example, an event may be `Received | Picked`, while an audited event might be `Event & { actorId: string }`.

Câu 60. What are discriminated unions?

Answer (English): A discriminated union gives each variant a shared literal tag, enabling safe narrowing in `switch`. Different inventory events can then require different fields without making every field optional.

Câu 61. What are generics, and when would you use them?

Answer (English): Generics parameterize a function or type so it preserves relationships between input and output types. A repository `getById<T>` or `Result<T, E>` can remain reusable without turning values into `any`.

Câu 62. What is the difference between optional properties and properties that can be `null`?

Answer (English): `name?: string` means the property may be absent/undefined; `name: string | null` requires the property but permits an explicit null. Database nullable columns and optional update fields often need different DTO types.

Câu 63. What does strict mode do in TypeScript?

Answer (English): `strict` enables a family of stronger compiler checks including strict null checks and function type checks. I combine it with appropriate lint rules and runtime validation; strict TypeScript cannot validate network payloads.

Câu 64. Why should you avoid using `any` in production code?

Answer (English): `any` lets invalid property access and assignments pass unnoticed, moving bugs into production. Prefer `unknown` at boundaries, inferred types internally, and narrow, documented assertions only where unavoidable.

Câu 65. How would you model an API response that can succeed or fail?

Answer (English): Use a discriminated response such as `{ ok: true; data: T } | { ok: false; error: { code: string; message: string } }`. The tag lets callers handle both cases exhaustively and keep expected failures separate from exceptions.

Câu 66. How would you define types for database entities and API DTOs?

Answer (English): Database entities reflect actual persisted columns and nullable fields; DTOs reflect public contracts and hide secrets/internal fields. Define both explicitly or derive carefully from a shared runtime schema.

Câu 67. Why should database models and API response types sometimes be different?

Answer (English): The persistence schema can contain internal IDs, audit fields, tokens, and nullable values that clients must not see. DTOs also aggregate or format information for UI consumers and maintain API compatibility during migrations.

Câu 68. What are utility types such as `Pick`, `Omit`, `Partial`, `Required`, and `Record`?

Answer (English): `Pick<T,K>` selects properties; `Omit<T,K>` removes them; `Partial<T>` makes fields optional; `Required<T>` requires them; `Record<K,V>` defines a key/value mapping. These operate at compile time only.

Câu 69. What is the difference between `as` type assertions and runtime validation?

Answer (English): `as Foo` tells the compiler to trust you without checking runtime data; parsing with a schema verifies actual values. Never use an assertion as proof that an external webhook payload is valid.

Câu 70. How would you validate untrusted data received from a webhook?

Answer (English): Verify the provider's signature against the unmodified raw body, enforce payload size limits, parse JSON, validate its schema and business identifiers, and write to a deduplicated inbox. Process asynchronously after durable acceptance.

Câu 71. What is the difference between compile-time type safety and runtime validation?

Answer (English): TypeScript types disappear at runtime; an API caller can still send malformed JSON. Runtime schemas, database constraints, and authorization checks defend real boundaries, while compile-time checks protect authored code.

Câu 72. What are type guards?

Answer (English): A type guard narrows a value after checking its shape, e.g. `function isString(x: unknown): x is string { return typeof x === 'string' }`. Guards should validate enough properties to justify the claimed type.

Câu 73. How do you narrow a union type?

Answer (English): Use `typeof`, `in`, instance checks when appropriate, and discriminant `switch` branches. A default assigning to `never` can flag newly added union cases that were not handled.

Câu 74. What are generics useful for in a reusable API client?

Answer (English): A typed API client can accept `request<TResponse, TBody>` parameters, preserving caller expectations. It must still validate the response at runtime when correctness matters; a generic alone never proves server content.

Câu 75. How would you represent an order state so invalid states are harder to create?

Answer (English): Represent order states as a literal union and implement an explicit transition table; validate transitions within a transaction. Better still, use distinct discriminated shapes where fields are required only in certain states.

Câu 76. What is the difference between an enum and a string literal union?

Answer (English): String-literal unions provide lightweight compile-time choices and work well with `as const`; enums can create runtime objects and sometimes less obvious behavior. Use a union unless runtime enum semantics are specifically useful.

Câu 77. How do `Promise<T>` and `async/await` work?

Answer (English): `Promise<T>` represents a future completion or rejection with a `T` value; `async` functions return promises and `await` pauses that function without blocking the whole event loop. Rejections must be caught at the right boundary.

Câu 78. What does `Promise.all()` do when one promise rejects?

Answer (English): `Promise.all` resolves when all succeed and rejects as soon as any rejects; it does not automatically cancel the other operations. Use it only when aggregate success is required and concurrency is controlled.

Câu 79. When would you use `Promise.allSettled()` instead?

Answer (English): `Promise.allSettled` reports fulfilled and rejected results for every task, useful for a batch sync where each product can independently fail. Record failed items and retry only safe operations.

Câu 80. What are the risks of using `as any` to fix a compilation error?

Answer (English): `as any` bypasses checks and can conceal missing/null data, invalid domain transitions, and runtime crashes. Fix the inferred type or validate inputs; document a rare assertion with a clear invariant.

Câu 81. How do you handle errors in asynchronous TypeScript functions?

Answer (English): Catch at service boundaries, distinguish expected domain errors from unexpected exceptions, attach context/correlation IDs, and never swallow failures silently. For

retries, classify transient versus permanent errors and preserve idempotency.

Câu 82. What are structural typing and nominal typing?

Answer (English): TypeScript uses structural compatibility: matching shape usually suffices regardless of declared name. Nominal typing distinguishes by declaration identity; branded types can approximate it for `SkuId` versus `OrderId`.

Câu 83. How would you design a strongly typed event system?

Answer (English): Define an `EventMap` mapping event names to payload types and a generic `publish<K extends keyof EventMap>(type: K, payload: EventMap[K])`. Validate external payloads and handle consumer idempotency separately.

Câu 84. How would you type a function that accepts different inventory event types?

Answer (English): A discriminated union of event variants allows `handle(event: InventoryEvent)` followed by an exhaustive `switch(event.type)`. Each case knows which payload fields are required and yields the correct inventory delta.

Câu 85. What does the `satisfies` operator do?

Answer (English): `satisfies` checks that a value conforms to a target type without discarding the value's useful inferred literal information. It is helpful for typed configuration and complete event-handler maps.

Câu 86. What are conditional types and mapped types?

Answer (English): Conditional types express type-level branches such as `T extends U ? X : Y`; mapped types transform properties across a key set. Use them to reduce duplication but avoid unreadable abstractions.

Câu 87. What are declaration files and ambient types?

Answer (English): `.d.ts` files declare types for JavaScript or external globals without providing runtime implementations. Ambient declarations use `declare` to describe symbols supplied by the environment.

Câu 88. How do you share types between frontend and backend without tightly coupling the two?

Answer (English): Share versioned contracts or schemas in a small package, not ORM objects and backend implementation details. Generate DTO types from validated API contracts where practical, and keep frontend release compatibility in mind.

Câu 89. How would you prevent unsafe assumptions when working with data from an external API?

Answer (English): Parse responses as `unknown`, validate required identifiers and versions, handle null/missing fields, apply rate limits, and treat external data as eventually consistent. Log discrepancies rather than casting a malformed response into trusted types.

Câu 90. How do you configure TypeScript for a production application?

Answer (English): Use `strict: true`, consistent ESM/module resolution for the target runtime, `noEmit` in type-check pipelines, and settings such as `noUncheckedIndexedAccess` where feasible. Combine compilation with linting and runtime schemas.

Câu 91. Define types for `Product`, `SKU`, `Warehouse`, `Bin`, `Order`, and `InventoryEvent`.

Answer (English): `Product` has stable product metadata; `SKU` points to a product variant; `Warehouse` owns `Bin` records; `Order` has line items; `InventoryEvent` stores SKU, bin, signed delta, type, time, actor, and idempotency key. Concrete, compilable declarations appear in the code appendix.

Câu 92. Create a discriminated union for `RECEIVED`, `PICKED`, `RETURNED`, and `ADJUSTED` inventory events.

Answer (English): Define `RECEIVED` and `RETURNED` with positive quantities, `PICKED` with positive requested units but negative derived delta, and `ADJUSTED` with signed delta plus reason. A discriminated union forces appropriate payloads per variant.

Câu 93. Write a function that calculates the inventory change represented by an event.

Answer (English): Use an exhaustive `switch`: `RECEIVED` / `RETURNED` return `+quantity`, `PICKED` returns `-quantity`, and `ADJUSTED` returns `delta`. Validate integers and reasons before calling it; the code appendix includes implementation.

Câu 94. Define a typed `Result<T, E>` structure for success and failure.

Answer (English): `type Result<T,E> = {ok:true; value:T} | {ok:false; error:E}`. The `ok` discriminator prevents accessing `value` in failure cases and supports explicit expected-error handling.

Câu 95. Implement a function that validates a Shopify webhook payload before processing it.

Answer (English): Accept the raw HTTP bytes, calculate the Shopify HMAC with the webhook secret, compare signatures in constant time, parse and validate the payload, then deduplicate on the event ID. Return success only after durably storing the event.

Câu 96. Refactor a function containing multiple `any` types into properly typed TypeScript.

Answer (English): Replace each `any` at the boundary with `unknown`, parse into an explicit runtime schema, and use a typed discriminated event union internally. Revisit property assumptions instead of replacing `any` with unjustified casts.

Câu 97. Design a typed API response that supports pagination.

Answer (English): A useful response is `{items:T[]; nextCursor:string|null; hasMore:boolean}` plus a stable sorting policy. With cursor pagination, use a deterministic `(created_at,id)` key to avoid duplicate/missing rows under inserts.

Câu 98. Model order states so that invalid transitions can be detected.

Answer (English): Represent state as `new | reserved | picking | packed | shipped | cancelled` and encode allowed transitions centrally. Reject impossible transitions under a transaction, checking race conditions and cancellation/partial fulfillment rules.

PHẦN 4. JAVASCRIPT VÀ ASYNCHRONOUS PROGRAMMING

Câu 99. Explain how the JavaScript event loop works.

Answer (English): The JavaScript event loop coordinates execution of synchronous callbacks and asynchronous completions. After the current stack empties, queued microtasks run before the next task; slow synchronous code delays all requests on that thread.

Câu 100. What is the difference between the call stack, task queue, and microtask queue?

Answer (English): The call stack runs JavaScript now; task queues hold callbacks from timers/I/O; the microtask queue holds promise reactions and similar work. The runtime drains microtasks at checkpoints, so excessive microtasks can starve tasks.

Câu 101. What is the difference between promises and callbacks?

Answer (English): Callbacks are functions invoked on completion; promises model one eventual result and enable chaining and `async/await`. Promises simplify error propagation but still need cancellation, timeouts, and concurrency limits.

Câu 102. What are closures, and where might they cause bugs?

Answer (English): A closure retains access to variables in its lexical environment. Bugs arise when async callbacks reference changing state or retain large objects, so snapshot values and clean up listeners when appropriate.

Câu 103. How does the `this` keyword behave in different contexts?

Answer (English): `this` depends on how a regular function is called; arrow functions capture lexical `this`. Losing the method receiver via destructuring/callbacks can make `this` undefined in strict mode.

Câu 104. What is the difference between shallow copying and deep copying?

Answer (English): A shallow copy duplicates only the outer container and shares nested references; a deep copy copies nested structures according to a defined policy. `structuredClone` supports many built-in types but not arbitrary functions and prototypes.

Câu 105. How do object references affect state and data mutation?

Answer (English): Objects are referenced, so mutating a nested value affects every reference to that object. In React, create new state objects/arrays instead of mutating existing ones; isolate shared mutable data in services.

Câu 106. What is the difference between `Map`, `Set`, arrays, and plain objects?

Answer (English): Arrays are ordered sequences; `Set` stores unique values; `Map` supports arbitrary key types; plain objects are convenient string/symbol-keyed records. Choose based on lookup, order, identity, and iteration needs.

Câu 107. What is the difference between synchronous and asynchronous execution?

Answer (English): Synchronous code finishes before the caller continues; asynchronous operations return control while pending work completes later. In Node.js, async I/O scales, but CPU-bound loops still block the event loop.

Câu 108. What problems can occur when multiple asynchronous requests modify the same data?

Answer (English): Two async operations can read the same balance and both write based on stale values, producing a lost update or overselling. Use database transactions with appropriate locks/constraints; a JavaScript mutex does not protect multiple service instances.

Câu 109. What is a race condition?

Answer (English): A race condition means the outcome depends on unpredictable ordering of concurrent actions. For stock allocation, concurrent checks without serialization may both approve more units than are available.

Câu 110. How would you limit the number of concurrent API requests?

Answer (English): Use a bounded worker pool or semaphore and obey service-specific rate limits. Queue excess jobs and enforce retry delays; creating 10,000 promises at once can overwhelm sockets, memory, or an API.

Câu 111. How would you implement retry logic with exponential backoff?

Answer (English): Use exponential backoff with jitter, a maximum attempt count, timeouts, and logging. Retry only transient failures; ensure requests with side effects carry idempotency keys so retries cannot multiply inventory movements.

Câu 112. Why can retrying a failed request sometimes be dangerous?

Answer (English): If the server completed a write but its response was lost, a retry may execute the write twice. Persist an idempotency key with the transaction and return the original result for repeated keys.

Câu 113. How would you implement a timeout for an external API call?

Answer (English): Create an `AbortController`, schedule a timeout that calls `abort()`, and pass its signal to `fetch`; always clear the timer. Treat timeouts as uncertain outcomes for writes and retry only through idempotent operations.

Câu 114. When would you use `AbortController`?

Answer (English): Use `AbortController` to cancel obsolete searches, enforce request deadlines, or stop jobs during shutdown. Cancellation of a client request does not guarantee a remote write was undone.

Câu 115. How do you prevent an unhandled promise rejection from crashing or destabilizing a service?

Answer (English): Await or `.catch()` every promise at an owned boundary, use centralized task-supervision, and log failures with context. For genuinely unexpected process-level errors, stop accepting work and restart cleanly rather than continue from an unknown state.

Câu 116. What is the difference between CPU-bound and I/O-bound work in Node.js?

Answer (English): CPU-bound work spends time computing and blocks JavaScript; I/O-bound work waits on databases, networks, or storage. Node.js handles I/O efficiently; send heavy computation to worker threads or separate workers.

Câu 117. How would you investigate a Node.js application that becomes slow under load?

Answer (English): Inspect event-loop lag, CPU, heap growth, connection pools, slow SQL, upstream latency, and p95/p99 traces. Reproduce representative load and change one bottleneck at a time rather than blindly increasing instances.

Câu 118. What is backpressure, and why does it matter when processing large amounts of data?

Answer (English): Backpressure prevents a producer from overwhelming a slower consumer. Use streams, bounded queues, and concurrency controls so importing a large catalog does not exhaust memory or flood PostgreSQL.

PHẦN 5. POSTGRESQL, SQL VÀ DATABASE DESIGN — ƯU TIÊN CAO NHẤT

Câu 119. [ƯU TIÊN CAO] What is a relational database, and why would you use PostgreSQL for an inventory system?

Answer (English): A relational database stores structured entities and relationships protected by keys and constraints. PostgreSQL is suitable for inventory because atomic transactions, isolation, joins, indexes, and durable audit records are essential.

Câu 120. What is the difference between a database, a schema, a table, and a view?

Answer (English): A database is an isolated collection; a schema is a namespace within it; a table stores rows; a view defines a query-backed virtual relation. A materialized view stores a refreshable query result.

Câu 121. What is a primary key?

Answer (English): A primary key uniquely identifies a row and cannot be null. Choose stable identifiers so foreign keys and historical events remain valid even if human-readable SKU codes change.

Câu 122. What is a foreign key?

Answer (English): A foreign key ensures a referenced value exists in its parent table, subject to declared actions. For example, inventory events should reference valid SKUs and bins.

Câu 123. What is a unique constraint?

Answer (English): A unique constraint disallows duplicate non-null combinations, with PostgreSQL's default handling of nulls; use `NULLS NOT DISTINCT` when needed. A `(tenant_id,sku_code)` constraint prevents duplicate codes within one company.

Câu 124. What is a check constraint?

Answer (English): A `CHECK` constraint validates a row expression, such as `quantity > 0` for a receiving line. Cross-row invariants typically require transactions/other mechanisms, not a simple check.

Câu 125. What is the difference between `NOT NULL` and a `CHECK` constraint?

Answer (English): `NOT NULL` forbids null values directly; a `CHECK (x > 0)` validates allowed non-null values but normally passes for null because the result is unknown. Combine both where appropriate.

Câu 126. What is the difference between `DELETE`, `TRUNCATE`, and `DROP`?

Answer (English): `DELETE` removes selected rows and fires row-level delete behavior; `TRUNCATE` efficiently removes all rows with different locking/trigger semantics; `DROP` removes the object definition. Audit ledgers should not allow these casual operations.

Câu 127. What is the difference between `WHERE` and `HAVING`?

Answer (English): `WHERE` filters rows before aggregation; `HAVING` filters grouped aggregate results. Example: `GROUP BY sku_id HAVING SUM(delta) < 0` finds negative balances.

Câu 128. What is the difference between `INNER JOIN`, `LEFT JOIN`, `RIGHT JOIN`, and `FULL OUTER JOIN`?

Answer (English): `INNER JOIN` keeps matching rows; `LEFT` retains all left rows; `RIGHT` retains all right rows; `FULL OUTER` retains unmatched rows from both sides. `LEFT JOIN` is useful to find products without sales.

Câu 129. What is the difference between `UNION` and `UNION ALL`?

Answer (English): `UNION` combines result sets and removes duplicates; `UNION ALL` retains duplicates and normally costs less. Use `UNION ALL` unless duplicate elimination is a business requirement.

Câu 130. What is the difference between `COUNT(*)` and `COUNT(column)`?

Answer (English): `COUNT(*)` counts all input rows; `COUNT(column)` counts only rows with a non-null value in that column. Neither counts distinct values unless `DISTINCT` is used.

Câu 131. How do `GROUP BY` and aggregate functions work?

Answer (English): `GROUP BY` partitions matching rows by selected keys, and aggregate functions such as `SUM` or `COUNT` compute values per group. Always check join cardinality to avoid accidentally multiplying quantities.

Câu 132. How do you find duplicate records in a table?

Answer (English): `SELECT tenant_id,sku_code,COUNT(*) FROM skus GROUP BY tenant_id,sku_code HAVING COUNT(*)>1;` finds duplicate natural keys. Inspect nulls and normalization/case rules before enforcing a unique index.

Câu 133. How would you find products that have never been sold?

Answer (English): Use `NOT EXISTS (SELECT 1 FROM order_items oi WHERE oi.sku_id = s.id AND oi.status='sold')` against SKUs. Define what counts as sold, including cancellations and returns, before writing the query.

Câu 134. How would you find orders that contain no order items?

Answer (English): Use `SELECT o.id FROM orders o WHERE NOT EXISTS (SELECT 1 FROM order_items i WHERE i.order_id=o.id);` . It is robust even if unrelated joins create duplicates.

Câu 135. How do you handle `NULL` values in SQL?

Answer (English): Use `IS NULL` , `IS NOT NULL` , `COALESCE` , and `NULLIF` intentionally. Remember that most comparisons with null evaluate to unknown, so missing values require explicit handling.

Câu 136. Why does `column = NULL` not work as expected?

Answer (English): `NULL` means unknown or absent, so `column = NULL` evaluates to unknown, not true. Use `column IS NULL` ; `IS NOT DISTINCT FROM` compares values with null-aware semantics.

Câu 137. What is the difference between a CTE and a subquery?

Answer (English): A CTE names a subquery with `WITH` , improving composition and sometimes reuse; a subquery is nested directly. PostgreSQL may inline non-materialized CTEs, so use `EXPLAIN` instead of assuming an optimization fence.

Câu 138. What is a recursive CTE, and when might it be useful?

Answer (English): A recursive CTE repeatedly expands a base result until no new rows remain, useful for hierarchical bin locations or product categories. Include cycle prevention and sensible limits.

Câu 139. What are window functions?

Answer (English): Window functions calculate results over partitions while preserving individual rows. For example `SUM(delta) OVER (PARTITION BY sku_id,bin_id ORDER BY sequence)` yields running inventory balance.

Câu 140. What is the difference between `ROW_NUMBER()`, `RANK()`, and `DENSE_RANK()`?

Answer (English): `ROW_NUMBER` assigns unique sequence numbers even on ties; `RANK` gives tied rows the same rank with gaps; `DENSE_RANK` gives ties the same rank without gaps.

Câu 141. How would you calculate the latest inventory event for every SKU?

Answer (English): Use `ROW_NUMBER() OVER (PARTITION BY sku_id ORDER BY occurred_at DESC,id DESC)` and select row 1, or PostgreSQL `DISTINCT ON (sku_id) ... ORDER BY sku_id,occurred_at DESC,id DESC`.

Câu 142. How would you calculate the total number of items received and picked for each product?

Answer (English): Use conditional sums: `SUM(CASE WHEN type='RECEIVED' THEN quantity ELSE 0 END)` and similarly for picked events. Group by SKU or join SKU to product to obtain product-level totals.

Câu 143. How would you find warehouse bins whose physical count differs from the system balance?

Answer (English): Join physical count sessions to a ledger-derived balance grouped by `(sku_id,bin_id)`, then filter `counted_qty IS DISTINCT FROM balance_qty`. Compare the same cutoff time and inventory status to avoid false mismatches.

Câu 144. When should data be normalized?

Answer (English): Normalize when separate concepts have different lifecycles and shared data would be duplicated, causing inconsistent updates. Products, variants, warehouses, bins, and order lines should have explicit keys and relationships.

Câu 145. What are the trade-offs of denormalization?

Answer (English): Denormalization can speed reads but increases write complexity and risk of stale or inconsistent data. Treat derived inventory projections as rebuildable caches, never as a second independent ledger.

Câu 146. What is a composite primary key?

Answer (English): A composite primary key uses multiple columns together, such as `(order_id,line_no)`. It encodes uniqueness within a parent, although a surrogate ID may ease references.

Câu 147. When would you use a surrogate key instead of a natural key?

Answer (English): A surrogate key stays stable when natural business identifiers can change or are reused. Keep natural-key uniqueness rules separately, such as tenant-scoped SKU codes.

Câu 148. What is the difference between `UUID`, `BIGINT`, and an auto-incrementing integer as identifiers?

Answer (English): `BIGINT` is compact and index-friendly but may expose sequential IDs; UUIDs support decentralized creation and hide sequence patterns but occupy more space. `GENERATED ... AS IDENTITY` creates database-managed integers.

Câu 149. How would you model product variants and SKU codes?

Answer (English): Model product → variant → SKU, with unique SKU code within a defined scope and optional attributes such as size/color. External listing IDs belong in mapping tables, not as replacement primary keys.

Câu 150. How would you enforce that a SKU is unique within a company or catalog?

Answer (English): Create `UNIQUE (company_id, sku_code)` after normalizing canonical SKU code rules. Consider a functional index on `lower(sku_code)` for case-insensitive uniqueness if business semantics require it.

Câu 151. How do you represent money and quantities accurately in PostgreSQL?

Answer (English): Use `numeric(p,s)` or integer minor currency units for money, paired with currency code. Use integer quantity for indivisible units or `numeric` with controlled precision for measured inventory; enforce valid scales.

Câu 152. Why should floating-point types generally be avoided for exact monetary values?

Answer (English): Binary floating-point values cannot represent many decimal fractions exactly and can produce rounding errors. Use `numeric` or integer cents, plus explicit rounding policy.

Câu 153. How would you represent timestamps and time zones?

Answer (English): Store instants as `timestamptz` and display in the user's local time zone; store an IANA zone if recurring local schedules depend on it. Preserve `occurred_at` and `recorded_at` for auditability.

Câu 154. What is the difference between `timestamp` and `timestamptz`?

Answer (English): `timestamp without time zone` stores a date-time with no absolute timezone interpretation; `timestamptz` represents an instant and converts presentation according to session zone. Prefer `timestamptz` for events.

Câu 155. How would you prevent invalid negative quantities where negative values are not permitted?

Answer (English): Use `CHECK (qty >= 0)` on directly stored nonnegative counts; for event-derived availability enforce nonnegative results inside serialized allocation transactions. A constraint on event delta alone cannot protect the aggregate.

Câu 156. What is referential integrity, and why does it matter?

Answer (English): Referential integrity means references point to valid records. It prevents orphaned order lines, invalid bin movements, and unexplained ledger events.

Câu 157. What are `ON DELETE CASCADE`, `RESTRICT`, and `SET NULL`?

Answer (English): `CASCADE` automatically deletes referencing rows; `RESTRICT` blocks parent deletion while dependents exist; `SET NULL` retains children without the reference when allowed. Choose by business lifecycle.

Câu 158. Why can cascading deletes be dangerous in an audit-sensitive inventory system?

Answer (English): Cascading from a product to event history could erase the audit trail and make inventory irreproducible. Prefer archival/soft deletion, restricted deletes, and immutable historical references.

Câu 159. [ƯU TIÊN CAO] How would you design a relational schema for a product and warehouse management system?

Answer (English): Use tenant, product, variant/SKU, warehouse, bin, orders/order lines, receipts, pick tasks, shipments, returns, and immutable inventory events. Constrain keys; relate each event

to SKU, location, actor, source action, and idempotency key.

Câu 160. What entities would you create for products, product variants, SKUs, warehouses, and bins?

Answer (English): `products` stores shared description, `variants` stores option combinations, `skus` stores stock-keeping identifiers, `warehouses` are facilities, and `bins` reference warehouses. A SKU has movements in many bins through event rows.

Câu 161. What is the difference between a product and a product variant?

Answer (English): A product is a merchandising concept; a variant is a purchasable configuration such as a size/color. One product can have multiple variants, each potentially mapped to its own inventory-tracked SKU.

Câu 162. Would you store SKU, product, and warehouse data in one table? Why or why not?

Answer (English): No: combined rows would duplicate names, make multi-bin storage awkward, and blur integrity constraints. Normalize stable entities, then join or project data for screens that need a consolidated view.

Câu 163. How would you model orders and order line items?

Answer (English): Use `orders(id,channel,status,...)` and `order_items(id,order_id,sku_id,ordered_qty,...)`, with fulfillment/reservation records separately. Preserve a snapshot of sale-time details such as price and displayed title.

Câu 164. How would you model goods receiving?

Answer (English): Create receiving headers and lines against purchase orders/shipments, scans with operator and timestamps, then append stock events only for accepted quantities. Keep pending-inspection inventory in a distinct status/location.

Câu 165. How would you model picking and packing operations?

Answer (English): Represent pick tasks, allocations, scans, pick confirmations, packing sessions, and shipments as separate lifecycles. Link inventory movement events to the exact pick/pack action and enforce valid transitions.

Câu 166. How would you model returns and returned product conditions?

Answer (English): Track return authorization, line items, received quantity, inspection result, condition, disposition, and linked original order. Only resellable returns create available-stock additions; damaged goods enter separate status.

Câu 167. How would you represent a product stored in multiple warehouse bins?

Answer (English): Store locations independently and record `(sku_id,bin_id)` on movements. A grouped balance or maintained projection gives current units in every bin; there is no one-bin-per-SKU assumption.

Câu 168. How would you model inventory transfers between bins?

Answer (English): Write two movements under one `transfer_id`: a negative delta at source and positive delta at destination, committed atomically. Reserve/lock the source availability and validate both bins and ownership.

Câu 169. How would you represent a product listing on Shopify versus the internal product record?

Answer (English): The internal product/variant is authoritative for operational identity; `channel_listings(channel,shop_id,external_product_id,external_variant_id,sku_id,...)` stores mappings and synchronization status.

Câu 170. How would you handle different identifiers for the same product across Shopify, eBay, TikTok, and other marketplaces?

Answer (English): Use a marketplace identity table keyed by `(platform,account_id,external_variant_id)` pointing to an internal SKU. Do not assume an external ID or SKU code is globally unique across accounts.

Câu 171. How would you prevent duplicate product records during data imports?

Answer (English): Stage imports, normalize keys, use scoped unique constraints and `INSERT ... ON CONFLICT ... DO UPDATE` with deliberate field ownership. Record ambiguous matches for review instead of silently merging products.

Câu 172. What is a many-to-many relationship, and where might you need one in this system?

Answer (English): A many-to-many relation uses a junction table, e.g. products sold in many channels and channels offering many products. Warehouse users may also be assigned to several warehouses.

Câu 173. How would you represent relationships between parent products and variants?

Answer (English): Use `variants.product_id REFERENCES products(id)`, with variant option attributes modeled consistently. If kits/bundles are supported, use a separate component relation rather than forcing everything into parent/child variants.

Câu 174. How would you store a product's history without overwriting historical information?

Answer (English): Preserve append-only change events or versioned snapshots for business-significant history. Stable foreign keys should still resolve historical orders after a product's current description changes.

Câu 175. What is a database constraint, and why should business rules be enforced in the database as well as the application?

Answer (English): Constraints are database-enforced statements of valid rows/relationships; they protect integrity even when multiple services, scripts, or internal tools write data. Pair them with higher-level application validation.

Câu 176. Which rules belong in the database, which belong in the backend, and which belong in the frontend?

Answer (English): Database: key uniqueness, referential integrity, atomic invariants; backend: permissions, workflows, state transitions, external effects; frontend: guidance and early validation. Critical rules must not depend solely on UI behavior.

Câu 177. How would you model a product that can be sold on multiple marketplaces but has one internal source of truth?

Answer (English): Maintain canonical internal entities and separate channel-specific listing/mapping rows with ownership and sync metadata. Each marketplace may have different titles/prices, while stock movement has one internal ledger.

Câu 178. How would you design a schema that can evolve as the business adds new warehouse processes?

Answer (English): Separate core invariant tables from optional process modules, use stable foreign keys and versioned event types, and release schema changes incrementally. Avoid a single overloaded status column representing unrelated lifecycles.

Câu 179. How would you decide whether to add a new column or create a separate table?

Answer (English): Add a column for a single-valued attribute of the same entity; create a table for repeatable values, distinct lifecycle, many-to-many links, or data needing separate permissions/history.

Câu 180. When is JSONB appropriate in PostgreSQL, and when should you use normalized columns instead?

Answer (English): JSONB is good for irregular external metadata rarely joined or constrained; normalized columns are better for frequently filtered, validated, or related business fields. Add expression indexes for frequently accessed JSONB fields if justified.

Câu 181. How would you store marketplace-specific metadata without making the entire schema difficult to maintain?

Answer (English): Store marketplace-specific raw metadata in `channel_listings.metadata jsonb` while promoting critical shared identifiers and statuses into typed columns. Keep version/schema validation for imported payloads.

Câu 182. How would you represent soft deletion and archival?

Answer (English): Use `archived_at` or `deleted_at` where historical references must remain, and restrict writes to archived entities. Queries and unique constraints should define whether archived identifiers can be reused.

Câu 183. How would you preserve historical product information after a product is discontinued?

Answer (English): Mark the product discontinued but preserve the record, SKU identity, historical event references, and order snapshots. Block new catalog listings/picks according to business rules rather than deleting history.

Câu 184. How would you handle a SKU being renamed without losing its transaction history?

Answer (English): Keep immutable `sku_id` as the relational identity; update a code only after collision checks, and record the previous code in an alias/history table. Events should reference the ID, not a mutable text code.

Câu 185. How would you document a database schema with approximately 30 tables?

Answer (English): Document an ER diagram, table ownership, key constraints, status lifecycles, ledger invariants, index/query rationale, and migration rules. Keep documentation reviewed alongside schema migrations.

Câu 186. [ƯU TIÊN CAO] What is a database index, and how does it improve query performance?

Answer (English): An index is an auxiliary lookup structure that can avoid scanning entire tables for selective filters/sorts and joins. Its benefit depends on query shape, cardinality, and maintenance costs.

Câu 187. What are the disadvantages of having too many indexes?

Answer (English): Indexes consume storage, slow inserts/updates/deletes, and increase vacuum and write amplification. Use query plans and workload evidence before adding overlapping indexes to a high-write ledger.

Câu 188. What is a composite index?

Answer (English): A composite index indexes multiple fields together, e.g. `(sku_id,bin_id,occurred_at DESC)`. It suits queries that filter by the leading keys and sort by a later key.

Câu 189. How does column order affect a composite index?

Answer (English): B-tree index access typically follows leading-column prefix rules; `(sku_id,bin_id)` helps queries by SKU and by SKU+bin, but not usually queries only by bin as efficiently. Check actual plans.

Câu 190. When would you create a partial index?

Answer (English): A partial index indexes only rows satisfying a predicate, such as `WHERE status='pending'`. It is valuable when queries repeatedly target a small subset and the predicate matches.

Câu 191. What is a unique index?

Answer (English): A unique index enforces uniqueness in its indexed key/expression across relevant rows. It underlies many unique constraints and can protect idempotency keys or external ID mappings.

Câu 192. What is a GIN index, and when might you use it with JSONB?

Answer (English): A GIN index can accelerate containment and existence lookups over JSONB, such as `metadata @> '{"source":"shopify"}'`. It costs writes/storage and should not replace normalized keys.

Câu 193. What is the difference between B-tree and GIN indexes?

Answer (English): B-tree works well for equality, ranges, and ordering on scalar keys; GIN indexes membership within multi-valued structures such as JSONB arrays/documents. Choose by operators actually used.

Câu 194. What does `EXPLAIN` show?

Answer (English): `EXPLAIN` shows the optimizer's estimated execution plan, scans, joins, row counts, and costs without running the query. It helps identify likely expensive operations.

Câu 195. What is the purpose of `EXPLAIN ANALYZE` ?

Answer (English): `EXPLAIN ANALYZE` executes the query and reports actual timing, rows, and loop counts. On production, beware that modifying queries really run; use safe contexts and `BUFFERS` when useful.

Câu 196. How would you investigate a slow query in a production database?

Answer (English): Capture query and parameters safely, measure with `pg_stat_statements` , inspect `EXPLAIN (ANALYZE,BUFFERS)` in a safe environment, check row estimates/indexes/locks, and validate an improvement under similar load.

Câu 197. How would you optimize a query that calculates inventory balances across millions of events?

Answer (English): Index common SKU/bin/time predicates, partition if volume justifies it, and maintain rebuildable incremental balance projections. Do not derive every dashboard cell by replaying millions of events on each request.

Câu 198. How would you determine whether a query is slow because of missing indexes or an inefficient join?

Answer (English): Inspect actual vs estimated row counts, sequential scans, join strategies, filtered rows, buffer hits, and time by plan node. Missing indexes and excessive fan-out produce different evidence.

Câu 199. What is the N+1 query problem?

Answer (English): N+1 happens when fetching N products triggers N extra queries for variants. Batch using joins/preloads or `WHERE product_id IN (...)` and confirm query counts in integration tests.

Câu 200. How would you prevent N+1 queries when loading products and their variants?

Answer (English): Load products plus variants in one correctly bounded join or two batched queries, then group by product. Handle pagination before joining one-to-many data to avoid incorrect page sizes.

Câu 201. How do offset pagination and cursor pagination differ?

Answer (English): Offset pagination uses `LIMIT ... OFFSET ...`; cursor/keyset pagination resumes after a stable ordered key. Cursor pagination scales better and reduces drift under frequent inserts.

Câu 202. Why can offset pagination become slow on large tables?

Answer (English): A large OFFSET forces the database to scan/skip many earlier rows, and concurrent changes can shift results. Keyset pagination filters by the last `(created_at,id)` key.

Câu 203. How would you implement pagination for a large product catalog?

Answer (English): Use indexed filters and deterministic sort (e.g. name plus ID), bounded page sizes, and cursor pagination for large catalogs. Separate total-count queries if exact counts are expensive.

Câu 204. When should you use a materialized view?

Answer (English): A materialized view is useful for expensive, read-heavy summaries where staleness is acceptable and refresh cadence is defined. It is inappropriate as the sole authority for allocation decisions.

Câu 205. How is a materialized view different from a normal view?

Answer (English): A normal view evaluates its query when read; a materialized view stores results until refreshed. Materialized results require explicit refresh and can be temporarily stale.

Câu 206. What happens when the underlying data changes after a materialized view is refreshed?

Answer (English): The materialized view continues showing its last refreshed snapshot until the next refresh, even if base tables have changed. Display refresh time when the freshness matters.

Câu 207. What are the trade-offs between real-time aggregation and precomputed aggregates?

Answer (English): Real-time aggregation is current but potentially expensive; precomputed projections are fast but require update/rebuild/reconciliation logic. The immutable ledger remains authoritative regardless of projection strategy.

Câu 208. How would you keep inventory reporting fast without making a cached balance the authoritative source of truth?

Answer (English): Maintain a projection transactionally or asynchronously for fast reads, tag its freshness, and reconcile periodically against ledger recomputation. Stock-critical reservations use a serialized authoritative check, not a stale dashboard cache.

Câu 209. How would you determine whether a query optimization actually improved performance?

Answer (English): Compare representative p50/p95 latency, query plans, buffer usage, CPU/IO, and write impact before/after under comparable datasets. Check result equality and concurrency behavior as well as speed.

Câu 210. What database metrics would you monitor in production?

Answer (English): Monitor query latency, slow statements, connections/pool waits, lock waits, deadlocks, transaction duration, cache hit rate, replication lag if used, table/index size, vacuum health, and disk usage.

Câu 211. [ƯU TIÊN CAO] What are the ACID properties of a database transaction?

Answer (English): Atomicity means all-or-nothing; consistency preserves defined invariants; isolation controls concurrent transaction visibility; durability retains committed changes. ACID makes inventory movements dependable but still requires correct domain design.

Câu 212. What is a transaction, and when should you use one?

Answer (English): A transaction groups operations that must succeed together, e.g. validating stock, inserting a transfer pair, and recording idempotency. Use it around the smallest set of database operations requiring atomicity.

Câu 213. What happens if one query in a transaction fails?

Answer (English): In PostgreSQL, an error normally marks the transaction aborted until rollback, unless a savepoint is used. Uncommitted changes do not become durable.

Câu 214. Why must a transaction either commit completely or roll back?

Answer (English): Partial commits could subtract stock without adding it at the destination or reserve stock without creating an order. A single commit preserves the business invariant or leaves the previous state.

Câu 215. What is the difference between a transaction and a database connection?

Answer (English): A connection is a session to the database; a transaction is an atomic unit of work conducted on a connection. One pooled connection can run many transactions over time, but a transaction must stay on one session.

Câu 216. What is transaction isolation?

Answer (English): Isolation defines how concurrent transactions observe one another and which anomalies are possible. Inventory writes require an isolation/locking design that prevents write skew and lost updates.

Câu 217. Explain the isolation levels supported by PostgreSQL.

Answer (English): PostgreSQL supports Read Committed (default), Repeatable Read (snapshot isolation), and Serializable; its Read Uncommitted behaves like Read Committed. Stronger isolation can require handling serialization failures and retries.

Câu 218. What is a dirty read?

Answer (English): A dirty read sees another transaction's uncommitted changes that may roll back. PostgreSQL prevents dirty reads even at its lowest effective isolation level.

Câu 219. What is a non-repeatable read?

Answer (English): A non-repeatable read occurs when rereading a row within one transaction returns a different committed value. Read Committed can allow it; Repeatable Read prevents it.

Câu 220. What is a phantom read?

Answer (English): A phantom read means repeating a predicate query finds a different set of rows due to concurrent writes. PostgreSQL's Repeatable Read snapshot prevents phantoms in the SQL-standard sense, though other serialization anomalies remain possible.

Câu 221. What is a lost update?

Answer (English): A lost update occurs when one writer overwrites another's change based on stale data. Conditional updates, row locking, or Serializable transactions help prevent it.

Câu 222. How can concurrent transactions produce incorrect inventory results?

Answer (English): Two workers may both observe 5 units available and each allocate 4, recording total reservations of 8. Serialize by stock key, recheck under lock, commit event and reservation atomically.

Câu 223. What is pessimistic locking?

Answer (English): Pessimistic locking reserves database resources before a conflicting change, e.g. `SELECT ... FOR UPDATE`. It favors preventing conflicts but may increase waits and deadlocks.

Câu 224. What is optimistic concurrency control?

Answer (English): Optimistic concurrency uses a version/compare-and-swap check and rejects stale writes. It works well when conflicts are rare; hot inventory rows may be better handled by short row locks.

Câu 225. What does `SELECT ... FOR UPDATE` do?

Answer (English): `SELECT ... FOR UPDATE` obtains row-level locks on selected rows and blocks conflicting writers/lockers until the transaction ends. Lock a stable stock-key row rather than a nonexistent aggregate row.

Câu 226. When would you use `FOR UPDATE` rather than relying only on application-level checks?

Answer (English): Use `FOR UPDATE` when a read-then-write decision must not race across app instances, such as reserving stock or changing order state. Frontend checks and in-process locks are insufficient.

Câu 227. What is a deadlock?

Answer (English): A deadlock occurs when transactions wait cyclically on locks held by each other. PostgreSQL aborts one transaction to break the cycle; applications must be prepared to retry.

Câu 228. How does PostgreSQL detect deadlocks?

Answer (English): PostgreSQL detects lock wait cycles after its deadlock timeout and terminates one participant with a deadlock error. That participant's transaction must roll back before retry.

Câu 229. How would you reduce the risk of deadlocks?

Answer (English): Acquire locks in a consistent deterministic order, keep transactions short, avoid external API calls under locks, and index selectors to reduce lock scope. Retry occasional deadlocks with bounded jitter.

Câu 230. What should an application do after a deadlock or serialization failure?

Answer (English): Roll back the failed transaction and retry the entire idempotent business operation with bounded backoff. For serialization failure `40001` or deadlock `40P01`, don't retry only the final SQL statement.

Câu 231. Why should database transactions be kept reasonably short?

Answer (English): Long transactions hold locks and old MVCC snapshots, increase contention, and impede vacuum. Keep external networking and expensive computation outside the critical

transaction.

Câu 232. Why should you avoid making slow external API calls while holding database locks?

Answer (English): An API call may hang while database rows remain locked, blocking warehouse operations. Commit local state/outbox quickly, then call the external service asynchronously and reconcile on failure.

Câu 233. What is an advisory lock in PostgreSQL?

Answer (English): An advisory lock is an application-defined lock on an integer key, scoped to a session or transaction. It can serialize work for a logical SKU/bin key with no physical row to lock.

Câu 234. When would an advisory lock be useful for serializing inventory operations?

Answer (English): Use transaction-scoped advisory locks when locking a computed stock key that might have no row yet. Hash collisions and lock ordering must be considered, and all writers must follow the same convention.

Câu 235. How would you make a stock adjustment atomic?

Answer (English): Validate and authorize the command, acquire the relevant stock key lock, recompute availability, append a uniquely keyed adjustment event, and commit in one transaction. Return a conflict for invalid negative results.

Câu 236. How would you protect an operation that affects two warehouse bins?

Answer (English): Lock both source and destination keys in deterministic order, check location ownership and source stock, then insert paired movements with one `transfer_id` in one transaction.

Câu 237. How would you ensure a failed warehouse transfer does not leave inventory partially updated?

Answer (English): A transfer writes the two event rows atomically; if any validation or insert fails, the transaction rolls back both. External notifications follow through an outbox, not inside the stock transaction.

Câu 238. How would you implement transaction-safe business logic in an application with concurrent workers?

Answer (English): Centralize stock writes in a transaction-safe service/function; lock by SKU/bin, enforce idempotency and invariants, append the event, and commit. Workers retry serialization failures using the same business idempotency key.

Câu 239. [ƯU TIÊN CAO] What is a database migration?

Answer (English): A migration is a versioned change to database schema or controlled data state. Examples include adding columns, constraints, indexes, functions, or backfilling data.

Câu 240. Why should database changes be version-controlled?

Answer (English): Version control makes schema evolution reviewable, reproducible, testable, and traceable to application releases. It also prevents environments from diverging through undocumented manual edits.

Câu 241. How do you write a migration that adds a new required column to a large table safely?

Answer (English): Add the column nullable or with a safe default first, deploy code that writes it, backfill in bounded batches, validate, then enforce `NOT NULL` when safe. Check table size and PostgreSQL version-specific lock behavior.

Câu 242. Why can adding a constraint or index affect a production system?

Answer (English): Creating indexes, validating constraints, and rewriting tables can consume I/O or hold locks, blocking operational writes. Evaluate lock level, use `CREATE INDEX CONCURRENTLY` where appropriate, and observe production impact.

Câu 243. What is the difference between a reversible and an irreversible migration?

Answer (English): A reversible migration can safely undo its schema change without losing required information. Data deletion and irreversible transformations often need backups, forward fixes, or staged replacement instead of a naive down migration.

Câu 244. What is a migration rollback?

Answer (English): A rollback restores a previous schema or application-compatible state when safe. It must account for data written after the migration; rollback scripts can be more dangerous than the forward change.

Câu 245. Why is rolling back application code different from rolling back database data?

Answer (English): Reverting code does not undo rows or data transformations already committed. Thus old code must remain compatible with the intermediate schema, or you need an explicit recovery strategy.

Câu 246. How would you introduce a new inventory event type without breaking existing reporting?

Answer (English): Version event schemas, teach readers and reports how to interpret the new type, deploy compatible consumers first, and only then produce the new event. Recompute balance projections to ensure the new type is accounted for.

Câu 247. How would you rename a column while allowing the old application version to continue running temporarily?

Answer (English): Use expand-and-contract: add the new column, support reads/writes during the transition, backfill, switch readers, and remove the old column later. Don't rename a referenced column in one incompatible deployment.

Câu 248. What is an expand-and-contract migration strategy?

Answer (English): Expand the schema compatibly, migrate/backfill data, switch application behavior, then contract/remove obsolete fields after old versions are gone. It minimizes downtime and enables rollback of application code.

Câu 249. How would you test a migration locally before staging or production?

Answer (English): Run migration on a representative staging snapshot, check row counts/constraints and query plans, test both old/new application compatibility, measure duration/locks, and rehearse recovery.

Câu 250. What would you do if a migration failed halfway through deployment?

Answer (English): Check whether the migration was transactional and what has committed; stop further rollout and inspect the actual schema state. Repair using a reviewed migration rather than blindly rerunning unsafe statements.

Câu 251. How would you deploy a migration that changes a large table without creating excessive locks?

Answer (English): Use staged DDL, batched backfills with small transactions, lock timeouts, concurrent index builds when supported, and operational monitoring. Avoid a single large rewrite during high-volume warehouse operations.

Câu 252. How do you ensure two developers do not accidentally create conflicting migrations?

Answer (English): Keep a migration registry and CI that applies all migrations in order to a fresh database. Coordinate merges, review generated SQL, and resolve conflicting changes before deployment.

Câu 253. What is your process for backing up data before a risky database change?

Answer (English): Confirm a recent recoverable backup and preferably a point-in-time recovery path, rehearse restoration, estimate recovery objectives, and capture a narrow export of affected records. Never assume a backup is valid without testing.

Câu 254. How would you recover if an incorrect migration corrupted inventory records?

Answer (English): Stop affected writes, preserve evidence, identify the exact bad migration and impacted IDs, restore/replay into a safe environment, and prepare a scoped, audited correction. Prefer compensating events over modifying immutable ledger rows.

Câu 255. What should a safe production database change checklist contain?

Answer (English): Checklist: reviewed SQL, dependencies, lock impact, backward compatibility, backup/restore plan, staging rehearsal, bounded rollout, verification queries, owner, metrics, and rollback/forward-fix procedure.

PHẦN 6. EVENT-DRIVEN INVENTORY LEDGER — CHỦ ĐỀ ĐẶC BIỆT QUAN TRỌNG

Câu 256. [ƯU TIÊN CAO] What is an append-only event ledger?

Answer (English): An append-only event ledger stores each inventory-changing action as a new immutable record. Current balances are derived by aggregating signed events, preserving a reproducible history.

Câu 257. How is an event ledger different from a conventional inventory table?

Answer (English): A conventional table overwrites a mutable quantity; a ledger records every movement with actor, reason, time, and reference. A projection may cache current quantity but can always be rebuilt from events.

Câu 258. Why might a company derive inventory balances from events instead of storing a mutable quantity?

Answer (English): Event history supports auditing, reconstruction at past times, discrepancy investigation, and repair through compensating entries. The cost is more complex writes/projections and rigorous concurrency handling.

Câu 259. What is event sourcing?

Answer (English): Event sourcing stores state changes as a primary series of domain events and rebuilds state by replaying them. It requires event schema evolution, deterministic projection logic, and careful side-effect handling.

Câu 260. Is every append-only audit log an event-sourced system? Why or why not?

Answer (English): No. An audit log may document changes while the authoritative state remains a mutable table. Event sourcing treats the event stream as the source from which domain state is reconstructed.

Câu 261. What is the difference between an event ledger and an audit log?

Answer (English): A ledger is part of operational accounting and directly determines stock; an audit log documents who changed something. Both can be append-only, but only the ledger is the authoritative movement record.

Câu 262. What information should an inventory event contain?

Answer (English): Include immutable event ID, tenant, SKU, bin/location, signed delta, event type, business reference, actor, idempotency key, recorded time, effective time, and reason/metadata. Distinguish movement status when availability differs from physical count.

Câu 263. Which fields should every event have regardless of event type?

Answer (English): Every event needs ID, type/version, relevant entity IDs, signed effect, timestamps, provenance, and a unique request/source identifier. Some transfer-specific fields may belong in a typed payload or linked transfer record.

Câu 264. Why should an event have a unique identifier?

Answer (English): A unique ID allows reliable referencing, deduplication, audit, replay ordering, and linking compensating events. A separate unique business idempotency key prevents duplicate effects across retries.

Câu 265. Why should an event record include a timestamp?

Answer (English): Recorded time establishes when the system learned of an event; effective time says when the business movement occurred. Keeping both supports historical investigation and

late-arriving events.

Câu 266. How would you identify the user or service that generated an event?

Answer (English): Store the authenticated `actor_user_id` or service principal, plus process/source type and correlation ID. Do not trust a caller-supplied actor if it conflicts with authenticated identity.

Câu 267. How would you link an event to an order, shipment, return, or adjustment?

Answer (English): Use stable foreign keys or a typed `(source_type,source_id)` reference, plus an optional causal event ID. Domain-specific linkage should preserve the ability to trace a movement to its receipt, pick, return, or count.

Câu 268. Why is the SKU and warehouse location important when recording inventory events?

Answer (English): An inventory movement has meaning only for a particular stock identity and location; a SKU may exist in multiple bins and warehouses. Correct keys prevent transfers and counts from silently mixing locations.

Câu 269. Should an inventory event be updated after insertion?

Answer (English): No business movement should be updated or deleted after posting. Correct mistakes with explicit reversal/correction events, preserving the original and actor/reason.

Câu 270. If an event is incorrect, how would you correct it without deleting history?

Answer (English): Post a compensating event referencing the erroneous event, validate the resulting balance, and log the reason and approval. Never overwrite history to make the current balance look right.

Câu 271. What is a compensating event?

Answer (English): A compensating event reverses or offsets a prior incorrect business effect while preserving audit history. For an accidental `+5`, a linked `−5` correction cancels the numerical impact.

Câu 272. How would you represent a correction made after a physical inventory count?

Answer (English): Record the counted quantity and count cutoff, compute the book balance, require approval if needed, and append an `ADJUSTED` event for the difference. Preserve the count

session and evidence.

Câu 273. How would you distinguish a business event from a technical retry?

Answer (English): A business event represents a real completed operation with an immutable business ID; a retry is another delivery attempt of that same operation. Deduplicate retries using an idempotency key before writing a second ledger row.

Câu 274. How would you prevent duplicate inventory events?

Answer (English): Enforce `UNIQUE (source, idempotency_key)` or a business-operation ID; insert or claim the key in the same transaction as the movement. Concurrent duplicates should resolve to the existing result.

Câu 275. How would you ensure all inventory-changing operations create the required event records?

Answer (English): Centralize writes through restricted stored procedures/services, deny direct ledger mutation privileges, and enforce database constraints/triggers where appropriate. Audit periodically that every confirmed operation has its expected event(s).

Câu 276. [ƯU TIÊN CAO] How would you calculate the current balance for a SKU in a specific bin from an event ledger?

Answer (English): `SELECT COALESCE(SUM(delta),0) FROM inventory_events WHERE sku_id=$1 AND bin_id=$2;` gives a simple bin balance when all relevant movements use signed deltas. Include status/dimensions according to the domain model.

Câu 277. How would you calculate inventory across all bins in a warehouse?

Answer (English): Group by `(warehouse_id,sku_id)` after joining bins to warehouses, or sum all `(sku,bin)` balances within the facility. Ensure transfers net to zero within the warehouse.

Câu 278. How would you calculate inventory across multiple warehouses?

Answer (English): Aggregate per warehouse first, then sum by SKU across authorized warehouses. Distinguish stock that is reserved, quarantined, or in transit rather than treating it as sellable.

Câu 279. How would you calculate the total quantity received, picked, returned, and adjusted over a given period?

Answer (English): Group events by `type` and aggregate absolute quantity or signed delta for each period using business effective time. Report adjustments with signs and explain whether returns are counted as received.

Câu 280. How would you reconstruct the inventory balance at a historical point in time?

Answer (English): Replay events with an `occurred_at` /sequence cutoff and sum deltas, including corrections and version rules. Use stable ordering and distinguish historical knowledge from events recorded later but effective earlier.

Câu 281. How would you find the exact event that caused an incorrect inventory balance?

Answer (English): Inspect the SKU/bin event sequence, idempotency IDs, actor, source operations, and running balance around the discrepancy. Match against scans, receipts, shipment records, and physical count snapshots.

Câu 282. How would you determine the balance immediately before a particular event?

Answer (English): Use a running sum over ordered event sequence and subtract the current event's delta, or sum all prior events by a monotonic sequence. Define ordering tie-breakers explicitly.

Câu 283. How would you represent movement from one bin to another without changing total warehouse inventory?

Answer (English): Append `−qty` at source and `+qty` at destination under one transfer ID. The warehouse-wide sum stays unchanged; both writes occur in a single transaction.

Câu 284. Should a transfer be one event or two events? How would you ensure the two movements remain consistent?

Answer (English): Two signed location entries make per-bin balance straightforward; link them as one logical transfer and commit atomically. An alternative single transfer object needs projections that reliably produce both legs.

Câu 285. How would you distinguish available, reserved, damaged, and quarantined stock?

Answer (English): Track physical quantity and status/location buckets separately from reservations. `available = sellable_on_hand − reserved` under defined rules; damaged or quarantined stock must not be counted as available.

Câu 286. How would you represent inventory that has been received but not yet inspected?

Answer (English): Receive goods into an inspection/pending location or status, not directly into sellable stock. After QC passes, append a status/location transfer to make units available.

Câu 287. How would you handle returns that cannot be sold again?

Answer (English): Create a return receipt, record inspection and disposition, and move it into damaged/quarantine inventory or scrap it according to policy. Never add an unsellable item to available-to-promise quantity.

Câu 288. How would you handle partially picked orders?

Answer (English): Track reserved, picked, and remaining quantities per order line; decrement physical stock only for units actually picked or shipped per the chosen model. Preserve partial progress and prevent duplicate scans.

Câu 289. How would you represent an order reservation separately from a physical stock movement?

Answer (English): Reservations consume availability but do not necessarily remove physical on-hand units. Record reservation/release/consumption in a separate ledger or state machine linked to order lines, with atomic allocation checks.

Câu 290. How do you prevent a cancelled order from permanently reserving inventory?

Answer (English): A cancellation transaction releases only the outstanding reservation and records why. Use idempotency and transition checks to prevent repeated cancellation from releasing stock twice.

Câu 291. What happens if an order is partially fulfilled and then cancelled?

Answer (English): Keep already-picked/shipped quantity and reverse only unconsumed reservation; picked goods may require a restock workflow if physically returned. Do not erase fulfillment history.

Câu 292. How would you handle a product that is physically present but missing from the system?

Answer (English): Quarantine/identify the item, verify SKU and location, inspect receiving/transfer records, then add a documented count adjustment only with proper authorization. Avoid an unexplained direct quantity overwrite.

Câu 293. How would you handle a product recorded in the system but not found on the shelf?

Answer (English): Verify location and scan history, search alternate bins, check pending picks/shipments, and recount. If truly missing, create an approved negative adjustment linked to the count evidence.

Câu 294. How would you reproduce an inventory balance from the event history for debugging?

Answer (English): Extract events up to a fixed cutoff, order deterministically, replay their signed deltas, and compare with the projection. Retain event IDs and checksum/results for repeatable debugging.

Câu 295. How would you check that a computed balance is consistent with the physical count?

Answer (English): Use a frozen or clearly defined count cutoff, compare count to event-derived balance by SKU/bin/status, investigate differences, and record documented adjustments. Concurrent movements must be excluded or reconciled correctly.

Câu 296. [ƯU TIÊN CAO] Two workers try to pick the same inventory at the same time. How do you prevent overselling?

Answer (English): Lock an authoritative stock-key row or transaction advisory lock, re-evaluate available balance inside the transaction, then reserve/pick atomically. Reject the second request if remaining units are insufficient.

Câu 297. If five orders try to reserve the same SKU simultaneously, how should the system process them?

Answer (English): All five operations contend on the same SKU stock key and are serialized or retried under Serializable isolation. Each allocation succeeds only if the current committed availability still covers its requested quantity.

Câu 298. How would you prevent the same item from being allocated to two orders?

Answer (English): Keep an allocation record keyed by order line and stock unit/bin; protect its uniqueness plus the available-stock invariant in a transaction. The same physical unit cannot appear in two active allocations.

Câu 299. If inventory is derived from events, how do you safely validate available stock before appending a new event?

Answer (English): Acquire a stable lock for SKU/bin, compute ledger balance minus active reservations from authoritative data, verify enough availability, append reservation/event, and commit. A read alone without locking is unsafe.

Câu 300. If two transactions both calculate that sufficient stock exists, what could go wrong?

Answer (English): Both transactions can pass the same stock check on stale snapshots and each insert a valid-looking event, oversubscribing inventory. Serialize writes or use Serializable isolation with whole-transaction retries.

Câu 301. How would you serialize inventory-changing operations for the same SKU and bin?

Answer (English): Use a lock row for `(sku_id,bin_id)` with `SELECT ... FOR UPDATE`, or a collision-managed advisory lock. Keep the transaction short; all writes must use the same lock discipline.

Câu 302. When would you use row-level locking, and when might an advisory lock be appropriate?

Answer (English): Row-level locks are simplest when a stable stock aggregate/key row exists; advisory locks suit logical resources without such rows. Advisory locks require an agreed key scheme and disciplined usage.

Câu 303. How would you avoid locking every inventory row when only one SKU needs to be updated?

Answer (English): Lock only the specific SKU/bin or SKU/warehouse key implicated by the command, supported by selective indexes. Avoid global mutexes or table locks that serialize unrelated products.

Câu 304. How would you ensure an atomic stock transfer between two bins?

Answer (English): Lock both stock keys in sorted order, validate source balance, insert debit/credit event pair linked by transfer ID, and commit once. A failure must roll back the entire transfer.

Câu 305. What would happen if a transaction inserts the outbound transfer event but fails before inserting the inbound event?

Answer (English): If both inserts are in one database transaction, the failure rolls back the outbound insert, leaving no partial transfer. Separate commits would violate atomicity and require repair.

Câu 306. How would you enforce that an inventory adjustment cannot reduce available stock below zero when negative stock is forbidden?

Answer (English): Recompute `available` while holding the stock key lock and reject commands that would make it negative. A `CHECK(delta)` cannot guard the sum of ledger rows across

concurrent transactions.

Câu 307. What if the business permits negative stock temporarily? How would you represent and report it?

Answer (English): Explicitly permit the state by policy, record negative availability in reports, alert/reconcile promptly, and prevent it from being mistaken for physical on-hand quantity. Scope permission and duration tightly.

Câu 308. What is the difference between physical inventory and sellable inventory?

Answer (English): Physical inventory is what is present (including non-sellable buckets); sellable inventory excludes damaged, quarantined, reserved, or otherwise unavailable units. ATP also considers commitments and business rules.

Câu 309. How would you prevent a stale frontend screen from causing an invalid inventory adjustment?

Answer (English): Treat the UI as advisory only; validate submitted revision, count cutoff, or expected balance on the server inside a transaction. On conflict, return the current balance and ask the operator to re-evaluate.

Câu 310. How should the backend respond if stock changed after the operator loaded the page?

Answer (English): Return `409 Conflict` with a stable error code and current permitted stock state, not a silent overwrite. The UI should refresh and allow a new intentional action with a new idempotency key.

Câu 311. How would you handle two employees counting the same bin at the same time?

Answer (English): Create count-session ownership/leases per bin or reconcile two independent counts through a review step; do not silently accept last write wins. Record actor and count cutoff.

Câu 312. How would you record a physical count without accidentally overwriting all events?

Answer (English): Store a count observation separately, compute difference at a consistent cutoff, and post only an approved compensating adjustment event. Existing ledger history remains untouched.

Câu 313. How would you reconcile a system balance against a physical count?

Answer (English): Recompute book quantity as of count cutoff, compare to observed units, investigate movements that crossed the cutoff, and post a traceable adjustment after approval.

Câu 314. Why should a discrepancy be represented as an explicit adjustment rather than silently changing historical records?

Answer (English): An explicit adjustment maintains causality and shows who approved the correction. Rewriting historic events would destroy auditability and could corrupt prior reports.

Câu 315. What safeguards would you require before allowing a bulk inventory correction?

Answer (English): Require preview, strict SKU/bin scope, nonzero reasons, authorization/approval thresholds, transaction limits, backout strategy, audit IDs, and post-execution reconciliation.

Câu 316. What happens when the inventory event table grows to tens or hundreds of millions of rows?

Answer (English): Full scans grow expensive, indexes and maintenance become significant, and reporting latency increases. Partitioning, incremental projections, archival policy, and selective querying become important, while the source event history stays intact.

Câu 317. How would you index an inventory event table?

Answer (English): Index `(tenant_id,sku_id,bin_id,sequence)` for replay and `(source,idempotency_key)` uniquely for deduplication; add timestamp/partition indexes for operational reporting based on actual query plans.

Câu 318. Which columns would you use in a composite index for querying a SKU's events by time?

Answer (English): Use `(sku_id,bin_id,occurred_at,id)` when filtering a SKU/bin and ordering by time, with tenant/warehouse prefixes if required. A monotonic sequence is often safer than timestamps alone for replay order.

Câu 319. When would you partition an event table?

Answer (English): Partition when table scale and retention/query patterns justify operational complexity, commonly by recorded time or tenant. Plan partition pruning and uniqueness constraints carefully across partitions.

Câu 320. How would you preserve the audit history if you archive old events?

Answer (English): Archive in immutable, verifiable chunks with checksums, documented restore/replay, and accessible event identifiers. Do not delete history needed to reconstruct

balances or investigate past orders.

Câu 321. How would you calculate inventory efficiently without repeatedly scanning every historical event?

Answer (English): Maintain an incremental, rebuildable balance projection with one update per committed movement or a transactional outbox consumer. Periodically compare against a full ledger aggregation and alert on divergence.

Câu 322. What are the trade-offs of using a materialized view for inventory reporting?

Answer (English): Materialized views simplify expensive summaries but refresh can lag and use resources; incremental updates may be better for hot stock. Materialized reports are not authoritative for concurrent allocation.

Câu 323. If the database contains a materialized view of inventory balances, is that view the source of truth? Why?

Answer (English): No: the append-only ledger is the source of truth. A view is derived state that might be stale or corrupted and must be independently rebuildable.

Câu 324. How would you refresh or incrementally maintain a balance projection safely?

Answer (English): Apply each new event once to a projection using unique `(projection_name,event_id)` tracking or update the projection in the same transaction as posting. Rebuild from a checkpoint and reconcile against a full replay.

Câu 325. How would you detect if a derived balance does not match a full recomputation from the ledger?

Answer (English): Run scheduled aggregate comparisons by SKU/bin over a consistent snapshot or cutoff; report mismatches, projection freshness, and the earliest diverging event. Do not silently overwrite without diagnosis.

Câu 326. How would you rebuild a derived inventory projection after a bug is fixed?

Answer (English): Create a new version of the projection, replay events in order up to a stable watermark, catch up changes, compare counts/checksums, then atomically switch readers. Do not re-trigger shipments during replay.

Câu 327. What if an event arrives late or is recorded with an incorrect effective time?

Answer (English): Keep both effective business time and recorded time, apply deterministic ordering/reconciliation, and correct incorrect timestamps with explicit metadata corrections rather than mutating committed movement rows.

Câu 328. How would you separate event creation time from the business time at which the movement occurred?

Answer (English): `recorded_at` is when the database received the event; `effective_at` is the real-world movement time claimed by the domain. Preserve both for late deliveries, period reports, and audit.

Câu 329. How would you design an event replay process that does not accidentally trigger external side effects again?

Answer (English): Separate pure projection rebuilding from outbound commands; disable external dispatch during replay and rely on an outbox keyed by original business action. Replaying past movements must never resend fulfillment instructions.

Câu 330. How would you make event processing idempotent?

Answer (English): Track each `(consumer,event_id)` under a unique constraint, apply projection change and acknowledgement atomically, and make downstream side effects separately idempotent. Expect at-least-once delivery.

PHẦN 7. THIẾT KẾ HỆ THỐNG QUẢN LÝ KHO

Câu 331. [ƯU TIÊN CAO] Design a warehouse management system for receiving, binning, picking, packing, returns, and inventory tracking.

Answer (English): I would model products/SKUs, warehouse/bin locations, receipts, orders, reservations, picks, shipments, returns, and immutable movements. Operators use a scan-first UI; transactional services protect stock, while background workers synchronize external systems through idempotent jobs.

Câu 332. What are the core entities and relationships in your design?

Answer (English): Core links are product→variant→SKU, warehouse→bin, order→order lines→SKU, receipt→received lines, pick task→order line/bin, and inventory event→SKU/bin/source action. Each entity has stable IDs and explicit constraints.

Câu 333. What happens from the moment a shipment arrives at the warehouse until the items are available for sale?

Answer (English): Create a receipt against a shipment/PO, scan and verify SKU/quantity, place units in a receiving or QC state, inspect if required, put them away into a bin, then post an event making approved units sellable.

Câu 334. How would you design the receiving workflow?

Answer (English): Receiving starts with expected lines and shipment IDs, scans actual units, records shortages/overages/damage, and posts accepted quantities idempotently. Validate a receipt state transition and retain operator evidence.

Câu 335. How would you design the binning or put-away workflow?

Answer (English): Put-away proposes valid destination bins based on capacity and rules, scans SKU and bin, then atomically transfers units from receiving to storage. Wrong-bin scans are rejected or require explicit override.

Câu 336. How would you design the picking workflow?

Answer (English): Allocate stock safely, generate prioritized pick tasks with locations/quantities, scan bin and SKU, confirm exact picked quantity, and record shortages as exceptions. Prevent two workers claiming the same pick without coordination.

Câu 337. How would you design the packing workflow?

Answer (English): At packing, verify the picked items against the order, capture packing materials and weight if needed, choose carrier/service, and create one shipment/label with idempotency. Do not pack unpicked units.

Câu 338. How would you design the shipping confirmation workflow?

Answer (English): Confirm only after label/manifest and handoff conditions are met; record carrier tracking, time, actor, and shipment-line quantities. A shipment update and inventory movement must follow the chosen fulfillment accounting rules.

Câu 339. How would you design the returns workflow?

Answer (English): Link return authorization to the original sale, receive/scan units, inspect condition, choose disposition, then move into resellable, quarantine, or scrap inventory. Refund processing is tracked separately from physical receipt.

Câu 340. How would you handle damaged, defective, or unsellable returned products?

Answer (English): Keep damaged and unsellable units in distinct non-ATP locations/statuses, record evidence and reason, and optionally schedule repair/disposal. Do not automatically restock

returned goods.

Câu 341. How would you model inventory movement between bins?

Answer (English): Treat a movement as a single transfer command that writes an immutable debit and credit event with a common `transfer_id`; both happen atomically and total warehouse quantity is unchanged.

Câu 342. How would you handle a worker scanning the wrong SKU?

Answer (English): Validate scanned code against expected SKU and bin; reject mismatch with distinct audible/visual feedback, retain the mistaken scan for diagnostics if appropriate, and require deliberate authorized override.

Câu 343. How would you prevent an item from being packed before it has been picked?

Answer (English): Enforce workflow transitions and `packed_qty <= picked_qty` in a backend transaction. A disabled frontend button is useful UX but not an authorization or integrity boundary.

Câu 344. How would you handle an order that requires items from multiple bins?

Answer (English): Represent a line allocation across multiple bins and generate independent pick tasks. Track planned versus actual quantities so shortages in one bin can be reallocated without duplicating fulfilled units.

Câu 345. What if the picker cannot find an item that the system says is available?

Answer (English): Stop that pick, search nearby/alternate locations, record a shortage reason, request recount or reallocation, and flag reconciliation. Never silently mark missing units as picked.

Câu 346. How would you support partial picking?

Answer (English): Store picked quantity per task and order line, allow additional scans only up to remaining reserved quantity, and maintain resumable task state. Partial pick does not imply full shipment.

Câu 347. How would you support partial shipments?

Answer (English): Create shipments with their own line quantities, tracking IDs, and fulfillment status. Validate the sum of shipped quantities never exceeds the order's accepted quantities.

Câu 348. How would you handle an order cancelled while picking is in progress?

Answer (English): Use a coordinated state transition: prevent new picking, release unpicked reservations, and handle picked-but-not-shipped units through restock. If already shipped, follow return/refund workflow instead.

Câu 349. How would you handle multiple workers performing operations on one order?

Answer (English): Use task ownership or optimistic versioning with atomic claim operations; multiple workers may act on separate tasks while order totals are guarded transactionally. Show real-time task status.

Câu 350. How would you track who performed each warehouse operation?

Answer (English): Capture authenticated actor, device, timestamp, action ID, source/target location, related order/receipt, and correlation ID. Preserve immutable events and audit logs for reconstructing each operation.

Câu 351. How would you make a workflow recoverable if a device loses network connectivity?

Answer (English): Use a local queue only where permitted, show clear offline status, and send each queued command later with a stable idempotency key. On reconnect, the server revalidates stock/state and reports conflicts for operator resolution.

Câu 352. Which actions require immediate consistency, and which can be processed asynchronously?

Answer (English): Stock reservation, allocation, picks, transfer, and order transitions need immediate transactional correctness. Dashboards, emails, marketplace syncs, and analytics usually tolerate asynchronous processing.

Câu 353. Which operations should be atomic database transactions?

Answer (English): Reserve+update order line, paired transfer movements, receipt acceptance+ledger event, and irreversible status changes should each commit as one unit. External API calls belong outside the transaction.

Câu 354. Which workflows should use background jobs?

Answer (English): Catalog imports, Shopify synchronization, notification sending, projection rebuilding, heavy reports, and reconciliation are appropriate background jobs. Use durable queues, retries, dead-letter handling, and idempotency.

Câu 355. How would you prevent duplicate labels, shipments, or receiving records?

Answer (English): Use business-specific unique keys (receipt line/scan ID, shipment request ID, carrier label request ID) and enforce them in the DB. Retried commands return the previous result, not a second external action.

Câu 356. How would you design a warehouse dashboard for operators?

Answer (English): Make expected task, SKU, bin, quantity, scan field, and next action prominent; support keyboard/scanner use, clear status, and fast error recovery. Show stale data when necessary but revalidate actions server-side.

Câu 357. What metrics would help a warehouse manager understand operational performance?

Answer (English): Measure inventory discrepancy rate, picks/hour, pick accuracy, receipt turnaround, late shipments, return disposition time, blocked tasks, sync lag, and exception resolution time.

Câu 358. How would you reduce the number of clicks and mistakes for warehouse workers?

Answer (English): Study real warehouse operations, prefill expected data, support scanning, reduce context switching, add clear validation, and keep success/error feedback immediate. Measure wrong scans and completion time after release.

Câu 359. How would you design an exception queue for failed operations or mismatched inventory?

Answer (English): Create an exception entity with type, affected SKU/order, evidence, severity, owner, status, retries, and resolution actions. Operators need searchable queues and a controlled correction process.

Câu 360. How would you onboard a new warehouse without modifying the entire application?

Answer (English): Use tenant/facility-scoped configurations, warehouse/bin master data, role assignments, carrier/label settings, and import validation. Shared workflows should use configuration, not one-off warehouse-specific code.

PHẦN 8. NODE.JS — BACKEND FUNDAMENTALS

Câu 361. [ƯU TIÊN CAO] Explain the architecture of a Node.js backend application.

Answer (English): A maintainable Node.js backend has an HTTP router/middleware boundary, validation/authentication, domain services, repositories, integration clients, and background jobs. Transactions and invariants belong in the service/data layer.

Câu 362. What makes Node.js appropriate for API and integration workloads?

Answer (English): Node.js excels at network/API/database-bound workloads due to nonblocking I/O and its TypeScript ecosystem. It simplifies sharing contracts with frontend, but correctness still relies on database transactions.

Câu 363. What are the limitations of Node.js?

Answer (English): Single-threaded JavaScript execution makes heavy CPU computation harmful to request latency; some native integrations and memory use also need care. Move CPU jobs to workers and monitor event-loop lag.

Câu 364. How does Node.js handle concurrent I/O?

Answer (English): Node registers I/O with the OS/libuv, continues other work, and executes callbacks/promise reactions when operations complete. Concurrent I/O does not mean each JavaScript request runs on a dedicated thread.

Câu 365. What happens when CPU-intensive work runs on the main event loop?

Answer (English): A CPU-heavy synchronous calculation blocks event-loop progress, delaying unrelated HTTP requests, timers, and I/O callbacks. Use worker threads/processes or a separate compute service.

Câu 366. What is the difference between Node.js's event loop and the operating system's thread model?

Answer (English): Node runs JavaScript callbacks on an event-loop thread while libuv/OS mechanisms handle many I/O operations and a worker pool handles certain tasks. It is not a one-thread-per-request server.

Câu 367. When would you use worker threads?

Answer (English): Use worker threads for CPU-intensive work that benefits from parallelism, such as expensive parsing or image processing. Avoid them for simple database waits; manage worker lifecycle and resource limits.

Câu 368. When would you use a separate background worker instead of handling work within the HTTP request?

Answer (English): Move work to a durable queue when it exceeds request deadlines, requires retries, depends on rate-limited APIs, or must survive service restarts. Return a job ID and expose status rather than holding a request open.

Câu 369. How would you structure a maintainable Node.js codebase?

Answer (English): Organize by domain (`inventory` , `orders` , `products` , `integrations`) with routes, services, repositories, schemas, and tests. Keep framework-specific code thin and dependency directions explicit.

Câu 370. How would you separate routes, business logic, database access, and external API clients?

Answer (English): Routes parse/authenticate and return responses; services enforce business policy; repositories execute SQL; external clients encapsulate provider contracts/retry policy. Transactions must span repositories invoked by one business command.

Câu 371. What is middleware?

Answer (English): Middleware runs between the incoming request and route handler to handle authentication, logging, tracing, rate limiting, validation, and CORS. It should be composable and have well-defined order.

Câu 372. How does middleware execution order affect an API?

Answer (English): If authorization runs before identity is established it lacks a principal; if error handling is placed incorrectly it may miss failures. Review request and response middleware ordering, including body/signature verification.

Câu 373. How would you implement centralized error handling?

Answer (English): Map typed domain errors to stable HTTP statuses and codes, log unexpected errors with request IDs, and return sanitized 500 responses. Handle asynchronous failures at the framework's appropriate boundary.

Câu 374. How would you handle uncaught exceptions and unhandled promise rejections?

Answer (English): Log with context and perform controlled shutdown/restart for unhandled fatal errors rather than pretending state is safe. Expected errors should be awaited/caught at task boundaries and not reach process-level handlers.

Câu 375. How would you validate and sanitize incoming requests?

Answer (English): Validate content type, size, schema, identifiers, ranges, and ownership, and use parameterized SQL. Context-sensitive output escaping protects HTML; blanket text stripping is not a substitute for validation.

Câu 376. Why should the backend never trust frontend validation alone?

Answer (English): The browser is user-controlled and can be bypassed with direct HTTP requests. All permissions and stock constraints must be enforced server-side and ideally protected by database invariants.

Câu 377. How would you design RESTful endpoints for inventory operations?

Answer (English): Use `GET /inventory?skuId=...`, `POST /inventory/adjustments`, `POST /inventory/transfers`, and `POST /orders/:id/reservations` with explicit domain commands. Support idempotency keys and `409` conflicts for stale stock.

Câu 378. When would you use `POST`, `PUT`, `PATCH`, and `DELETE`?

Answer (English): `POST` creates/executes commands; `PUT` replaces a resource representation idempotently; `PATCH` partially modifies it; `DELETE` removes it when domain policy permits. Ledger movements are appended with POST, not deleted.

Câu 379. What is the difference between `401 Unauthorized` and `403 Forbidden`?

Answer (English): `401` means the request lacks valid authentication; `403` means an authenticated identity lacks permission. A hidden resource may deliberately return `404` to avoid disclosing its existence.

Câu 380. How would you design consistent API error responses?

Answer (English): Use a stable structure such as `{error:{code,message,details?,requestId}}`; distinguish validation (`400` / `422`), unauthorized (`401`), forbidden (`403`), conflicts (`409`), and transient server failures.

Câu 381. How would you implement request logging and correlation IDs?

Answer (English): Generate or accept a validated correlation/request ID at ingress, propagate it through service, SQL/application job metadata, and outbound calls, and log structured fields. Avoid recording raw tokens or personal data.

Câu 382. How would you handle request timeouts?

Answer (English): Apply a server deadline and upstream `AbortSignal`, configure reverse-proxy timeouts, and move long jobs to queues. For writes, a timeout does not prove failure; resolve uncertainty with idempotency status.

Câu 383. How would you protect an API from abusive or excessive requests?

Answer (English): Authenticate, authorize, rate-limit, cap request/body sizes, bound concurrency, and detect misuse. Protect expensive mutation endpoints more tightly than harmless cached reads.

Câu 384. How would you implement rate limiting?

Answer (English): Use a token bucket or fixed/sliding window keyed by appropriate identity/IP/tenant, preferably in shared infrastructure across instances. Respond with `429` and `Retry-After`; account for trusted proxies.

Câu 385. How would you handle graceful shutdown?

Answer (English): On termination, stop accepting new requests, allow bounded in-flight work to finish, release queue leases, flush telemetry, and close DB pools. Durable jobs must be resumable after the process stops.

Câu 386. How would you manage database connection pools?

Answer (English): Create a bounded pool per service instance, reuse it, monitor active/idle/waiting connections, and set timeouts. Total max connections across replicas must fit PostgreSQL's limit with room for admin access.

Câu 387. What problems can occur if every request creates a new database connection?

Answer (English): Opening new connections per request adds handshake overhead and can exhaust the DB's connection capacity, causing cascading failures. Use a pool and avoid long idle transactions.

Câu 388. How would you handle long-running tasks in a backend service?

Answer (English): Persist a job record, enqueue once, return `202 Accepted` plus a status URL, and process with retries/backoff and dead-letter routing. Keep side effects idempotent.

Câu 389. How would you implement retries without creating duplicate side effects?

Answer (English): Persist the business operation key before the side effect, guard with a uniqueness constraint, and reuse it on every retry. Classify transient errors and never blindly re-

send irreversible commands.

Câu 390. How would you protect secrets and API credentials?

Answer (English): Use a secret manager or protected server environment, least-privilege credentials, rotation, and access auditing. Do not expose credentials in client bundles, Git history, logs, or AI prompts.

Câu 391. How would you investigate memory leaks in a Node.js service?

Answer (English): Track heap usage over time, create heap snapshots under controlled conditions, look for retained listeners/caches/closures, and profile allocations. Test with representative long-running load and check whether memory returns after GC.

Câu 392. How would you handle a third-party API that frequently returns `429 Too Many Requests`?

Answer (English): Honor `Retry-After`, apply per-provider rate budgets, use exponential backoff with jitter, and queue work. Do not amplify a 429 storm through synchronized retries.

Câu 393. How would you handle temporary database or network failures?

Answer (English): Use bounded retries for transient reads/jobs, fast health checks, graceful degradation, and circuit breakers if needed. Critical writes need explicit transaction outcomes and idempotent retry semantics.

Câu 394. How would you ensure API logs never expose sensitive customer information?

Answer (English): Redact credentials, contact details, address fields, payment data, and full raw payloads. Use structured allowlisted log fields, access controls, retention policies, and trace IDs instead.

Câu 395. How would you version an API without breaking existing consumers?

Answer (English): Use versioned routes/contracts where necessary, backward-compatible additive changes by default, and staged deprecation. Use contract tests to protect consumers during schema evolution.

PHẦN 9. DENO, HONO VÀ BACKEND MODERN

Câu 396. What is Deno, and how is it different from Node.js?

Answer (English): Deno is a TypeScript/JavaScript runtime with web-standard APIs, a granular permission model, and built-in tooling; Node.js is older with a large npm ecosystem. Modern Deno

supports npm/Node compatibility but not every native dependency behaves identically.

Câu 397. What advantages does Deno offer for TypeScript development?

Answer (English): Deno can run TypeScript directly with integrated tooling and a web-standards-oriented API, reducing setup complexity. Teams still need type checks, dependencies locked, operational monitoring, and compatibility tests.

Câu 398. How does Deno's permission model work?

Answer (English): Deno restricts access to resources such as files, environment variables, and network by default; permissions can be granted by flags/configuration and context. Limit production permissions to what a service actually needs.

Câu 399. How are npm packages used in modern Deno projects?

Answer (English): Modern Deno can import packages using `npm:` specifiers and use npm dependencies with compatibility support. Lock dependency versions and test native modules/Node-specific APIs in the target deployment.

Câu 400. How do environment variables and secrets work in Deno?

Answer (English): Read environment variables via `Deno.env.get` subject to `--allow-env` permission; inject secrets from deployment controls, not source code. Validate required configuration at startup.

Câu 401. How would you port a small Node.js API to Deno?

Answer (English): Preserve HTTP contract and domain services, replace runtime-specific filesystem/process imports, review npm/native compatibility, and test requests, auth, DB pooling, and shutdown. Deploy a staging version before cutover.

Câu 402. What compatibility problems might arise when porting Node.js code to Deno?

Answer (English): Watch for Node built-ins/native add-ons, file paths, runtime permissions, package resolution, HTTP server behavior, and deployment constraints. Run integration tests in actual Deno instead of trusting compilation alone.

Câu 403. What is Hono?

Answer (English): Hono is a lightweight, web-standards-based HTTP framework for TypeScript/JavaScript runtimes. It provides routing, middleware, and typed request/response utilities.

Câu 404. Why might a team choose Hono instead of Express?

Answer (English): Hono may fit teams targeting Deno/edge/web-standard APIs with a small footprint and straightforward routing. Express remains mature; choose based on deployment, middleware compatibility, and team familiarity.

Câu 405. How do routes and middleware work in Hono?

Answer (English): Define routes with `app.get/post/...` and attach middleware globally or to paths; middleware can execute before and after `next()`. Keep business services separate from the router.

Câu 406. How would you validate a request schema in a Hono application?

Answer (English): Use a runtime schema validator (for example Zod with appropriate Hono middleware) for params, queries, headers, and JSON bodies. A TypeScript annotation alone cannot validate attacker-controlled input.

Câu 407. How would you organize a Hono application as it grows?

Answer (English): Group routes and validators by domain, inject database/services through typed context, and centralize errors/authentication. Keep routes thin and test behavior through the public request interface.

Câu 408. How would you implement global error handling in Hono?

Answer (English): Register `app.onError` to log unexpected errors and translate domain exceptions to consistent responses. Add `app.notFound` for missing routes and avoid leaking stack traces.

Câu 409. How would you connect a Hono API to PostgreSQL?

Answer (English): Instantiate a compatible Postgres client/pool at appropriate scope, keep credentials server-side, and inject repositories into handlers. For transactional commands, use a single session/client through the entire transaction.

Câu 410. How would you test a Hono endpoint?

Answer (English): Use Hono's request testing interface or invoke `app.request` with Request objects, asserting statuses/body/auth/validation. Add integration tests against a real PostgreSQL test database for side effects.

Câu 411. How would you implement authentication middleware?

Answer (English): Verify session/JWT signatures and expiry, load identity/roles, then check resource permissions before handling the request. Also enforce database RLS/least privilege where

relevant.

Câu 412. How would you handle CORS correctly?

Answer (English): Allow only explicitly trusted origins, methods, and headers; correctly handle preflight and credentials. CORS is a browser policy, not authentication, and cannot replace server-side authorization.

Câu 413. How would you manage database connections in a serverless or edge-oriented runtime?

Answer (English): Use a pooled/shared connection service or HTTP-based DB access appropriate for the platform, cap connections and avoid opening pools per invocation. Confirm whether the edge runtime supports required TCP/protocol features.

Câu 414. How would you identify libraries that do not work in the target Deno environment?

Answer (English): Check the official support matrix, runtime import errors, Node/native dependencies, permission requirements, and staging integration tests. Verify packaging and deployment, not just the type checker.

Câu 415. How would you decide whether to adopt a new runtime in an existing production system?

Answer (English): Adopt a runtime only when its operational/developer benefits exceed migration and compatibility costs. Test a small service under production-like load with monitoring and recovery before committing broadly.

PHẦN 10. SUPABASE — DATABASE SERVICES VÀ RPC

Câu 416. What is Supabase?

Answer (English): Supabase is a hosted backend platform built around PostgreSQL with APIs, authentication, storage, and other services. PostgreSQL schemas, transactions, and permissions still require careful design.

Câu 417. Which Supabase components have you used?

Answer (English): I would specify only the components I have truly used; typical components include hosted Postgres, Auth, Storage, RLS, Edge Functions, and database RPC. Familiarity with the product does not imply production ownership.

Câu 418. How is Supabase different from managing PostgreSQL yourself?

Answer (English): Supabase reduces effort for hosting/integrated services but introduces platform conventions, quotas, and permission boundaries. Self-managed PostgreSQL provides more operational control at the cost of backups, patching, monitoring, and high availability.

Câu 419. How does Supabase authentication work?

Answer (English): Supabase Auth verifies identity using sessions/tokens; the API applies database roles and RLS using authenticated context. A valid session proves identity, not that every warehouse row is permitted.

Câu 420. What is Row Level Security (RLS)?

Answer (English): PostgreSQL Row Level Security lets table policies restrict which rows a database role may select/insert/update/delete. Enable policies carefully and test with real role claims; privileged roles can bypass RLS.

Câu 421. [ƯU TIÊN CAO] How would you write an RLS policy so warehouse workers can access only the records they are permitted to access?

Answer (English): Store warehouse memberships `(user_id,warehouse_id,role)`, enable RLS, and allow row access only when membership exists and operation rights match. Validate both `USING` for existing rows and `WITH CHECK` for inserted/updated values.

Câu 422. Why is hiding a button in the frontend not sufficient authorization?

Answer (English): Users can bypass UI and call APIs directly, or manipulate browser state. The backend and database must independently authorize each record and stock-changing command.

Câu 423. What is the difference between the `anon`, authenticated, and service-role contexts?

Answer (English): Supabase's public anon and authenticated contexts typically have limited database privileges/RLS-scoped access; privileged service-role credentials can bypass RLS. The exact deployment key model must be verified for the project version.

Câu 424. Why must a service-role key never be exposed to the browser?

Answer (English): A service-role key can bypass normal row restrictions, potentially exposing or altering all data if leaked. Keep it exclusively in trusted server environments and rotate after exposure.

Câu 425. How would you securely access Supabase from a backend service?

Answer (English): Use server-side credentials and connection settings, authenticate incoming users, enforce role checks, and restrict database privileges. Never trust that possession of a server

key authorizes a user action.

Câu 426. What are database functions in PostgreSQL?

Answer (English): PostgreSQL functions encapsulate reusable SQL/PLpgSQL logic that executes in the database, possibly inside the caller's transaction. They are appropriate for atomic operations requiring several SQL statements.

Câu 427. What is a Supabase RPC call?

Answer (English): A Supabase RPC exposes an allowed Postgres function through the API client. Treat it as a secured business endpoint: validate arguments, permissions, and function execution context.

Câu 428. When would you put business logic in a PostgreSQL function rather than in the application service?

Answer (English): Put logic in a function when atomic multi-statement invariants and lock discipline must apply to all writers. Keep external API calls, broad orchestration, and presentation formatting in application services.

Câu 429. What are the trade-offs of putting business logic inside the database?

Answer (English): Database functions enforce correctness near the data and reduce round trips, but can be harder to version, observe, and test for application developers. Keep contracts narrow and cover them with integration tests.

Câu 430. How would you create an RPC function for a safe inventory adjustment?

Answer (English): Create an `adjust_stock(sku,bin,delta,key,reason)` function that authorizes the caller, locks the stock key, verifies quantity constraints, inserts an event with unique key, and returns the resulting balance atomically.

Câu 431. How would you ensure a database function cannot be abused by an unauthorized user?

Answer (English): Grant execution only to appropriate roles; inside the function verify caller identity, warehouse membership, permitted adjustment magnitude, and explicit reason. Never rely on clients hiding the RPC button.

Câu 432. What security concerns arise with `SECURITY DEFINER` functions?

Answer (English): `SECURITY DEFINER` executes with the function owner's privileges and may bypass callers' table permissions/RLS. Restrict EXECUTE, validate all IDs, use a safe fixed search path, and avoid unsafe dynamic SQL.

Câu 433. What does `search_path` mean in PostgreSQL, and why can it matter for security?

Answer (English): `search_path` controls where unqualified object names resolve. In elevated functions, a caller-influenced path can redirect names to attacker-controlled objects; schema-qualify references and set a safe path.

Câu 434. How would you version and test database functions?

Answer (English): Keep function definitions in versioned migrations, test permission differences and concurrent requests against a real DB, and review execution plans/lock behavior. Treat function signatures as public contracts.

Câu 435. How would you troubleshoot a slow Supabase query?

Answer (English): Use query logging/monitoring and `EXPLAIN (ANALYZE, BUFFERS)` safely, inspect RLS predicates, filters/indexes, joins, and pool waits. Different query plans may appear under different permission contexts.

Câu 436. How would you manage connection limits and pooling?

Answer (English): Limit pools across workers and replicas, use the appropriate Supabase pooler/direct connection for the workload, avoid long transactions, and monitor saturation and connection churn.

Câu 437. How would you migrate a Supabase schema across development, staging, and production?

Answer (English): Keep migrations in Git, apply them in a controlled pipeline from dev to staging to production, review policy/function privileges, seed non-sensitive test data, and validate compatibility before deployment.

Câu 438. How would you test RLS policies for users with different roles?

Answer (English): Create actual test users/claims for each role, warehouse membership, and missing permissions; assert select/insert/update/delete allowed and denied cases. Include attempts to change warehouse IDs and cross-tenant access.

Câu 439. How would you avoid accidentally granting a warehouse operator permission to edit financial or administrative records?

Answer (English): Separate warehouse and finance/admin permissions in roles, RLS policies, and backend endpoints; revoke unnecessary table/function grants. Negative authorization tests should cover operator attempts on financial records.

Câu 440. What would you do if a change to an RLS policy suddenly blocked legitimate warehouse operations?

Answer (English): Pause affected deployment if needed, reproduce with the exact role claims, inspect policy expressions and table grants, restore a reviewed last-known-good policy, and add regression tests before re-releasing.

PHẦN 11. DRIZZLE ORM VÀ DATABASE ACCESS LAYER

Câu 441. What is an ORM?

Answer (English): An ORM maps application representations and queries to database operations. It can improve type safety and consistency but does not eliminate SQL, transaction design, or query-plan analysis.

Câu 442. What are the advantages and disadvantages of using an ORM?

Answer (English): Advantages include typed queries, schema integration, and reduced boilerplate; disadvantages include hidden SQL patterns, difficult advanced queries, and abstraction leakage. For inventory-critical logic, inspect generated SQL.

Câu 443. What is Drizzle ORM?

Answer (English): Drizzle ORM is a TypeScript-first query builder/ORM with SQL-oriented schema definitions and typed query APIs. It keeps relational concepts and SQL relatively visible.

Câu 444. How does Drizzle differ from a traditional object-oriented ORM?

Answer (English): Drizzle usually represents tables/queries directly rather than tracking long-lived mutable object graphs. It emphasizes typed SQL-style operations and explicit migrations, making query behavior easier to inspect.

Câu 445. How do you define a schema in Drizzle?

Answer (English): Define tables using the dialect schema helpers, declaring column types, primary/foreign keys, unique indexes, and references. Generate and inspect migration SQL from these definitions.

Câu 446. How do TypeScript types relate to the database schema?

Answer (English): Drizzle can infer select/insert TypeScript types from table definitions, but runtime input still needs validation. Database constraints are the final authority and migration state must match the code schema.

Câu 447. How would you write a join using Drizzle?

Answer (English): Join explicitly on foreign keys, e.g. `db.select().from(skus).innerJoin(products,eq(skus.productId,products.id))`. Check one-to-many cardinality before pagination or aggregation.

Câu 448. How would you execute a transaction using Drizzle?

Answer (English): Use `db.transaction(async tx => { ... })` and pass `tx` into every repository write/read that belongs to the atomic command. Throw on failure so all writes roll back.

Câu 449. How would you write raw SQL when an ORM query is insufficient?

Answer (English): Use parameterized `sql` fragments or a driver with bound arguments for CTEs, window functions, locks, or advanced aggregates. Document why raw SQL is preferable to composing ORM calls.

Câu 450. How do you avoid SQL injection when using raw SQL?

Answer (English): Use parameter placeholders/bound values, never interpolate untrusted strings into SQL text. Dynamic identifiers require allowlisted names or safe identifier handling, not ordinary parameter placeholders.

Câu 451. How does Drizzle manage migrations?

Answer (English): Drizzle Kit can generate SQL migration files from schema changes and track applied migrations. Review generated SQL for locks, compatibility, constraints, and data migration needs before production.

Câu 452. What is schema drift, and how would you detect it?

Answer (English): Schema drift means the actual DB schema differs from the version-controlled intended state, often due to manual changes or skipped migrations. Detect it via migration checks and schema comparison in CI/staging.

Câu 453. How would you migrate an existing database into Drizzle without disrupting production?

Answer (English): Introspect existing tables carefully, align definitions without destructive recreation, baseline current migration state, and migrate incrementally. Rehearse on a copy with real row counts and unchanged data checks.

Câu 454. How would you decide whether to use Drizzle or direct SQL for a complex inventory aggregation?

Answer (English): Use Drizzle for ordinary typed CRUD/joins; for complex ledger windowing/locking, clear parameterized SQL may be more readable and optimizable. Correctness and explainable query plans outrank ORM purity.

Câu 455. How would you map PostgreSQL errors to useful application errors?

Answer (English): Translate unique violations, foreign-key failures, check violations, and serialization/deadlock errors into stable domain conflicts or retry decisions. Log database details internally, not in client responses.

Câu 456. How would you ensure a transaction spans all necessary database writes?

Answer (English): Reuse the transaction object throughout all repository calls, and avoid starting independent transactions/connections mid-command. Test rollback by inducing a failure after the first write.

Câu 457. How would you prevent a partial inventory operation if one query fails?

Answer (English): Group dependent writes under one transaction, propagate errors rather than swallowing them, and avoid external side effects before commit. Use unique operation keys so retries stay safe.

Câu 458. How would you test repository or database-access code?

Answer (English): Use a real disposable PostgreSQL instance and migrations, seed known fixtures, run repository operations, and assert rows/rollback/concurrency behavior. Small unit tests can cover pure mapping functions separately.

Câu 459. How would you separate database schema types from API response types?

Answer (English): Map internal rows with potentially private fields into an explicitly versioned public DTO; do not return raw ORM objects by default. Contracts should survive benign schema changes.

Câu 460. How would you diagnose a bug caused by an ORM generating an unexpected query?

Answer (English): Enable safe SQL logging for the affected query, inspect generated statement/parameters, compare with intended joins and indexes, and reproduce against real data.

Fix query semantics and add an integration regression test.

PHẦN 12. REACT — FRONTEND FUNDAMENTALS

Câu 461. [ƯU TIÊN CAO] How does React's rendering model work?

Answer (English): React renders UI descriptions from props/state, reconciles changes, and commits updates to the host DOM. Rendering should be pure; effects handle synchronization with external systems after commits.

Câu 462. What causes a React component to rerender?

Answer (English): State updates, parent renders, context changes, or external-store subscriptions can trigger rerender. React may render without changing the DOM, so profile actual bottlenecks before memoizing.

Câu 463. What is the difference between props and state?

Answer (English): Props are values provided by a parent; state is local reactive data managed by the component. Lift state only when several consumers truly need the same source of truth.

Câu 464. Why should React state be treated as immutable?

Answer (English): React compares references and relies on state updates to signal changes; mutating existing objects may not trigger correct renders and can corrupt previous snapshots. Use immutable copies or a well-understood immutable update helper.

Câu 465. What are controlled and uncontrolled components?

Answer (English): Controlled inputs use React state as the source of truth; uncontrolled inputs retain DOM-managed values accessed by refs/form submission. Controlled forms simplify conditional validation; uncontrolled can reduce rerenders.

Câu 466. What are React hooks?

Answer (English): Hooks let function components use state, refs, effects, context, and reusable reactive logic. Custom hooks encapsulate behavior while still following the same execution-order rules.

Câu 467. What are the Rules of Hooks?

Answer (English): Call hooks only at the top level of React function components or custom hooks, not inside loops, conditions, event callbacks, or ordinary functions. This preserves consistent hook ordering.

Câu 468. How does `useEffect` work?

Answer (English): `useEffect` synchronizes with external systems after a render commit, and cleanup disposes subscriptions or requests. It is not required for values derivable directly during render.

Câu 469. What are common mistakes when using `useEffect` ?

Answer (English): Common mistakes include missing dependencies, effect loops, stale closures, setting derived state unnecessarily, and forgetting cleanup. Strict development behavior can expose unsafe setup/cleanup patterns.

Câu 470. What is the purpose of the dependency array?

Answer (English): A dependency array lists reactive values used by the effect; React reruns it when those values change. An empty array does not make code exempt from stale state or cleanup requirements.

Câu 471. How do you avoid stale closures in React?

Answer (English): Use correct effect dependencies, functional state updates, or refs when holding the latest value without rerendering. Avoid suppressing lint warnings that reveal captured stale values.

Câu 472. When should you use `useMemo` ?

Answer (English): Use `useMemo` to cache an expensive pure calculation when profiling shows a benefit or to stabilize a value needed by memoized children. It is a performance hint, not a correctness mechanism.

Câu 473. When should you use `useCallback` ?

Answer (English): Use `useCallback` to stabilize a function reference when passing it to memoized children or other hooks that depend on identity. Ensure dependencies capture the right state.

Câu 474. Why should you not use memoization everywhere?

Answer (English): Memoization adds complexity and comparison/memory overhead, and may not improve user-perceived speed. Fix unnecessary state ownership, huge renders, and query inefficiency first.

Câu 475. What is `React.memo` ?

Answer (English): `React.memo` can skip rendering a component when its props are equal according to the comparison rule. It does not prevent rerenders caused by its own state or

consumed context.

Câu 476. What is the difference between component state and global application state?

Answer (English): Component state belongs to one UI area; global state represents information shared across distant components. Server state/cached queries should usually be handled differently from client-only UI state.

Câu 477. When would you use Context?

Answer (English): Context is useful for low-frequency shared values such as theme, authenticated user, or locale. Frequently changing large contexts can rerender many consumers; split providers or use stores when needed.

Câu 478. When would you use a dedicated state management library?

Answer (English): Use a store for complex cross-screen state, sophisticated updates, or subscription granularity. Do not introduce one solely to hold data already managed by a server-state query library.

Câu 479. How do React keys affect list rendering?

Answer (English): Keys identify list items across renders so React preserves the right component state when data changes. Use stable database IDs rather than transient array positions.

Câu 480. Why is using an array index as a key sometimes dangerous?

Answer (English): When inserting/reordering rows, index keys may associate component input state with the wrong SKU/order. Stable keys prevent errors in editable warehouse tables.

Câu 481. How would you manage a form with complex validation?

Answer (English): Use a form schema for types/constraints, controlled or library-assisted fields, field-level and submit-level validation, and display precise errors. Revalidate in the backend and guard submission idempotency.

Câu 482. How would you implement optimistic updates?

Answer (English): Optimistically update local UI immediately and keep a rollback snapshot; on server success reconcile with authoritative response, on failure restore and show a conflict. Use cautiously for stock-critical writes.

Câu 483. What problems can occur with optimistic updates if the server rejects a request?

Answer (English): The client may show a quantity that the transaction later rejects because stock changed. Roll back, invalidate the query, and communicate the new balance; never assume a successful visual update means the database committed.

Câu 484. How would you handle loading, empty, error, and success states consistently?

Answer (English): Define a standard request state model, show skeleton/progress for loading, helpful empty-state actions, retry/error messages, and accessible success feedback. Avoid silently reusing stale counts as live truth.

Câu 485. How would you prevent duplicate form submissions?

Answer (English): Disable or gate the action while submitting, use one stable idempotency key per user command, and enforce uniqueness in the backend. UI disabling alone cannot prevent network retries or multi-tab duplicates.

Câu 486. How would you cancel or ignore stale API requests?

Answer (English): Abort stale GETs via `AbortController`, or ignore responses older than the latest request ID; query libraries help cache/invalidate. For writes, cancellation does not guarantee rollback at the server.

Câu 487. How would you handle API pagination in React?

Answer (English): Use cursor-based queries, stable filter/sort keys, and a reusable query cache. Reset the cursor on filter change, show fetch-next state, and handle list mutations consistently.

Câu 488. How would you make a large inventory table performant?

Answer (English): Use server-side filtering/sorting/pagination, virtualize visible rows, memoize only expensive cells, and avoid per-row network requests. Preserve keyboard navigation and screen-reader semantics.

Câu 489. What is list virtualization?

Answer (English): List virtualization renders only rows near the viewport to reduce DOM nodes and rendering cost. It needs care for variable-height rows, focus management, accessibility, and sticky headers.

Câu 490. How would you implement search and filtering without sending too many API requests?

Answer (English): Debounce user typing, cancel older searches, query with indexed filters, and cache by filter key. Scanner-driven exact SKU lookup should usually respond immediately rather

than use a long debounce.

Câu 491. How would you prevent unnecessary rerenders in a complex dashboard?

Answer (English): Profile with React DevTools, colocate state, split broad contexts, stabilize keys/props where worthwhile, and virtualize large lists. Fix underlying data-fetch patterns before widespread memoization.

Câu 492. How would you structure a large React application?

Answer (English): Organize by domain feature with shared UI primitives, API clients, runtime schemas, hooks, and tests. Keep inventory rules on the server; frontend modules should not reimplement authoritative stock logic.

Câu 493. How would you reuse UI components across different warehouse workflows?

Answer (English): Share design primitives (scan field, status badge, quantity input, confirmation dialog) and domain-aware hooks while allowing each workflow to enforce its own steps. Avoid abstracting away important business differences.

Câu 494. How would you handle errors in React components?

Answer (English): Use error boundaries for render failures, local handling for async/action errors, and consistent error states for data fetching. Error boundaries do not automatically catch all event handler or network errors.

Câu 495. How would you implement role-based UI without treating it as the security boundary?

Answer (English): Hide actions the user lacks for usability, but authorize the corresponding API/RPC and record access on the server. UI roles do not protect data from crafted requests.

Câu 496. How would you make a keyboard-driven interface more efficient for warehouse workers?

Answer (English): Support autofocus in scan fields, predictable Tab/Enter behavior, keyboard shortcuts, fast success feedback, and a safe escape/retry path. Never make destructive actions trigger accidentally on barcode suffix keys.

Câu 497. How would you make a React application accessible?

Answer (English): Use semantic elements, labeled controls, logical keyboard order, visible focus, accessible status announcements, contrast, and tested screen-reader paths. Warehouse devices

and gloves can demand larger touch targets.

Câu 498. How would you handle responsive layouts for desktop warehouse terminals and mobile devices?

Answer (English): Use responsive layouts with compact task-first mobile screens and richer desktop tables, ensuring scan controls remain visible and usable. Test real terminal/scanner resolutions and poor network conditions.

Câu 499. How would you write unit tests for React components?

Answer (English): Test behavior with React Testing Library and a runner, asserting rendered states and user interactions rather than internal implementation details. Include loading, success, validation, conflict, and accessibility cases.

Câu 500. How would you debug a UI that displays stale inventory data?

Answer (English): Check network/cache freshness, request ordering, invalidation after writes, projection lag, and browser state. Compare API data with a ledger-derived authoritative query before blaming React rendering.

PHẦN 13. TAILWIND CSS VÀ UI CHO NHÂN VIÊN KHO

Câu 501. What are the advantages and disadvantages of Tailwind CSS?

Answer (English): Tailwind offers fast, consistent utility styling and easy responsive rules; trade-offs include verbose markup and hard-to-manage ad hoc variants. A component system and tokens prevent inconsistency.

Câu 502. How do you organize reusable styles with Tailwind?

Answer (English): Define shared tokens for colors/spacing/typography and wrap repeated patterns in components. Keep contextual one-off utility classes local rather than creating excessive global CSS abstractions.

Câu 503. How would you implement a consistent design system with utility classes?

Answer (English): Create consistent tokens for stock statuses, action hierarchy, spacing, focus states, and typography; apply them through shared components. Validate contrast and accessibility across screen sizes.

Câu 504. How do you avoid duplicating long class strings?

Answer (English): Extract genuinely repeated UI patterns into components, use small variant utilities for controlled class combinations, and avoid giant hand-concatenated class strings. Keep domain semantics visible.

Câu 505. How do you handle conditional classes?

Answer (English): Use a class helper with explicit conditional variants, such as `clsx` or a variant utility, rather than arbitrary unvalidated string construction. Ensure Tailwind's build can discover used class names.

Câu 506. How do you make a dense inventory table easy to scan?

Answer (English): Prioritize SKU, bin, status, available quantity, and alerts; align numbers, support filtering, sticky headers, and highlighting of exceptions. Avoid relying only on color to indicate risk.

Câu 507. Which information should be visible immediately on a product detail screen?

Answer (English): Show canonical SKU/variant, barcode, sellable/on-hand/reserved quantities by location, status, latest movement, and relevant actions. Make sync freshness and discrepancy warnings clear.

Câu 508. How would you design a safe confirmation flow for a destructive or irreversible action?

Answer (English): Show a precise impact summary, require an explicit reason and any approval, and distinguish undoable operations from compensating corrections. Confirm the exact SKU/bin/quantity instead of a generic 'Are you sure?'.

Câu 509. How would you visually distinguish available, reserved, damaged, and quarantined inventory?

Answer (English): Use distinct textual labels, icons, and colors with accessible contrast; show availability as a breakdown rather than one ambiguous number. Reserve color warnings for exceptional states.

Câu 510. How would you prevent operators from confusing similar SKUs?

Answer (English): Display barcode, SKU, variant attributes, image, and bin, and confirm by scanning the expected item. Reject mismatched codes, especially where product names are similar.

Câu 511. How would you design a barcode-scanning workflow?

Answer (English): Keep a focused input that accepts scanner keyboard events, parse scanner suffixes, resolve a canonical SKU, verify task/bin expectations, and return immediate feedback. Support manual entry with additional validation.

Câu 512. How would you display a stale-data warning?

Answer (English): Display last updated time, connection/reconciliation status, and an explicit refresh action. Disable stock-critical commits only when the server cannot validate current state; never imply cached stock is guaranteed current.

Câu 513. How would you make errors actionable rather than merely showing “Something went wrong”?

Answer (English): Return specific guidance such as 'SKU does not match this pick task' or 'Only 2 sellable units remain', with retry, refresh, or escalation options. Hide internal SQL errors and tokens.

Câu 514. How would you design loading indicators for long-running warehouse operations?

Answer (English): Use local progress during short requests; for long jobs show a job ID, phase, count completed, errors, and a resumable status. Never show fake percent progress disconnected from real work.

Câu 515. How would you ensure critical actions are usable with a keyboard?

Answer (English): Make every critical action reachable by keyboard with visible focus, predictable shortcuts, and confirmation where needed. Test physical barcode-scanner suffix behavior to avoid accidental submission.

PHẦN 14. RETOOL VÀ INTERNAL TOOLS

Câu 516. What is Retool?

Answer (English): Retool is a low-code platform for building internal tools with data sources, UI components, queries, and workflows. It is useful for operational dashboards and forms when rapid delivery matters.

Câu 517. When would you use Retool rather than building an entire frontend from scratch?

Answer (English): Choose Retool for trusted internal CRUD, dashboards, and simple approvals where built-in components suffice. Choose custom React when workflows need complex scanning, offline behavior, deep accessibility, or fine-grained UX control.

Câu 518. What are the limitations of low-code platforms?

Answer (English): Low-code tools can limit complex interaction, automated testing, code organization, version control, and portability. Avoid putting core business invariants solely into scattered client-side scripts.

Câu 519. How would you connect a Retool application to PostgreSQL or a backend API?

Answer (English): Configure a tightly scoped Retool database or REST resource with protected credentials, then build parameterized queries/actions. Prefer backend APIs for inventory mutations rather than unrestricted direct SQL.

Câu 520. How do Retool queries, components, and event handlers interact?

Answer (English): Components display query results; user events invoke validated queries/workflows; results update state and trigger refetches. Keep sequencing explicit and surface errors rather than chaining fragile UI callbacks.

Câu 521. How would you pass selected product information from a table to a detail panel?

Answer (English): Use the table's selected row ID, fetch the detail by that stable ID, and show an empty state if none is selected. Treat displayed row data as stale input, not authority for mutations.

Câu 522. How would you refresh data after a warehouse operation?

Answer (English): After successful server confirmation, invalidate/trigger only affected queries and show the new balance; distinguish pending projection refresh from committed ledger data. Do not assume a button click succeeded.

Câu 523. How would you implement form validation in Retool?

Answer (English): Validate required fields/ranges before triggering a query and validate again in the backend/RPC. For adjustments, require reason, expected SKU/bin, and permission checks.

Câu 524. How would you prevent duplicate actions caused by repeated clicks?

Answer (English): Disable repeat interactions while pending but also send a persistent idempotency key enforced by the service/DB. Multiple Retool tabs or network retries can bypass UI-only protection.

Câu 525. How would you protect a Retool app so only authorized employees can perform stock adjustments?

Answer (English): Enforce SSO/authentication, role-specific app access, least-privilege resources, backend authorization, and scoped adjustment RPCs. Audit actor and reason for every stock mutation.

Câu 526. Why might it be dangerous to let an internal tool run arbitrary SQL with excessive database permissions?

Answer (English): Broad SQL permissions let mistaken or compromised internal queries bypass validations, delete history, or access cross-tenant data. Restrict privileges and expose narrow approved operations.

Câu 527. How would you avoid putting sensitive credentials into client-visible state?

Answer (English): Store credentials in protected resource settings or secret management, not client JS, local state, or URL parameters. Avoid logging sensitive response bodies and credentials.

Câu 528. How would you troubleshoot a Retool query that succeeds for an administrator but fails for an operator?

Answer (English): Compare identity, warehouse memberships, resource grants, RLS context, and RPC execution privileges using both roles. A query working for admin may hide a missing operator permission.

Câu 529. How would you decide which logic belongs in Retool and which belongs in a backend service?

Answer (English): Retool should orchestrate presentation and simple interaction; the backend/DB should own authorization, stock invariants, transactionality, and idempotency. This lets multiple clients use identical business rules.

Câu 530. How would you migrate a Retool prototype into a more customized React application?

Answer (English): Extract business calls behind a stable API, build feature parity with component/UI tests, run pilot operators side by side, then cut over gradually. Keep authorization and state transitions in the shared backend.

Câu 531. How would you test a workflow implemented using Retool?

Answer (English): Test data access, roles, form validation, successful commands, duplicate clicks, API errors, and stale data with controlled fixtures. Use actual staging services for transaction-critical paths, not only UI previews.

Câu 532. How would you document internal tools so non-engineers can operate them reliably?

Answer (English): Document purpose, input requirements, allowed roles, normal steps, common error resolutions, contacts, and recovery paths, with short screenshots where helpful. Keep it beside the versioned workflow.

Câu 533. How would you simplify a workflow that currently requires operators to navigate five different screens?

Answer (English): Observe the actual user task, eliminate repeated lookups, place scanning/confirmation together, reuse selected SKU context, and automate safe transitions. Measure task time and error rate before/after.

Câu 534. What would make you reject a low-code solution and build custom software instead?

Answer (English): Reject low-code when requirements demand sophisticated offline/scanning UX, strict interactive performance, complex multi-step state, versioned automated testing, or unsupported security control.

Câu 535. Have you used Retool, Appsmith, or Superblocks? If not, how would you become productive with one quickly?

Answer (English): I would answer honestly about direct low-code experience, then build one staging workflow: list products, adjust through secured RPC, handle authorization and errors, and write a concise runbook.

PHẦN 15. SHOPIFY ADMIN API, GRAPHQL VÀ E-COMMERCE

Câu 536. [ƯU TIÊN CAO] What is the difference between REST and GraphQL?

Answer (English): REST exposes resource-oriented endpoints with server-selected shapes; GraphQL lets clients request specific fields through a schema and queries/mutations. GraphQL avoids overfetching but needs cost/rate/error handling.

Câu 537. Why might Shopify expose GraphQL APIs for administrative operations?

Answer (English): GraphQL lets Shopify expose interconnected products, variants, metafields, inventory, and pagination through typed operations. Clients must request only needed fields and handle cost-based limits.

Câu 538. What is a GraphQL query?

Answer (English): A GraphQL query reads data according to a schema, selecting nested fields and passing typed variables. It can return partial data alongside errors.

Câu 539. What is a GraphQL mutation?

Answer (English): A GraphQL mutation requests a change, such as modifying product metadata. Check both top-level `errors` and mutation-specific `userErrors`, and account for partial outcomes.

Câu 540. What are fields, arguments, variables, and aliases in GraphQL?

Answer (English): Fields select response data; arguments configure fields; variables pass typed values separately; aliases rename response keys or request a field multiple ways. Reusable fragments reduce duplicate field selections.

Câu 541. What are GraphQL fragments?

Answer (English): Fragments are reusable field selections on a GraphQL type or interface. They keep related product/variant queries consistent without duplicating field lists.

Câu 542. What are edges and nodes in Relay-style pagination?

Answer (English): Relay-style connections return `edges` containing `node` and cursor, plus `pageInfo` for continuation. Shopify GraphQL commonly uses cursor pagination rather than offset pages.

Câu 543. How does cursor-based pagination work in GraphQL?

Answer (English): Request a bounded `first` page, store `endCursor`, and continue with `after: endCursor` while `hasNextPage` is true. Keep pagination order/query filters stable.

Câu 544. Why is cursor pagination useful when retrieving a large product catalog?

Answer (English): Cursors continue from a position without scanning large offsets and behave better on large, changing catalogs. Jobs still need deduplication because concurrent updates may affect traversal.

Câu 545. How would you fetch products and their variants from Shopify?

Answer (English): Use Admin GraphQL with necessary `read_products` scope and a query selecting product IDs, variants, SKU, and pagination fields. For large catalogs, evaluate Shopify bulk operations and persist checkpoints.

Câu 546. How would you handle product updates made in Shopify?

Answer (English): Treat webhooks as change notifications, enqueue reconciliation of the affected product, fetch its current authoritative fields, and upsert only owned data. Use version/timestamp checks to prevent stale overwrites.

Câu 547. How would you update product metadata without accidentally overwriting unrelated fields?

Answer (English): Use precise field-level mutations with the provider's current API version and optimistic/version checks if available. Do not send unrelated fields copied from stale snapshots.

Câu 548. What is the difference between a Shopify product and a product variant?

Answer (English): Shopify products group merchandising data; variants represent individual option combinations and typically carry variant-specific identifiers. Internal SKU mapping should target the correct variant/inventory identity.

Câu 549. How would you map an internal SKU to a Shopify variant?

Answer (English): Store `(shop_id,shopify_variant_gid) -> internal_sku_id` in a unique mapping table, with canonical SKU code as a secondary match key. Do not assume code uniqueness across shops.

Câu 550. How would you handle the same SKU appearing across different sales channels?

Answer (English): Separate the canonical internal SKU from multiple channel listings/account mappings. Define which fields are shared and which are channel-specific; do not let all channels overwrite the same fields.

Câu 551. How would you synchronize product title, description, images, and variant information?

Answer (English): Declare field ownership, consume changes via webhook plus scheduled reconciliation, normalize images/options carefully, and sync through idempotent jobs with error tracking. Respect the marketplace API version and limitations.

Câu 552. Which system should be the source of truth for a given product field?

Answer (English): Define ownership per field: internal system may own SKU identity, warehouse stock, and operational status, while Shopify owns storefront copy/merchandising, subject to agreed policy. Never assume one system owns every field.

Câu 553. How would you decide whether Shopify or the internal platform owns inventory quantities?

Answer (English): Choose one authority for stock movements, usually the internal warehouse ledger, and publish derived sellable quantities outward. If Shopify also accepts orders, ingest reservations/fulfillments and reconcile explicitly.

Câu 554. What could go wrong if two systems both treat themselves as the source of truth?

Answer (English): Two-way writes can cause oscillating updates, stale overwrites, duplicate movements, and hard-to-debug drift. Define source ownership and echo suppression, plus reconciliation instead of symmetrical blind updates.

Câu 555. How would you resolve conflicts between the internal product record and Shopify?

Answer (English): Compare source owner, timestamps/versions, and changed fields; accept only permitted changes, otherwise queue a conflict for review. Preserve both source values and decisions for audit.

Câu 556. How would you deal with Shopify API rate limits?

Answer (English): Honor API cost/throttle feedback and `Retry-After` where applicable, bound parallelism, use exponential backoff, and persist cursor/checkpoints. Do not retry permanent schema/userErrors indefinitely.

Câu 557. How would you implement pagination over thousands of products?

Answer (English): Traverse GraphQL pages with `pageInfo.endCursor` / `hasNextPage`, or use the provider's bulk export capability for very large collections. Checkpoint each page and upsert idempotently.

Câu 558. How would you resume a sync after the process crashes halfway through?

Answer (English): Persist cursor, sync run ID, last completed page, and affected external IDs. On restart resume safely and reprocess the last page using unique keys/upserts to avoid loss.

Câu 559. How would you detect products that have been deleted or archived in Shopify?

Answer (English): Use deletion/archive webhooks where available and periodic full/delta reconciliation. Mark mappings inactive without deleting historical orders or ledger references.

Câu 560. How would you handle API version changes?

Answer (English): Pin supported API versions, monitor deprecation notices, test queries/mutations in staging, and migrate fields and scopes before sunset. Contract tests should cover actual provider error shapes.

Câu 561. How would you protect Admin API credentials?

Answer (English): Keep Admin API tokens in server-side secret storage, use least-privilege scopes, rotate after compromise, and redact them in logs. Never ship them in frontend JavaScript.

Câu 562. What permissions should a Shopify integration have?

Answer (English): Request only required Shopify Admin scopes such as needed product/inventory reads and specific writes. Review permissions periodically and separate environments/stores where possible.

Câu 563. How would you test Shopify integration logic without affecting real product data?

Answer (English): Use a development store or approved test environment with isolated products, deterministic fixtures, contract tests, and strict safeguards around mutations. Never use real customer orders as a test harness.

Câu 564. How would you log integration failures without exposing secrets?

Answer (English): Log provider name, shop, operation, correlation ID, sanitized error type/status, and retry attempts. Omit access tokens and unnecessary customer/order personal data.

Câu 565. What would you do if Shopify returns HTTP 200 but the GraphQL response contains errors?

Answer (English): Parse the JSON body and check top-level GraphQL `errors` and operation-level `userErrors`, even when HTTP status is 200. Treat each error by its semantics rather than marking every 200 as success.

Câu 566. How would you handle partial success in a GraphQL response?

Answer (English): Inspect `data` and `errors` together, identify which fields/mutations succeeded, and retry only unresolved operations with stable keys. Do not blindly repeat already successful side effects.

Câu 567. How would you ensure that an update sent to Shopify does not create a feedback loop when the change returns through a webhook?

Answer (English): Tag outbound changes with origin/run ID where possible, use field ownership and deduplicate incoming webhooks, and reconcile to expected state. Suppress echoes instead of

issuing reciprocal updates endlessly.

Câu 568. How would you reconcile Shopify inventory with the internal warehouse inventory?

Answer (English): Compare derived internal sellable stock with published Shopify quantities per mapped inventory location; account for lag, holds, and in-flight orders. Investigate and repair through authoritative commands.

Câu 569. How would you handle different sales channels that update product information at different times?

Answer (English): Track update timestamps/versions per channel and field, and reconcile from the designated owner. Late updates from a non-owner cannot overwrite newer canonical state.

Câu 570. How would you build a reliable product syndication pipeline?

Answer (English): Use a canonical product schema, channel mappings, field ownership, durable outbox/jobs, bounded API concurrency, idempotent upserts, versioned transformations, and scheduled reconciliation with drift alerts.

PHẦN 16. WEBHOOKS VÀ ĐỒNG BỘ DỮ LIỆU

Câu 571. [ƯU TIÊN CAO] What is a webhook, and how does it differ from polling an API?

Answer (English): A webhook is a provider-initiated HTTP notification when an event occurs; polling periodically asks for changes. Webhooks reduce latency and needless requests but may be duplicated, delayed, or missed.

Câu 572. What are the advantages and disadvantages of webhook-based integrations?

Answer (English): Webhooks provide near-real-time updates but introduce signature verification, retry/order uncertainty, and endpoint uptime requirements. Combine them with polling/reconciliation for recovery.

Câu 573. Why should a webhook endpoint respond quickly?

Answer (English): Providers impose response deadlines and may retry on slow acknowledgements. Verify and durably store the event quickly, then process expensive work asynchronously.

Câu 574. Should a webhook handler perform all business processing synchronously?

Answer (English): Usually not. Verify signature, validate minimal envelope, store in an inbox transaction, and return 2xx; consumers handle business processing with retries and observability.

Câu 575. How would you validate the authenticity of an incoming webhook?

Answer (English): Compute the provider-defined HMAC over the exact raw request bytes using the shared secret and compare to the transmitted signature in constant time. Also validate topic, source, payload shape, and size.

Câu 576. How does HMAC signature verification work?

Answer (English): HMAC signs the exact message with a shared secret, allowing the receiver to recompute and verify integrity/authenticity. It does not provide replay protection by itself; event IDs and timestamp policies help.

Câu 577. Why must you pay attention to the exact request body when verifying a signature?

Answer (English): Re-serializing parsed JSON changes whitespace/order/escaping and therefore its signature bytes. Read the raw body before parsing and calculate HMAC over those original bytes.

Câu 578. How would you handle a webhook with an invalid signature?

Answer (English): Reject with an appropriate unauthorized/invalid response, record minimal diagnostic metadata, and do not enqueue the payload. Avoid disclosing the signing secret or detailed verification internals.

Câu 579. What should your endpoint return after successfully accepting a webhook?

Answer (English): Return 2xx only once the valid event is durably accepted or recognized as an already accepted duplicate. Never acknowledge before storage if losing the process would lose the event.

Câu 580. What happens if the webhook sender retries the same event?

Answer (English): A sender may deliver the same event multiple times; without dedupe it can double-change stock or reissue shipments. Persist the provider event ID under a unique constraint and return success for duplicates.

Câu 581. [ƯU TIÊN CAO] What is idempotency, and why is it essential for webhook processing?

Answer (English): Idempotency means applying the same logical command any number of times has one business effect. It is essential because network delivery is generally at least once and responses can be lost.

Câu 582. How would you design an idempotency table?

Answer (English): Create an inbox/operation table keyed by `(provider,account,event_id)` with payload hash, status, attempts, timestamps, and result/error. Claim/update it transactionally with business effects.

Câu 583. Which unique identifier would you use to detect duplicate events?

Answer (English): Prefer the provider's stable delivery/event ID scoped by shop/account and topic as documented. Use a separate business operation ID when distinct deliveries represent the same underlying transaction.

Câu 584. What if a webhook has no globally unique event ID?

Answer (English): Construct a carefully chosen semantic key from provider, entity ID, version/event time, and event type where available. A raw payload hash may dedupe identical bytes but cannot guarantee distinct business events are separated.

Câu 585. How would you prevent concurrent workers from processing the same event twice?

Answer (English): Use a unique inbox key and atomic job claim such as `FOR UPDATE SKIP LOCKED`, or lease with compare-and-swap; complete effects and processed marker transactionally where possible.

Câu 586. How would you record the processing status of a webhook?

Answer (English): Store `received`, `processing`, `completed`, `retry_pending`, `dead_letter`, with attempt count, last error, next retry time, and lease expiry. Make transitions atomic and observable.

Câu 587. What states would you use for a webhook-processing lifecycle?

Answer (English): A practical lifecycle is RECEIVED → CLAIMED/PROCESSING → COMPLETED or RETRY_PENDING; exhausted/permanent failures move to DEAD_LETTER. Expired leases return to eligible state without double effects.

Câu 588. How would you handle a webhook that fails repeatedly?

Answer (English): Classify error; cap retries with exponential backoff and jitter; preserve payload/evidence and move persistent failures to a dead-letter queue for investigation. Alert when failure/backlog thresholds are exceeded.

Câu 589. What is a dead-letter queue?

Answer (English): A dead-letter queue stores events that cannot complete normally after suitable attempts or because of permanent validation failures. It preserves evidence for review and controlled replay.

Câu 590. When would you move a failed event to a dead-letter queue?

Answer (English): Move to dead letter after bounded transient retries or immediately for known non-retryable failures that still require investigation. Never hammer a provider indefinitely with malformed requests.

Câu 591. How would you retry transient failures without retrying permanent validation errors indefinitely?

Answer (English): Retry timeouts, 429s, and temporary 5xx with capped backoff; reject invalid signatures/schema/unauthorized operations without endless retries. Preserve the exact failure category for operators.

Câu 592. How would you process webhooks that arrive out of order?

Answer (English): Use event versions/sequence numbers when available, or treat webhooks as a trigger to refetch current state. Prevent an older event from overwriting a newer processed version.

Câu 593. What if the product-update event arrives before the product-created event?

Answer (English): Stage the update until the product exists or refetch the product via API and upsert it. Do not assume order of create/update deliveries across the network.

Câu 594. How would you process an event whose referenced product does not exist locally?

Answer (English): Validate the external mapping, schedule a product import/reconciliation, then retry dependent processing. If the reference is truly invalid, dead-letter with context instead of creating an arbitrary SKU.

Câu 595. How would you prevent a delayed webhook from overwriting more recent data?

Answer (English): Track the latest accepted source revision or timestamp for owner-controlled fields, and discard/reconcile older updates. Prefer fetching current provider state over applying stale webhook snapshots.

Câu 596. What is the difference between event time and processing time?

Answer (English): Event time describes when an external business change occurred; processing time is when this system handled the notification. Latency and out-of-order behavior require both timestamps.

Câu 597. What if Shopify delivers the same webhook ten minutes after the original event?

Answer (English): The provider event ID should already exist in the inbox, so acknowledge the duplicate without repeating business effects. If the original attempt was incomplete, resume idempotent processing safely.

Câu 598. How would you reconcile missed webhooks?

Answer (English): Perform scheduled incremental syncs using provider cursors/update times plus periodic broader reconciliation; compare external IDs/versions and requeue missing changes. Retain watermarks and audit metrics.

Câu 599. Why should webhooks often be treated as notifications to reconcile state rather than the sole source of truth?

Answer (English): A webhook can be delayed, duplicated, reordered, or lost; it is often best used to wake up a fetch of current provider state. The owning system remains authoritative for the fields it controls.

Câu 600. How would you design periodic reconciliation jobs?

Answer (English): Schedule incremental scans with checkpoints and occasional full audits, compare records and field owners, repair small drift idempotently, and escalate large discrepancies. Expose lag, drift count, and last-success metrics.

Câu 601. How would you keep the internal database and external marketplace systems eventually consistent?

Answer (English): Use an outbox for outbound changes, durable webhook inbox for inbound changes, field ownership, retries, deduplication, and periodic reconciliation. Track version/watermark so divergence is measurable.

Câu 602. What is eventual consistency?

Answer (English): Eventual consistency means replicas/services may temporarily disagree but converge if communication resumes and reconciliation rules are correct. It is not permission to lose events or accept permanent drift.

Câu 603. When is eventual consistency acceptable in an inventory system?

Answer (English): Catalog descriptions, dashboard reports, and marketplace display stock may tolerate bounded lag. Internal reservation and picking decisions cannot rely on an eventually consistent stale balance.

Câu 604. Which operations require stricter consistency than others?

Answer (English): Stock allocation, transfer atomicity, authorization, and single-shipment issuance require strong local invariants. Notifications, reports, and external catalog sync can be asynchronous.

Câu 605. How would you stop a webhook retry from sending duplicate emails or triggering duplicate shipments?

Answer (English): Use a business-operation uniqueness key and an outbox message with a stable downstream idempotency key. A duplicate webhook must not create a second email/shipment regardless of delivery count.

Câu 606. How would you observe webhook latency, retry count, failure rate, and processing backlog?

Answer (English): Measure receive-to-complete latency percentiles, oldest pending age, queue depth, retries, dead letters, and permanent failure rate by provider/topic. Alert on actionable thresholds and monitor recovery progress.

Câu 607. How would you replay a failed webhook safely?

Answer (English): Replay from stored inbox payload or refetch provider state with the same event/business idempotency key; reset status under controlled conditions, and suppress already completed side effects.

Câu 608. How would you test duplicate, delayed, malformed, and out-of-order webhook deliveries?

Answer (English): Integration tests send repeated event IDs, invalid signatures, reordered revisions, delayed webhooks, and concurrent deliveries. Assert one business effect, safe rejection, correct final state, and recovery.

Câu 609. How would you prevent a webhook from triggering an infinite synchronization loop?

Answer (English): Track origin/change ID, define field owners, and ignore/reconcile webhook echoes of locally published changes. Do not perform symmetric writes merely because a notification arrived.

Câu 610. How would you recover from a multi-hour integration outage?

Answer (English): Keep a durable backlog, recover provider auth/connectivity, rate-limit catch-up processing, resume from checkpoints, run reconciliation, and compare expected vs observed state. Communicate degraded freshness during outage.

PHẦN 17. N8N VÀ TỰ ĐỘNG HÓA WORKFLOW

Câu 611. What is n8n?

Answer (English): n8n is a visual workflow automation platform connecting triggers, APIs, transformations, databases, and notifications. It works well for internal orchestration but does not replace transactional domain services.

Câu 612. When would you use n8n instead of writing a custom integration service?

Answer (English): Use n8n for relatively simple, observable cross-service workflows and low-volume notifications. Build a custom service when requirements need high throughput, strict atomicity, complex state, or advanced testing/version control.

Câu 613. What are the advantages and limitations of workflow automation tools?

Answer (English): Workflow tools accelerate integrations and make flows visible, but can create scattered business logic, brittle node chains, and permission/retry risks. Keep core invariants in audited services.

Câu 614. How would you connect Shopify, PostgreSQL, and an internal notification channel through n8n?

Answer (English): Use a Shopify webhook trigger, verify or route through a secure inbox, fetch/normalize product data, call a protected backend upsert API, then notify only on meaningful outcomes. Record job IDs and errors for recovery.

Câu 615. How would you pass data between nodes in an n8n workflow?

Answer (English): Nodes receive structured items; expressions or mapping nodes transform selected fields into the next node's input. Use explicit stable keys and validated transformations to avoid shape-related failures.

Câu 616. How would you handle an individual node failure?

Answer (English): Capture node error context, classify transient/permanent failure, and route to a retry or exception workflow. Ensure side effects before the failed node are not repeated blindly.

Câu 617. How would you implement retry and error-handling branches?

Answer (English): Use built-in retry options where appropriate plus a dedicated error branch, capped exponential delays, and an alert/dead-letter destination. Do not retry invalid credentials or schema errors forever.

Câu 618. How would you prevent an automation workflow from executing the same business action twice?

Answer (English): Generate/preserve a logical operation ID and enforce uniqueness in the backend database. Visual workflow retries cannot guarantee exactly-once effects by themselves.

Câu 619. How would you store and manage credentials securely in n8n?

Answer (English): Use protected n8n credentials/environment secrets, role access, rotation, and minimal scopes; do not put tokens into node JSON or logs. Restrict who can edit production flows.

Câu 620. How would you version and document a workflow?

Answer (English): Export/version workflow definitions, track credentials by reference, document input/output contracts, error paths, owners, and deploy changes through review/staging. Keep a runbook for failed executions.

Câu 621. How would you debug a workflow that works in development but fails in production?

Answer (English): Compare credential scopes, environment values, webhook URLs, network access, API versions, execution data and quotas. Reproduce with sanitized production-like inputs and inspect node-level errors.

Câu 622. When should business logic remain in a tested TypeScript service instead of moving into an automation workflow?

Answer (English): Keep stock adjustments, pricing ownership rules, access control, and event-ledger transactions in tested TypeScript/SQL services. n8n should call these safe commands rather than duplicate their logic.

Câu 623. How would you prevent an n8n workflow from bypassing database business rules?

Answer (English): Grant n8n only permission to call scoped backend APIs/RPCs; revoke general write access to ledger tables. Database constraints still guard against incorrect invocations.

Câu 624. How would you monitor failed executions?

Answer (English): Monitor error workflow notifications, execution age/count, retry backlog, last success, provider errors, and stale automations. Assign an owner and a specific replay process.

Câu 625. How would you replay a failed workflow without duplicating previous side effects?

Answer (English): Identify which nodes already caused side effects, reuse the original operation keys, and replay from a safe checkpoint or rerun idempotently. Verify external state before sending another shipment/notification.

PHẦN 18. TESTING — UNIT, INTEGRATION VÀ END-TO-END

Câu 626. [ƯU TIÊN CAO] What is the difference between unit testing, integration testing, and end-to-end testing?

Answer (English): Unit tests isolate pure logic; integration tests exercise components such as real PostgreSQL and APIs together; end-to-end tests follow actual user workflows across the stack. Critical inventory correctness requires more than mocks.

Câu 627. What should you test first when building a new inventory feature?

Answer (English): Start by specifying invariants and failure cases—stock cannot be oversold, duplicate submissions have one effect, transfers balance, permissions hold—then test pure rules and real DB transactions before UI polish.

Câu 628. How would you unit-test a function that calculates inventory balances?

Answer (English): Feed deterministic event sequences into a pure reducer, assert signed totals, empty history, corrections, and ordering behavior. Property tests can additionally assert transfer conservation.

Câu 629. How would you test that incorrect inventory events cannot change the balance?

Answer (English): Validate bad type, missing SKU, fractional units, unauthorized actor, invalid idempotency key, and forbidden negative balance. Assert no new ledger row and unchanged authoritative balance after rejection.

Câu 630. How would you test database triggers?

Answer (English): Run a migrated disposable PostgreSQL instance, execute writes that should fire the trigger, and assert affected rows and rollback behavior. Include forbidden direct writes and concurrent cases.

Câu 631. How would you test a PostgreSQL function?

Answer (English): Call the actual function with valid/invalid roles and stock levels, assert inserted events/result, and verify no partial writes on exceptions. Test SQLSTATE/error mapping as part of integration tests.

Câu 632. How would you test transaction rollback?

Answer (English): Begin a transaction, insert a valid movement, intentionally fail a later statement, roll back, and verify neither event nor side effect was committed. Use the real database.

Câu 633. How would you verify that two concurrent requests cannot oversell inventory?

Answer (English): Start two transactions concurrently against the same SKU with insufficient stock for both; verify at most one succeeds and final balance is nonnegative. Repeat under realistic isolation and multiple worker connections.

Câu 634. How would you test idempotency?

Answer (English): Submit the same operation key repeatedly and concurrently, assert one ledger movement and identical returned result. Different keys must still be able to create distinct legitimate operations.

Câu 635. How would you test duplicate webhook delivery?

Answer (English): Deliver the same signed webhook twice and concurrently, then assert one inbox record/business effect and valid acknowledgements. Also test retry after response loss.

Câu 636. How would you test out-of-order webhook events?

Answer (English): Send newer product version before older version and verify stale data does not overwrite it; test missing-create reconciliation. Store last applied revision and inspect final mapping.

Câu 637. How would you simulate Shopify API failures?

Answer (English): Use a controlled test provider or HTTP stub for status 429, 5xx, timeout, malformed JSON, and partial GraphQL errors. Assert bounded retries, safe deduplication, and useful error status.

Câu 638. What is the difference between mocking and using a real test database?

Answer (English): Mocks replace dependencies to test isolated logic; a real test database executes actual locking, SQL semantics, constraints, migrations, and transactions. Use both, but never trust mocks to prove stock concurrency safety.

Câu 639. When can excessive mocking make tests misleading?

Answer (English): A mock may confirm what test authors assumed rather than what PostgreSQL or Shopify actually does. Contract/integration tests catch schema drift, ordering, query errors, and concurrent behavior that fakes conceal.

Câu 640. How would you create a test database that resembles production?

Answer (English): Apply production migrations to a disposable PostgreSQL instance with representative synthetic/anonymized data volumes, extensions, policies, and indexes. Avoid exposing raw customer data to test environments.

Câu 641. How would you isolate tests so they do not affect each other's data?

Answer (English): Use isolated schemas/databases or transactional cleanup, create unique test keys, and never share mutable fixtures between parallel tests. Concurrency tests need dedicated connections and deterministic synchronization.

Câu 642. How would you test database migrations?

Answer (English): Apply migrations from scratch and from previous release snapshots, verify row/constraint invariants, old/new application compatibility, and rollback/forward-fix behavior. Include large-table lock/duration rehearsal.

Câu 643. How would you test that RLS policies correctly restrict warehouse operators?

Answer (English): Execute real DB requests as operators/managers across multiple warehouses and tenants; assert denied select, insert, update, and delete. Test escalation attempts through RPCs and role membership changes.

Câu 644. How would you test React components that depend on asynchronous API calls?

Answer (English): Use test HTTP responses for loading, empty, success, conflict, and failure; assert user-visible behavior and cancellation of stale reads. Cover keyboard/scanner interactions and accessibility.

Câu 645. How would you test a Retool workflow?

Answer (English): Test authorized/unauthorized resource access, input validation, duplicate clicks, query failures, and post-success refresh. Run against a staging service to validate actual DB effects.

Câu 646. How would you write a regression test for a bug that has already occurred?

Answer (English): Recreate the minimal failing input first, assert wrong behavior in a regression test, fix the root cause, and keep that test in CI. Include the production invariant the bug violated.

Câu 647. What is property-based testing?

Answer (English): Property-based testing generates many input sequences and checks invariant properties instead of fixed expected values only. It is valuable for discovering edge cases in state machines and event sequences.

Câu 648. How could property-based testing help validate inventory invariants?

Answer (English): Generate random valid receiving/picking/transfer/correction histories and assert deterministic replay, transfer conservation, no forbidden negative availability, and deduplicated event effects.

Câu 649. What is a contract test for an external API?

Answer (English): A contract test verifies a provider/client agrees on request and response shape, errors, auth, and version semantics. It helps detect Shopify API and internal endpoint compatibility changes.

Câu 650. How would you test a workflow that crosses React, backend services, and PostgreSQL?

Answer (English): Use an E2E flow that creates stock, receives it, reserves/picks, and displays the resulting React balance against real staging PostgreSQL. Assert consistent API responses and audit events.

Câu 651. What is a flaky test, and how would you diagnose one?

Answer (English): A flaky test fails nondeterministically because of timing, shared state, external dependencies, or order dependence. Capture seeds/logs, isolate fixtures, control asynchronous conditions, and fix rather than endlessly rerun.

Câu 652. How do you decide which tests should run on every pull request?

Answer (English): Run typecheck, lint, fast unit tests, focused DB migrations/integration tests, and high-value API contract tests on every PR. Schedule slower full E2E/replay/performance suites where appropriate.

Câu 653. What would you include in a critical production release test suite?

Answer (English): Include migration compatibility, critical stock workflows, concurrency and duplicate-key tests, auth/RLS, webhook error/retry tests, and deployment smoke checks. A release gate should match business risk.

Câu 654. How would you test a bulk inventory adjustment safely?

Answer (English): Use a staging ledger, preview selected rows, validate approved scope, apply within bounded atomic batches, and assert totals before/after. Test abort, retry, partial batch failure, and audit output.

Câu 655. How would you verify that replaying all events produces the expected final balance?

Answer (English): Replay all events into a separate projection and compare per `(sku,bin,status)` balance with an independently expected snapshot; test corrections/duplicates and ensure external side effects never fire.

PHẦN 19. PRODUCTION DEBUGGING VÀ REAL-WORLD PROBLEM SOLVING

Câu 656. [ƯU TIÊN CAO] The shelf says there are two items, but the system says there are three. How would you investigate?

Answer (English): First confirm SKU, bin, status, and count time; check UI freshness and reservations, recount physically, then replay ledger events and inspect recent picks/receipts/transfers. Correct only after determining cause.

Câu 657. What would you check before changing any inventory records?

Answer (English): Freeze scope/cutoff, identify affected stock keys and operations, verify authoritative event history, evaluate downstream orders, and preserve evidence. Determine whether data or display is wrong before making any write.

Câu 658. How would you determine whether the physical count or system balance is incorrect?

Answer (English): Have a second count, compare scans/shipments and cutoff timestamps, inspect event history and pending transactions. The physical observation may be wrong too, so don't

automatically force system state to the shelf count.

Câu 659. How would you trace a discrepancy back to the events that caused it?

Answer (English): Compute running balance for the affected SKU/bin, order by stable event sequence, and link unusual movements to receipt/pick/adjustment IDs and actors. Identify the first divergence from verified count evidence.

Câu 660. What if the discrepancy originated from a duplicate webhook?

Answer (English): Find duplicate source/event IDs and resulting movements; stop further duplicate ingestion, verify dedupe constraints, then append compensating events for proven duplicate effects with approvals and regression tests.

Câu 661. What if it came from a transaction that was partially processed by application logic?

Answer (English): Check transaction boundaries and external side effects; a proper PostgreSQL transaction cannot partially commit its own writes. If application operations span multiple transactions, reconcile each committed step and repair with audited commands.

Câu 662. What if a warehouse worker performed an operation twice?

Answer (English): Inspect scanner logs, actor/device IDs, request keys, and timestamps; determine whether it was one retry or two genuine actions. Repair the extra movement with a linked correction and improve UI/backend idempotency.

Câu 663. What if an external marketplace reported an outdated inventory value?

Answer (English): Compare external revision and timestamp with internal ledger/projection, identify source ownership, and avoid overwriting authoritative stock with stale marketplace quantities. Reconcile and republish the correct derived value.

Câu 664. How would you distinguish a data corruption issue from a stale UI issue?

Answer (English): Query authoritative ledger/projection directly and compare with API cache/network responses/UI state. If DB is correct but UI is stale, fix invalidation/freshness rather than posting stock adjustments.

Câu 665. How would you investigate a bug that occurs only in production?

Answer (English): Use production traces, feature flags, safe instrumentation, and sanitized reproduction data; compare environment configuration, concurrency, and real query plans. Minimize blast radius and avoid experimental mutations in live data.

Câu 666. How would you reproduce a production bug locally without exposing sensitive production data?

Answer (English): Reproduce with synthetic or anonymized records, matching schema/indexes/API versions and important workload characteristics. Never copy credentials or raw PII to a developer laptop.

Câu 667. What information should be included in an error log?

Answer (English): Include timestamp, severity, request/correlation ID, tenant/warehouse ID where permitted, operation, error code, duration, and safe context. Avoid secrets, raw PII, and unbounded payload logging.

Câu 668. Why should logs include correlation IDs?

Answer (English): Correlation IDs join frontend actions, API calls, DB/service logs, queues, and external requests into one traceable operation. They reduce investigation time when multiple requests overlap.

Câu 669. What is the difference between logs, metrics, and traces?

Answer (English): Logs record discrete events, metrics summarize numeric behavior over time, and traces follow latency and causal paths across services. Use all three for incidents affecting distributed workflows.

Câu 670. How would you investigate a sudden increase in API latency?

Answer (English): Compare p95 latency, error rate, recent deploys, upstream response time, query plans, connection waits, event-loop lag, and queue depth. Identify which span regressed before changing resources.

Câu 671. How would you investigate database connection exhaustion?

Answer (English): Inspect pool waiters/limits across replicas, `pg_stat_activity`, long-running/idle-in-transaction sessions, leaked clients, and slow queries. Limit concurrency and recycle leaked connections safely.

Câu 672. What would you do if a deployment caused a sudden spike in errors?

Answer (English): Pause rollout or disable the feature, restore the known-good release if schema-compatible, inspect errors and data impact, and communicate status. Validate invariants before

resuming normal operations.

Câu 673. How would you roll back a faulty release?

Answer (English): Redeploy a verified previous application artifact while preserving database compatibility; use feature flags for immediate mitigation. If schema/data changed irreversibly, use a forward fix or prepared compatibility path.

Câu 674. What if the release includes a database migration that cannot simply be reversed?

Answer (English): Keep the new schema if backward compatible and roll back code, or use a tested forward repair; do not blindly execute destructive down migrations. Verify inventory event and row-count integrity.

Câu 675. How would you stop the impact of a bug before fully understanding the root cause?

Answer (English): Disable the faulty writer/integration, narrow access, pause affected jobs, or place features in read-only mode. Preserve event evidence and continue unaffected operations where safely possible.

Câu 676. What does root-cause analysis mean?

Answer (English): Root-cause analysis seeks the specific technical and process mechanisms that produced an incident, not merely the last visible symptom. It should lead to testable prevention actions.

Câu 677. How would you write a postmortem after a production incident?

Answer (English): Document timeline, impact, detection gaps, root/contributing causes, safe recovery steps, and assigned follow-up actions. Keep a blameless tone and include verification that invariants were restored.

Câu 678. How would you prevent the same incident from occurring again?

Answer (English): Add regression/invariant tests, strengthen constraints/idempotency, improve monitoring/runbooks, and adjust the workflow or review process that let the bug escape. Track action completion and recurrence.

Câu 679. What is an incident runbook?

Answer (English): A runbook is an executable operational checklist for detecting, containing, diagnosing, recovering from, and verifying a known failure mode. It names owners and escalation paths.

Câu 680. What should a runbook include for a failed marketplace sync?

Answer (English): Include provider/job identifiers, credentials check, queue/backlog metrics, retry policy, checkpoint recovery, reconciliation queries, and proof that products and quantities have converged.

Câu 681. What should a runbook include for an inventory discrepancy?

Answer (English): Include SKU/bin/count cutoff, query to derive event balance, scan/pick/receipt checks, approval rules for adjustment, exact command, post-fix verification, and audit record.

Câu 682. How would you safely write a one-time SQL correction script?

Answer (English): Select an explicit bounded SKU/bin/time scope, preview affected records, require reason/approval, run in a short transaction, and write compensating events rather than updating historic ledger rows.

Câu 683. Why should a repair script first show exactly which rows it will affect?

Answer (English): A preview protects against missing filters, wrong tenant, unexpected joins, and excessive blast radius. It enables a reviewer to verify row identities and expected net change before commit.

Câu 684. How would you implement a dry-run mode?

Answer (English): Have the script build the exact candidate set and print counts, IDs, expected deltas, and no-write validation; only apply after explicit approval. Use the same selection query in dry-run and apply paths.

Câu 685. How would you prevent a repair script from accidentally updating every SKU?

Answer (English): Require tenant/warehouse filters, maximum row count, explicit allowlist/manifest, expected affected-row assertions, and transaction-scoped locks. Reject an empty or unexpectedly broad filter.

Câu 686. Why should correction scripts be repeatable or idempotent where possible?

Answer (English): Scripts may be interrupted or rerun, so the same correction key must not post a second adjustment. Use stable operation IDs and uniqueness constraints to make retries harmless.

Câu 687. How would you verify a correction after it has executed?

Answer (English): Recalculate balances from the ledger, compare before/after expectations for each affected key, verify downstream projections, and preserve a report of exactly what changed.

Câu 688. How would you document a manual production intervention?

Answer (English): Record incident/ticket, reason, approval, operator, cutoff, SQL/script version, selected IDs, before/after checks, and monitoring outcome. This provides accountability and repeatable recovery.

Câu 689. When should you escalate an incident instead of continuing to debug alone?

Answer (English): Escalate immediately for potential widespread stock corruption, security exposure, irreversible actions, unclear ownership, or risk beyond agreed limits. Share facts, impact, and safe containment, not guesses.

Câu 690. How would you communicate an incident to nontechnical warehouse operators?

Answer (English): Explain which tasks are affected, whether writes are paused, what staff should do instead, who owns recovery, and when status will be updated. Avoid speculative technical detail and provide clear operational instructions.

PHẦN 20. SECURITY VÀ DATA INTEGRITY

Câu 691. What are the most common security risks in a full-stack web application?

Answer (English): Major risks include broken authorization/IDOR, injection, XSS, CSRF, leaked secrets, insecure webhooks, weak RLS, dependency vulnerabilities, and accidental bulk mutations. Mitigate with layered validation, least privilege, and monitoring.

Câu 692. What is SQL injection, and how do parameterized queries prevent it?

Answer (English): SQL injection occurs when untrusted input changes SQL syntax. Bound parameters keep data separate from commands; dynamic table/column names need explicit allowlists.

Câu 693. What is cross-site scripting (XSS)?

Answer (English): XSS lets attacker-controlled content execute in another user's browser; React escapes ordinary text by default, but unsafe HTML injection still requires sanitization/CSP and careful handling.

Câu 694. What is cross-site request forgery (CSRF)?

Answer (English): CSRF tricks an authenticated browser into sending unwanted requests when credentials are automatically attached. Use same-site cookies, CSRF tokens or origin checks, and appropriate session architecture.

Câu 695. How do authentication and authorization differ?

Answer (English): Authentication establishes who the caller is; authorization determines what actions and resources that identity may access. Both must be validated on every protected server action.

Câu 696. How would you implement role-based access control for warehouse workers, managers, and administrators?

Answer (English): Define roles and fine-grained permissions such as `inventory.read`, `inventory.adjust`, `orders.ship`, scoped by warehouse. Managers may approve large adjustments while operators can only execute assigned tasks.

Câu 697. What is the principle of least privilege?

Answer (English): Least privilege grants only the permissions needed for a principal's job and limits lifetime/scope. It reduces damage from mistakes and compromised credentials.

Câu 698. Why is frontend-only authorization dangerous?

Answer (English): Client controls are editable and direct API calls bypass them. A backend without authorization can expose other warehouses or allow unauthorized inventory changes regardless of hidden buttons.

Câu 699. How should API keys and third-party tokens be stored?

Answer (English): Store tokens in server-side secret management with access control and rotation, not source code or public environment variables. Separate provider scopes and redaction rules by integration.

Câu 700. What would you do if an API key was accidentally committed to GitHub?

Answer (English): Revoke/rotate the secret immediately, inspect usage logs, remove it from active configs, assess exposure, and scrub history where feasible. Assume the key was compromised once publicly committed.

Câu 701. How would you rotate a compromised integration credential?

Answer (English): Provision a new credential, update secured deployments, test functionality, revoke the old one, and audit suspicious access. Coordinate rotation to minimize downtime and avoid exposing both keys.

Câu 702. How should webhook signatures be verified?

Answer (English): Use raw bytes and provider-specific HMAC/signature headers, constant-time comparison, and appropriate secret/timestamp policy. Reject invalid signatures before any business effects.

Câu 703. How would you prevent an unauthorized user from adjusting inventory?

Answer (English): Authorize authenticated role, tenant/warehouse membership, and adjustment limits in the service/database; require reason and audit entry. Restrict direct table writes and use an atomic RPC/transaction.

Câu 704. How would you protect bulk database operations against accidental misuse?

Answer (English): Require explicit scope, maximum affected rows, preview/dry-run, approvals, transaction checks, and immutable audit events. Bulk changes should be reversible through compensating records where possible.

Câu 705. What sensitive information should not be included in application logs?

Answer (English): Exclude passwords, API keys, cookies, auth headers, payment details, customer addresses/contact information, and unnecessary raw payloads. Retain only minimal identifiers needed for debugging.

Câu 706. How would you protect personal information in order and returns records?

Answer (English): Collect only needed information, restrict access by role/warehouse, encrypt transport/storage appropriately, set retention policies, and redact exports/logs. Avoid copying PII to testing or AI services.

Câu 707. How would you secure internal tools that connect directly to a database?

Answer (English): Use SSO/MFA as supported, least-privilege database/RPC resources, role-based access, audit trails, and isolated production credentials. Never allow arbitrary SQL with administrative permissions.

Câu 708. What security concerns should you consider before enabling a database function for authenticated users?

Answer (English): Review EXECUTE grants, SQL injection through dynamic SQL, `SECURITY DEFINER`, search path, RLS interactions, parameter validation, and cross-tenant access. Test with the least privileged role.

Câu 709. How would you prevent a user from accessing another warehouse's data?

Answer (English): Require warehouse membership in API authorization and RLS `USING` / `WITH CHECK`, including joins and RPC access. Test direct requests for another warehouse's IDs.

Câu 710. How would you audit inventory adjustments performed by administrators?

Answer (English): Every adjustment records the authenticated administrator, reason, approval, affected SKU/bin, signed delta, timestamp, and correlation/source ID. Audit logs and ledger links must be immutable.

Câu 711. How would you protect production data when debugging?

Answer (English): Use read-only credentials by default, sanitized data, bounded queries, and controlled access approvals. Avoid direct production writes unless executing a reviewed audited repair procedure.

Câu 712. How would you safely handle third-party payloads?

Answer (English): Treat external payloads as `unknown`, authenticate source, enforce size/schema/identifier limits, normalize values, and avoid trusting embedded URLs or instructions. Store provenance and process idempotently.

Câu 713. How would you validate file uploads if the product system supports product images or documents?

Answer (English): Check content length, MIME/signature, allowed extensions, dimensions where relevant, and malware scanning as risk warrants. Store outside executable paths with randomized names and strict access permissions.

Câu 714. How would you handle dependency vulnerabilities?

Answer (English): Pin/lock dependencies, automate vulnerability scanning, assess exploitability, patch promptly, and test compatibility before deployment. Minimize unused packages and monitor advisories.

Câu 715. How would you respond if you discovered that an internal tool had excessive database permissions?

Answer (English): Remove excessive grants, rotate credentials if necessary, inspect audit logs for misuse, and replace direct SQL access with scoped APIs/RPCs. Add negative permission tests and communicate the change.

PHẦN 21. CI/CD, GIT, DEPLOYMENT VÀ FLY.IO

Câu 716. Describe your normal Git workflow from starting a task to merging a pull request.

Answer (English): Create a short branch, clarify requirements, implement with tests/migration reviews, open a focused PR with risks and screenshots/SQL plan, address review, merge via CI, and monitor deployment.

Câu 717. What makes a good pull request?

Answer (English): A good PR clearly states problem, chosen approach, changed contracts/schema, testing evidence, rollout/rollback, risks, and affected business invariants. Small, reviewable diffs help catch mistakes.

Câu 718. How large should a pull request be?

Answer (English): Keep a PR small enough to review as one coherent behavior change, often a focused vertical slice. Split mechanical refactors/migrations from unrelated features while keeping atomic functionality testable.

Câu 719. How do you resolve merge conflicts?

Answer (English): Fetch latest changes, understand both edits, resolve deliberately, run type checks and tests, and re-review affected SQL/contracts. Never resolve a schema conflict by deleting an unfamiliar migration.

Câu 720. What is the difference between merging and rebasing?

Answer (English): Merge creates a merge commit combining branches; rebase reapplies commits onto a new base for linear history. Avoid rewriting shared/public history without team agreement.

Câu 721. How do you handle a pull request that introduces a database schema change?

Answer (English): Include schema diff and generated SQL, backward-compatibility plan, data migration/backfill strategy, locks, index effects, tests, and release order. Ensure migration has an explicit owner and staging rehearsal.

Câu 722. What should a pull request template contain?

Answer (English): Include summary, linked issue, acceptance criteria, risks, DB changes, test commands/results, screenshots/API samples where useful, and deployment/rollback instructions.

Câu 723. What automated checks should run before merging?

Answer (English): Run formatting/linting, TypeScript typecheck, unit/integration tests, migration checks, security scans, and relevant build/E2E tests. Require approval for high-risk stock or security code.

Câu 724. How would you prevent unreviewed code from reaching production?

Answer (English): Protect main branches with required passing CI, reviewer approvals, CODEOWNERS for critical paths, restricted release permissions, and audited emergency override procedures.

Câu 725. What is continuous integration?

Answer (English): Continuous integration automatically builds and tests changes on commits/PRs, detecting compatibility and correctness failures before merge. It supports, but does not replace, code review.

Câu 726. What is continuous deployment?

Answer (English): Continuous deployment automatically releases passing changes to production; continuous delivery may stop for manual approval. High-risk inventory migrations may appropriately require a controlled gate.

Câu 727. How would you design a CI/CD pipeline for a TypeScript application?

Answer (English): Install locked dependencies, lint/typecheck, run unit+database integration tests, build immutable artifact, stage migrations, deploy with health checks, smoke-test and monitor. Keep secrets out of CI logs.

Câu 728. What is the difference between development, staging, and production environments?

Answer (English): Development is for rapid coding, staging mirrors production for verification, and production serves real users/data. Separate credentials, tenants, stores, and migration tracking to prevent test mutations affecting real inventory.

Câu 729. How should environment variables differ between environments?

Answer (English): Each environment has its own DB, secrets, providers/test stores, URLs, logging levels, and feature flags. Validate typed configuration and forbid production keys in local builds.

Câu 730. How would you deploy a backend service without interrupting active operations?

Answer (English): Deploy health-checked instances gradually, ensure backward-compatible schema, drain old connections/workers, and monitor error/queue metrics. Avoid removing schema fields while old instances still run.

Câu 731. What is a health check?

Answer (English): A health check is a route or probe reporting whether a process/service is functioning for orchestration and operations. It should be fast and avoid performing expensive queries.

Câu 732. What is the difference between liveness and readiness checks?

Answer (English): Liveness answers whether a process should be restarted; readiness answers whether it can safely receive traffic, e.g. configuration and critical dependencies are usable. Do not make readiness depend on fragile optional integrations.

Câu 733. What is a deployment rollback?

Answer (English): Rollback means restoring the previous known-good application release when possible. Database changes may require compatibility or a forward repair rather than reverting data.

Câu 734. What is a blue-green deployment?

Answer (English): Blue-green deploys maintain two environments and switch traffic after validating the new version. They simplify application rollback but require compatible shared database schemas.

Câu 735. What is a canary release?

Answer (English): Canary releases expose a small fraction of traffic/work to the new version while monitoring outcomes, then expand gradually. For inventory writers, correctness checks and consistent schema behavior remain critical.

Câu 736. How would you deploy an application when the new code depends on a database migration?

Answer (English): Use expand-and-contract: deploy compatible schema, backfill/test, roll out new code, then remove old fields later. Separate migration execution from per-instance app startup.

Câu 737. What is Fly.io?

Answer (English): Fly.io is a platform for running applications on distributed compute machines with deployment, networking, and secret/configuration features. It is suitable for containerized TypeScript services.

Câu 738. How would you deploy a TypeScript service on Fly.io?

Answer (English): Build a production container for Node/Deno, define service ports/health checks in Fly configuration, set secrets, deploy via CI, and validate logs and connectivity. Handle migrations separately and carefully.

Câu 739. What would you check if a Fly.io application works locally but fails after deployment?

Answer (English): Check required env/secrets, listening address `0.0.0.0` and expected port, database/network permissions, build target, startup command, and health checks. Inspect release and machine logs.

Câu 740. How would you inspect application logs and diagnose startup failures?

Answer (English): Inspect platform log streams and events for startup errors, missing env, port mismatch, and database timeouts; reproduce with the same image/config in staging. Verify the machine stays healthy.

Câu 741. How would you manage secrets in a deployed service?

Answer (English): Set platform-managed secrets with restricted access, rotate them, and avoid embedding them in images, source control, or build logs. Prefer separate credentials for each environment.

Câu 742. How would you handle database connection limits from multiple application instances?

Answer (English): Calculate global connection budget across machines/workers, cap local pools, use an appropriate pooler, and monitor waits. Horizontal scaling can worsen DB exhaustion unless connection usage is controlled.

Câu 743. How would you scale a service that processes webhooks?

Answer (English): Scale stateless webhook receivers separately from durable processing workers, add queue capacity/backpressure, tune per-provider concurrency, and preserve idempotent job claims.

Câu 744. How would you determine whether scaling horizontally would help?

Answer (English): Measure CPU, event-loop delay, latency, DB/queue bottlenecks, and upstream rate budgets. Add replicas only if the bottleneck is parallelizable and external dependencies have capacity.

Câu 745. How would you keep deployment configuration documented and reproducible?

Answer (English): Version Docker/build steps, Fly config, env variable names, migration commands, health checks, rollback steps, and runbooks alongside the repository. Never include actual secrets in documentation.

PHẦN 22. SYSTEM DESIGN — BÀI PHỎNG VẤN KIẾN TRÚC 60 PHÚT

Câu 746. [ƯU TIÊN CAO] Design an inventory management platform for a business operating several warehouses and online sales channels.

Answer (English): Design tenant-scoped catalog/SKU mappings, warehouse/bin ledger, orders/reservations/fulfillment, role-gated APIs, operator UI, webhook inbox/outbox, worker queues, and reconciliation. Strong consistency protects local stock; external channels converge asynchronously.

Câu 747. What functional requirements would you clarify before designing the system?

Answer (English): Clarify facilities, stock statuses, reservation lifecycle, lot/serial tracking, order source, partial picks, returns, audit requirements, and exception workflows before choosing a schema.

Câu 748. What nonfunctional requirements would you clarify?

Answer (English): Clarify throughput, p95 latency, uptime, recovery objectives, permissible stock-sync lag, data retention, security, number of tenants/warehouses, and operational support model.

Câu 749. How would you estimate data volume and request volume?

Answer (English): Estimate SKUs × bins × movements/day, order peak bursts, webhook rates, average payload size, growth/retention, and concurrent operators. Use estimates to drive index/partition/queue needs, revisiting real metrics later.

Câu 750. What are the system's authoritative data sources?

Answer (English): Define field-level ownership: internal inventory ledger for physical movements, order/marketplace systems for originating orders, and channel-specific owners for storefront fields.

Derived balances are not independent authoritative records.

Câu 751. How would you divide the system into frontend, backend, database, and external integrations?

Answer (English): React/Retool provides workflow UI, typed Node/Deno/Hono APIs authorize/validate commands, PostgreSQL enforces transactions/ledger, and workers handle Shopify/n8n synchronization using inbox/outbox.

Câu 752. How would you design the core relational schema?

Answer (English): Use normalized entities for products, variants/SKUs, warehouses, bins, orders/lines, picks, receipts, returns, reservations, events, channel mappings, jobs, and inbox/outbox. Add uniqueness, FKs, checks, and indexes.

Câu 753. How would you design the event ledger?

Answer (English): Append immutable signed movement rows with actor, source operation, idempotency key, location/status, and dual timestamps. Corrections are compensating events; projections are rebuildable.

Câu 754. How would you calculate inventory balances?

Answer (English): Compute `SUM(delta)` grouped by SKU/bin/status and subtract active commitments according to agreed semantics. Maintain fast projections with replay/reconciliation, but validate mutations against serialized authoritative state.

Câu 755. How would you prevent overselling?

Answer (English): Serialize stock-changing commands per stock key, recheck available stock under lock, atomically record reservation/movement, and reject insufficient requests. Retry using unique idempotency keys.

Câu 756. How would you handle simultaneous order creation and warehouse picking?

Answer (English): Order creation reserves stock while picking consumes reservation and transitions tasks under ordered locks. Enforce line-level totals and never allow both flows to decrement availability independently.

Câu 757. How would you synchronize product information across multiple marketplaces?

Answer (English): Use canonical product mapping, field ownership, webhooks as hints, durable outbox, bounded API retries, and scheduled reconciliation. Prevent stale webhook echoes and

preserve external IDs.

Câu 758. How would you design a reliable webhook ingestion pipeline?

Answer (English): Verify raw-body signature, durably dedupe in inbox, return fast 2xx, process with leased workers, classify failures, and expose dead-letter/reconciliation tooling.

Câu 759. How would you process long-running synchronization tasks?

Answer (English): Persist import/sync jobs with cursors and stages, process bounded batches asynchronously, checkpoint progress and metrics, and resume after worker crashes.

Câu 760. How would you make failed tasks retryable?

Answer (English): Store job status, attempt count, lease, next run, and durable operation ID. Retry transient failures with backoff, route permanent failures to dead letter, and make steps idempotent.

Câu 761. How would you prevent duplicate side effects?

Answer (English): Use uniqueness constraints, inbox/outbox operation IDs, downstream provider idempotency support, and transactional state transitions. Distinguish message redelivery from a genuinely new business command.

Câu 762. How would you handle external API rate limits?

Answer (English): Use provider-specific token/cost budgets, backpressure, retry-after handling, jitter, and checkpoints. Avoid flooding the integration immediately after an outage.

Câu 763. How would you distinguish transient failures from permanent failures?

Answer (English): Transient failures include timeouts, 429, and temporary 5xx; permanent ones include invalid schema, authorization, or unsupported operations unless conditions change. Classification drives retry versus intervention.

Câu 764. How would you design auditability into the system?

Answer (English): Store append-only events with actor, reason, references, and timestamps, plus approval records and structured request tracing. Ensure repair scripts also create auditable changes.

Câu 765. How would you protect against accidental bulk updates?

Answer (English): Require explicit tenant/SKU scope, approval, preview, max batch size, idempotency, and guarded transactions. Do not expose unrestricted SQL write tools to operators.

Câu 766. How would you design authorization for operators, product managers, and administrators?

Answer (English): Use scoped RBAC plus RLS where appropriate, per-warehouse membership, separate adjustment/approval capabilities, and tests for cross-tenant access. All API/RPC calls reauthorize server-side.

Câu 767. How would you keep operational dashboards responsive?

Answer (English): Serve indexed paginated queries and rebuildable projections with freshness markers, avoiding full ledger scans per request. Isolate expensive reporting from stock-critical transactions.

Câu 768. How would you design a reporting layer without making writes slower?

Answer (English): Use read replicas/materialized/incremental reporting projections, asynchronous refresh, and query limits. Keep authoritative writes lean and verify reports against ledger periodically.

Câu 769. What would you cache, and what would you never trust from a cache?

Answer (English): Cache product detail, counts for dashboards, and reference data with clear TTL/invalidation; never accept cached stock as authority for final reservations/adjustments.

Câu 770. What would happen when the external marketplace is unavailable for several hours?

Answer (English): Queue outbound changes, show sync lag, retain ordered checkpoints, and continue safe internal operations according to policy. On recovery, rate-limit catch-up and reconcile actual external state.

Câu 771. How would you recover when the internal database and external marketplace diverge?

Answer (English): Compare IDs/versions and field owners, find missing/failed jobs, replay idempotently, and produce a discrepancy report. Stock corrections must flow through ledger commands rather than direct overwrites.

Câu 772. How would you scale the event table?

Answer (English): Use targeted indexes, projections, time/tenant partitions only when justified, and archive-with-replay infrastructure. Monitor write overhead and maintain reliable event reconstruction.

Câu 773. How would you structure background workers?

Answer (English): Separate ingress, stock command, provider sync, report, and reconciliation workers by workload and concurrency/rate budget. Use durable jobs, leases, retries, and dead letters.

Câu 774. How would you monitor the health of the whole platform?

Answer (English): Monitor API latency/error rate, DB locks/connections, ledger/projection invariants, queue lag, webhook failures, Shopify throttling, and physical-vs-system discrepancy rates. Alert with runbooks and ownership.

Câu 775. What parts of the design need strong consistency?

Answer (English): Strong consistency is essential for local stock allocation, paired transfers, ledger writes, uniqueness, order transition, and authorization. Enforce these inside PostgreSQL transaction boundaries.

Câu 776. What parts could tolerate eventual consistency?

Answer (English): Eventual consistency is acceptable for cross-channel catalog propagation, dashboards, search indexing, notifications, and external published stock within agreed freshness limits.

Câu 777. How would you deploy a breaking schema change safely?

Answer (English): Stage schema additions, dual-read/write/backfill if required, deploy compatible clients, validate counts/invariants, and remove old schema only after old versions drain. Test rollback compatibility.

Câu 778. What are the main failure modes of this architecture?

Answer (English): Risks include duplicate/out-of-order webhooks, stock races, DB outage, lost external acknowledgements, stuck workers, bad migrations, stale projections, and operator scan mistakes. Design detectors and recovery for each.

Câu 779. What trade-offs would you make if the engineering team is very small?

Answer (English): Start with a modular monolith and one PostgreSQL database, a simple durable job/outbox path, strong constraints, and a focused UI. Avoid premature microservices and partitioning until scale demands them.

Câu 780. How would you prioritize reliability versus speed of implementation?

Answer (English): Protect correctness and recoverability first, then deliver small slices with measured performance. Reduce scope rather than bypass idempotency, stock locks, authorization, or migration safeguards.

PHẦN 23. BÀI SYSTEM DESIGN CHUYÊN SÂU

Câu 781. Design a reliable product synchronization service between Shopify and an internal PostgreSQL database.

Answer (English): Build a Shopify adapter that fetches authoritative product versions, maps external variants to internal SKUs, upserts owned fields under unique keys, and queues outbound updates via an outbox. Webhooks trigger sync, while polling/reconciliation catches missed changes.

Câu 782. How would you decide whether to use polling, webhooks, or both?

Answer (English): Use webhooks for responsiveness and polling for reconciliation/backfill or APIs without reliable notifications. Choose frequency from freshness needs, API limits, and outage recovery targets.

Câu 783. How would you recover from a missed webhook?

Answer (English): Store last successful sync watermark, query for updated entities with overlap, deduplicate upserts, and perform periodic full audits. A missed webhook should not create permanent divergence.

Câu 784. How would you handle a product updated simultaneously in two systems?

Answer (English): Define an owner per field, compare versions/timestamps, and never blindly apply last-writer-wins across unrelated clocks. Queue genuine ownership conflicts for resolution and log the decision.

Câu 785. Design a service that imports 100,000 products without exceeding API rate limits.

Answer (English): Use Shopify's appropriate bulk/paginated API, bounded parallelism and rate budget, durable cursor/checkpoints, staged validation, and batch upserts. Resume by idempotent unique external mapping keys rather than restarting blindly.

Câu 786. How would you resume the import after a crash?

Answer (English): Persist job run ID, cursor, last completed batch, and record-level outcomes transactionally. On restart reprocess the last batch with upsert keys to avoid loss or duplication.

Câu 787. How would you ensure repeated imports do not produce duplicate records?

Answer (English): Use `UNIQUE(platform,shop_id,external_variant_id)` and explicit canonical match rules; stage raw imports, upsert owned fields, and investigate ambiguous SKU matches rather than guessing.

Câu 788. Design an event-processing service capable of processing duplicate events safely.

Answer (English): Persist each provider event in a uniquely keyed inbox, atomically claim work, guard downstream business operation IDs, and mark complete only after committing effects. Concurrent duplicates return the previous outcome.

Câu 789. How would you support replaying historical events?

Answer (English): Archive immutable events with schema versions, replay into a new projection at a known checkpoint, and compare a deterministic checksum/balance. Keep a controlled watermark for catch-up.

Câu 790. How would you prevent replay from triggering duplicate external actions?

Answer (English): Separate pure state projections from command handlers, suppress outbound notifications during replay, and retain original operation IDs/outbox dedupe keys. Verify no shipments/emails are resent.

Câu 791. Design a safe bulk inventory adjustment tool.

Answer (English): Provide upload/template parsing, server-side SKU/bin validation, preview of old/new projected balances, approval, and a scoped idempotent apply job that posts ledger adjustments. Keep immutable audit reports.

Câu 792. How would you allow previewing the impact before applying a bulk adjustment?

Answer (English): Create a read-only dry-run that resolves actual IDs, expected deltas, projected negative balances, affected count, and approval threshold. The confirmed apply must verify the preview's revision/cutoff to detect stale results.

Câu 793. How would you implement approval for unusually large adjustments?

Answer (English): Apply role-based thresholds, two-person approval for high-risk changes, explicit reason/ticket, and time-limited signed approval tied to the exact preview manifest. A changed manifest requires new approval.

Câu 794. Design an inventory reconciliation process that compares database quantities with physical counts.

Answer (English): Capture count sessions and a consistent movement cutoff, compare physical observations to ledger-derived stock per SKU/bin/status, investigate mismatches, and post linked compensating adjustments only after review.

Câu 795. How would you report discrepancies and track their resolution?

Answer (English): Show old balance, counted quantity, difference, likely related movements, affected orders, assignee, reason, approvals, and resolution state. Track recurring discrepancies by location/process.

Câu 796. Design a workflow that receives products, assigns bins, and updates their availability.

Answer (English): Receive to pending/QC, scan actual quantities, assign storage bin based on rules, transfer from receiving to bin, and release to sellable only after checks. Each step has actor, idempotent command, and event trace.

Câu 797. How would you handle partial failure during receiving?

Answer (English): Keep received lines and progress independently; post only completed atomic batches, resume partially processed receipt tasks, and reconcile unaccepted units. Never mark the entire receipt complete after one partial write failure.

Câu 798. Design an interface for a warehouse worker using a barcode scanner.

Answer (English): Show a single prominent scan field, expected SKU/bin/quantity, immediate feedback for correct/mismatched scans, task progress, and exception actions. Backend validates every scan against assigned work.

Câu 799. How would you avoid creating two shipments after a worker double-clicks a button?

Answer (English): Use a stable shipment command key and unique `(order_id,shipment_request_id)` plus provider-side idempotency when supported. Both clicks retrieve the same shipment rather than generating two labels.

Câu 800. Design an internal dashboard showing inventory, order status, returns, and integration failures.

Answer (English): Show indexed summaries of stock by status, order/pick queues, returns requiring inspection, failed sync jobs, and exception age. Provide links to exact event/order records and freshness timestamps.

Câu 801. How would you keep the dashboard up to date without running expensive queries on every page load?

Answer (English): Use incremental projections, materialized reporting with scheduled refresh, cached filtered queries, and websocket/SSE invalidation where useful. Critical mutation reads still consult the authoritative state.

Câu 802. Design a job queue for background processing. What state transitions would a job have?

Answer (English): Jobs transition `queued→leased/running→succeeded` or `retry_pending→queued`; exhausted/invalid jobs go to `dead_letter`. Store lease expiry, attempts, next run, error, payload reference, and idempotency ID.

Câu 803. How would you handle a job that repeatedly crashes?

Answer (English): Release expired leases, cap retries with jitter, classify permanent failures, and preserve job state for investigation. Alert on stuck jobs and make replay an explicit authorized operation.

Câu 804. How would you make a database migration safe while old and new application versions are temporarily running?

Answer (English): Use expand-and-contract schema changes, old/new compatible API releases, staged backfill, and deprecation after old workers stop. Rehearse both versions against the intermediate schema.

Câu 805. Design an audit trail for every stock adjustment made by employees or automated integrations.

Answer (English): Every adjustment event records actor/service, reason, approval/ticket, tenant, SKU/bin, delta, source operation, and timestamps; link corrections to originals. Restrict mutation and support audit queries.

PHẦN 24. PRODUCT SYSTEMS CHAT — HIỂU NGHIỆP VỤ VÀ QUY TRÌNH KHO

Câu 806. What do you understand about our business and the problem we are trying to solve?

Answer (English): The product appears to combine internal product catalog and physical warehouse workflows with multiple marketplace connections. The central challenge is trustworthy, traceable inventory and reliable integrations without slowing operators; confirm details with the team.

Câu 807. Why does inventory accuracy matter to our business?

Answer (English): Incorrect stock causes overselling, canceled orders, unnecessary recounts, delays, lost trust, and financial reconciliation problems. Accurate per-location/status balances make fulfillment dependable.

Câu 808. What is the difference between product management and inventory management?

Answer (English): Product management handles descriptions, variants, identifiers, channel listings, and lifecycle; inventory management tracks quantities, locations, status, movements, reservations, and fulfillment.

Câu 809. What is the difference between a product, a variant, and a SKU?

Answer (English): A product is the overall item concept; a variant defines a particular configuration; a SKU is the internal stock-keeping identifier for a tracked unit/variant. Their exact relationship depends on the catalog's fulfillment rules.

Câu 810. Why might one product have multiple SKUs?

Answer (English): Different sizes/colors, packaging sizes, bundles, or region-specific stock-keeping units may need separate SKUs. Never assume one merchandised product equals one physical inventory identity.

Câu 811. What does receiving mean in a warehouse workflow?

Answer (English): Receiving is confirming that expected goods physically arrived, scanning identifiers and quantities, documenting discrepancies/conditions, and posting accepted stock into the appropriate receiving status.

Câu 812. What is put-away or binning?

Answer (English): Put-away is moving accepted received goods from receiving to a suitable storage location/bin, verifying the destination and recording the movement so pickers can find them.

Câu 813. What is the difference between picking and packing?

Answer (English): Picking retrieves the required units from storage for an order; packing verifies and prepares those picked units for shipment. They have different locations, responsibilities, and validation rules.

Câu 814. Why should picking and packing be separate workflow steps?

Answer (English): Separating them catches wrong/missing items before shipping, supports partial picks and quality checks, and records operational accountability. The system must enforce `packed <= picked`.

Câu 815. What is a return authorization?

Answer (English): A return authorization grants permission and defines conditions for a customer return, linking expected items to the original order. It does not mean returned items have been physically received or approved for resale.

Câu 816. Why should returned products not automatically become sellable inventory?

Answer (English): Returned stock may be damaged, missing parts, counterfeit, or require cleaning/QC. Only inspected resellable units should re-enter sellable inventory.

Câu 817. How would you handle a return that is damaged or missing components?

Answer (English): Inspect condition, record evidence/missing components, place goods into quarantine/damaged status, and choose repair/refund/disposal policy. Do not add to ATP until approved.

Câu 818. What problems arise when an item moves between warehouse bins but the system is not updated?

Answer (English): The old bin appears overstated and the new bin understated, causing failed picks, unnecessary replenishment, and false discrepancy reports. Scan-enforced transfer events prevent the mismatch.

Câu 819. Which warehouse operations are most vulnerable to human error?

Answer (English): High-risk operations include receiving overages, wrong-bin put-away, picking lookalike SKUs, duplicate scans, returns disposition, and manual adjustments. Instrument errors and create confirmations at those boundaries.

Câu 820. How could software reduce scanning errors and incorrect product selection?

Answer (English): Require barcode verification against task/expected bin, provide visual and audible mismatch feedback, reduce free-text entry, and confirm quantities. Track error patterns without penalizing normal exception reporting.

Câu 821. How would you discover the actual needs of warehouse workers before building an interface?

Answer (English): Shadow real operators, observe device/scanner constraints and peak workflows, map exceptions, and time current tasks. Validate prototypes with staff before committing to architecture.

Câu 822. What questions would you ask a warehouse operator who reports that the inventory system is confusing?

Answer (English): Ask what task they were trying to finish, which screen/field blocked them, what information they needed, how often it happens, and what workaround they use. Observe a real example.

Câu 823. What would you do if operators consistently bypass a workflow you designed?

Answer (English): Assume the workaround reveals friction or a missing business rule; observe, interview, measure consequences, then simplify or change process. Do not merely add warnings without understanding motivation.

Câu 824. How would you determine whether a workflow needs better software or better operational procedures?

Answer (English): Separate usability gaps from missing physical labels, training, shelf layout, and inconsistent policies. Trace root cause through observation and metrics, then choose the smallest effective operational/software intervention.

Câu 825. How would you measure the success of a new warehouse feature?

Answer (English): Measure task duration, scan accuracy, exception volume, stock discrepancies, adoption, and error recovery time before and after release. Add qualitative operator feedback and guard against worsened correctness.

Câu 826. Which business metrics would you use to evaluate an inventory platform?

Answer (English): Key metrics include inventory accuracy, order fill rate, oversell incidents, pick/pack accuracy, cycle-count variance, time to receive/ship, returns disposition lag, and integration lag/failure.

Câu 827. How would you prioritize reducing stock discrepancies versus adding a new feature?

Answer (English): Prioritize discrepancies if they threaten shipments or ledger integrity; size the user impact, risk and remediation. A feature that increases sales while corrupting stock should not ship without safeguards.

Câu 828. What would you do if the product owner requested a feature that could compromise inventory accuracy?

Answer (English): Explain the invariant at risk, demonstrate a concrete failure path, and offer a safe alternative or reduced-scope release. Escalate if the proposed design still permits corrupt stock.

Câu 829. What would you do if an urgent business request conflicted with an upcoming technical migration?

Answer (English): Assess urgency and blast radius, protect in-progress migration safety, and consider separating a small urgent fix from the risky release. Communicate trade-offs and use a clear decision owner.

Câu 830. How would you explain a technical limitation to a nontechnical stakeholder?

Answer (English): Translate the constraint into workflow impact: 'Two people could reserve the last item, so we must serialize allocation; otherwise orders may fail.' Offer timing, safe options, and observable success criteria.

Câu 831. What would you do if product requirements were incomplete or contradictory?

Answer (English): Identify contradictions and unmade decisions explicitly, propose examples and a default, and confirm with the responsible stakeholder. Avoid coding different assumptions into different layers.

Câu 832. How would you clarify acceptance criteria before starting development?

Answer (English): Write scenarios with actor, input, allowed transition, error behavior, concurrency, permissions, and measurable acceptance tests. Confirm how partial fulfillment and retries behave before implementation.

Câu 833. How would you handle a workflow that changes after implementation has begun?

Answer (English): Assess what changed, which invariants and tests are affected, re-scope in small iterations, and document the decision. Avoid silently expanding an already high-risk migration.

Câu 834. How would you avoid building a technically elegant solution that operators find difficult to use?

Answer (English): Observe tasks early, build a narrow prototype, test with barcode/scanner hardware, and iterate from operator metrics. Technical elegance should not increase scans, confusion, or unsafe workarounds.

Câu 835. Tell me about a time you improved a process rather than merely implementing a requested feature.

Answer (English): I would use a real example where I mapped a manual workflow, removed unnecessary steps, measured impact, and established durable ownership. I would not invent a process improvement solely to fill an interview answer.

PHẦN 25. OWNERSHIP, PRIORITIZATION VÀ COMMUNICATION

Câu 836. [ƯU TIÊN CAO] Tell me about a project you owned from requirements through production.

Answer (English): I would present a genuine STAR example: requirement and business goal, my own decisions about schema/API/UI, testing and deployment, and the post-release result. Ownership includes support after release; I would state the actual scope.

Câu 837. Tell me about a time you had to make a technical decision independently.

Answer (English): I would choose a real decision made with incomplete information, explain alternatives, constraints, why I chose one, and what later evidence showed. I would distinguish my decision from team consensus.

Câu 838. Tell me about a time you made a mistake in production.

Answer (English): I would acknowledge an actual mistake, its impact, how I stopped further damage, notified stakeholders, fixed the cause, and prevented recurrence. If I have no production incident, I would say so instead of inventing one.

Câu 839. What did you learn from that mistake?

Answer (English): The important lessons should follow from the true incident: explicit invariants, safer migrations, tests at concurrency boundaries, and faster escalation. I would state precisely what changed in my workflow.

Câu 840. Tell me about a time you had to debug an unfamiliar codebase.

Answer (English): I begin with local setup, request/data flow diagrams, tests, logs, schema and recent changes, then reproduce one behavior. I make a small verified fix before large refactoring.

Câu 841. Tell me about a time you worked with incomplete requirements.

Answer (English): I would describe a real ambiguity, the questions I asked, the documented assumptions, incremental prototype, and how I verified acceptance. I would not present conjecture as delivered work.

Câu 842. Tell me about a time you disagreed with a product owner.

Answer (English): I would show how I framed a disagreement around business risk and options, listened to the owner, and reached a recorded decision. Critical integrity risks should be escalated respectfully.

Câu 843. Tell me about a time you received critical feedback on your code.

Answer (English): I would give a genuine feedback example, distinguish style from correctness, apply the improvement, and describe how I changed review habits. I would not claim feedback that did not occur.

Câu 844. Tell me about a time you had to prioritize between technical debt and a business deadline.

Answer (English): I would assess technical debt by risk and time cost, protect safety/security/stock integrity, and negotiate scope around the deadline. The example should include the decision and observed outcome.

Câu 845. Tell me about a time you improved a system without being explicitly asked.

Answer (English): I would identify a real improvement motivated by logs, user friction, or repetitive toil, explain the small proposal, implementation, and evidence of value. No fabricated metrics.

Câu 846. Tell me about a time you had to learn a new technology under time pressure.

Answer (English): I would describe how I used documentation, a vertical slice, integration tests, and reviewer support to learn one real tool. I would distinguish learning a demo from safely shipping production use.

Câu 847. Tell me about a time you had to communicate a technical problem to a nontechnical person.

Answer (English): I would explain a real technical issue in plain workflow terms, give impact and safe options, then check that the stakeholder understood the next step. The interview answer should include outcome.

Câu 848. How do you estimate the effort required for an unfamiliar task?

Answer (English): I break down discovery, schema/API/UI, test and rollout tasks; identify unknowns, spikes, dependencies, and risk buffers. I give ranges and update estimates when evidence changes.

Câu 849. How do you decide what to do when several tasks are urgent?

Answer (English): Rank work by business impact, urgency, reversibility, incident severity, and dependencies, then clarify priority with owners. Protect active production incidents and correctness before cosmetic work.

Câu 850. What do you do when you realize you will miss a deadline?

Answer (English): Communicate early with a revised estimate, reasons, completed work, remaining risks, and options to reduce scope. Do not hide delays until the deadline.

Câu 851. How do you communicate progress in an asynchronous team?

Answer (English): Post concise updates with what shipped, what is in progress, decisions needed, blockers, next steps, and links to code/issues. Make written handoffs actionable across time zones.

Câu 852. How do you handle unclear ownership of a technical problem?

Answer (English): Identify affected systems and likely owners, take responsibility for initial triage, and establish an explicit incident/task owner. Do not let a cross-team boundary leave customers waiting.

Câu 853. How do you balance independence with asking for help?

Answer (English): Work independently on bounded decisions, but ask early when data integrity, security, schema direction, or unclear business policy makes mistakes expensive. Bring evidence and proposed options with the question.

Câu 854. When should a developer escalate a risk to the Engineering Manager?

Answer (English): Escalate potential corruption, security exposure, customer/warehouse outage, risky irreversible change, or a decision beyond agreed authority. Share severity, affected scope, containment, and options.

Câu 855. What does accountability mean when using AI-generated code?

Answer (English): I remain responsible for requirements, security, tests, and outcomes even if AI wrote the initial code. The tool has no accountability for a duplicate shipment or broken transaction.

Câu 856. How do you ensure your code is maintainable by another engineer?

Answer (English): Use clear boundaries, meaningful names, tests, schema/docs, small changes, and observable failures. Ensure another engineer can diagnose and extend a workflow without reverse-engineering prompts.

Câu 857. How do you decide when to refactor existing code?

Answer (English): Refactor when change cost, duplication, or defect risk justifies it; preserve behavior with tests and separate refactor from business feature changes when possible. Avoid broad rewrites during incidents.

Câu 858. How do you prevent scope creep?

Answer (English): Define acceptance criteria and non-goals, track new requests separately, and evaluate their impact on deadline and risk. New stock invariants require explicit review rather than quiet scope expansion.

Câu 859. How do you make progress visible without attending excessive meetings?

Answer (English): Use issue boards, focused PRs, asynchronous demos, changelogs, and regular concise updates. Meetings should resolve blockers/decisions rather than substitute for written work visibility.

Câu 860. How do you work effectively when teammates are in different time zones?

Answer (English): Set predictable overlap hours, document decisions, hand off with context, use async-first communication, and schedule rare real-time discussions for urgent blockers. Protect time for deep work and recovery.

Câu 861. What do you include in a written technical update?

Answer (English): Include current status, shipped changes, tests/results, measurable impact, blockers/risks, decision needed, owner, and next action. Link to a ticket/PR rather than repeating large logs.

Câu 862. How do you communicate a decision that has long-term architectural consequences?

Answer (English): Write a short ADR with context, options, invariants, trade-offs, consequences, rollout/reversal, and decision owner. Surface risks early to affected teams.

Câu 863. Tell me about a time you helped a teammate become more effective.

Answer (English): I would use a real mentoring/code review example and say what the teammate needed, how I helped, and what became easier. I would not assert mentorship experience without evidence.

Câu 864. What would you do if you discovered a critical issue shortly before the end of your workday?

Answer (English): Contain serious risk, notify the incident owner, document state and next safe steps, and arrange an explicit handoff. Don't leave a stock-corruption incident unowned overnight.

Câu 865. What would you do if a warehouse operator asked for an immediate manual fix but the cause was unclear?

Answer (English): Acknowledge urgency but avoid an unverified manual update; temporarily stop affected action, inspect SKU/bin event history and count, and escalate. Apply a scoped approved correction only after evidence is clear.

PHẦN 26. CLAUDE CODE VÀ AI-ASSISTED DEVELOPMENT

Câu 866. [ƯU TIÊN CAO] How do you currently use AI coding assistants in your development workflow?

Answer (English): I use an AI assistant to locate code, suggest test cases, draft routine code/docs, and compare approaches; I verify generated changes through code review, type checks, tests, SQL plans, and domain invariants. I never let it authorize production stock writes.

Câu 867. Have you used Claude Code, Cursor, GitHub Copilot, or similar tools?

Answer (English): I would name only assistants I have genuinely used, their actual roles, and limitations. Tool familiarity is less important than owning and verifying the code produced.

Câu 868. How would you use Claude Code to understand an unfamiliar codebase?

Answer (English): Ask for a read-only architectural map, identify domain entry points, schema/migrations, call graph, and tests; then verify suggestions by reading the repository myself. Protect secrets and sensitive code.

Câu 869. How would you use AI to help design a database schema?

Answer (English): Provide real business requirements and invariants, request candidate relationships/constraints and failure cases, then review with SQL knowledge and operators. Use migrations and a real test DB to validate the design.

Câu 870. How would you use AI to generate unit and integration tests?

Answer (English): Use AI to enumerate boundary and concurrency cases and draft tests, then ensure the tests assert independent expected outcomes rather than simply repeating the implementation's assumptions.

Câu 871. How would you use AI to investigate a production bug?

Answer (English): Share sanitized logs/stack traces and safe code snippets, ask for hypotheses, then validate each with instrumentation or reproduction. Do not send raw credentials/customer data or execute suggested production repairs blindly.

Câu 872. How would you use AI to draft database migrations?

Answer (English): Generate a preliminary migration and checklist, manually inspect every SQL statement for table rewrites/locks/data loss, rehearse on staging, and prepare compatibility/recovery plans.

Câu 873. How would you use AI to document an API or create a runbook?

Answer (English): Draft API examples or incident steps with AI, then run each command in a safe environment and have the owner verify it. Documentation must match actual deployed contracts.

Câu 874. How do you verify code generated by an AI assistant?

Answer (English): Read the diff line by line, check dependencies, validate types and schemas, run unit and real DB integration tests, inspect query plans/security/transactions, and compare behavior to acceptance criteria.

Câu 875. What are the risks of accepting AI-generated SQL without understanding it?

Answer (English): AI-generated SQL can omit WHERE conditions, misunderstand key relationships, cause table rewrites, bypass RLS, or break concurrent invariants. Review and test against representative PostgreSQL schemas before running.

Câu 876. How would you verify that an AI-generated migration preserves data integrity?

Answer (English): Compare row counts/key sets, checksums and ledger balances before/after, test old/new compatibility, measure lock impact, and rehearse failure/restore paths. Review generated SQL rather than trusting a 'migration passed' message.

Câu 877. How would you prevent an AI assistant from introducing incorrect assumptions into a business workflow?

Answer (English): Give explicit domain invariants and field ownership, ask AI to list assumptions, and validate them with operators/schema. Reject generated code that invents business behavior or creates hidden state changes.

Câu 878. What would you do if Claude Code generated code that passed tests but was architecturally incorrect?

Answer (English): Passing tests do not prove architectural correctness if tests miss invariants. Reject or redesign the code, add tests for the violated boundary, and explain the trade-off in review.

Câu 879. How would you review AI-generated code that modifies inventory events?

Answer (English): Verify immutability, event schema/version, signed delta, actor/source, unique operation ID, transaction lock scope, and compensation behavior. Require concurrency/rollback tests before merging ledger changes.

Câu 880. How would you use AI without leaking customer data, API keys, or confidential source code?

Answer (English): Use organization-approved AI tools and data policies; remove credentials/PII, minimize uploaded code, review retention/sharing settings, and keep production repair commands under human control.

Câu 881. What information should you avoid sending to an external AI service?

Answer (English): Never send API keys, session cookies, customer personal data, sensitive contracts, production dumps, or confidential source without permission. Use synthetic/anonymized examples instead.

Câu 882. How do you decide whether an AI-generated test is actually testing the intended behavior?

Answer (English): Require assertions on business invariants and independent oracles, deliberately break implementation to check the test fails, and test negative/edge cases. An AI-generated happy-path test is not enough.

Câu 883. How would you use AI to explore unfamiliar Deno or Hono code?

Answer (English): Ask AI to explain runtime-specific imports/routes and recommend a small read-only walkthrough; verify against official docs and execution in the actual Deno/Hono environment.

Câu 884. How would you verify an answer produced by AI when working with an unfamiliar library?

Answer (English): Check official versioned documentation, types/source, actual runtime behavior, and a focused test or spike. Do not treat an AI citation or invented API as authority.

Câu 885. What is the difference between using AI to accelerate implementation and delegating engineering judgment to AI?

Answer (English): AI accelerates drafting/search; engineers still choose invariants, transaction boundaries, permissions, trade-offs, and deployment decisions. Judgment cannot be outsourced to a tool without accountability.

Câu 886. How do you maintain a clear understanding of the code produced with AI?

Answer (English): Keep changes small, explain every unfamiliar function, trace side effects, and add tests; reject code I cannot debug or justify. A tool's speed is not worth opaque critical infrastructure.

Câu 887. How would you use AI to refactor code without introducing regressions?

Answer (English): Lock in behavioral tests, refactor small modules with clear diffs, profile if performance-related, and rerun integration/contract/concurrency suites. Avoid unrelated changes in the same PR.

Câu 888. How would you establish a review process for AI-generated code?

Answer (English): Require normal PR review, automated checks, schema/security review for risky code, real DB tests, and explicit author responsibility regardless of AI assistance. Document unusual assumptions and generated dependencies.

Câu 889. Have you ever rejected code suggested by an AI tool? What was wrong with it?

Answer (English): I would cite only a real rejected suggestion, such as a transaction split or unchecked cast, and explain the failing invariant. If I have no such event, I would discuss a hypothetical review criterion, clearly labeled.

Câu 890. How would you explain to a teammate why an AI-generated implementation is unsafe?

Answer (English): Show a concrete failure trace—for example two concurrent allocations both succeeding—then explain why a unique idempotency key and row lock are needed. Focus on evidence rather than criticizing the tool.

Câu 891. What kinds of tasks benefit most from AI coding assistance?

Answer (English): AI helps with boilerplate, documentation drafts, code navigation, refactor suggestions, test outlines, and repetitive transformations. It is less reliable at hidden business rules and high-risk migrations without human verification.

Câu 892. What kinds of tasks should always receive especially careful human review?

Answer (English): Authorization, transaction boundaries, inventory movements, migrations, security configurations, production scripts, and outbound financial/fulfillment effects need especially careful human review.

Câu 893. How would you document a decision that was informed by AI-assisted research?

Answer (English): Record the decision context, sources and versions checked, assumptions, alternatives, verification experiments, and responsible reviewer in an ADR. The fact that AI suggested something is not the rationale.

Câu 894. If you have not used Claude Code extensively, how would you become productive with it before starting?

Answer (English): Review official workflow and access controls, practice on a disposable repository, inspect generated diffs, learn test/review commands, and complete a small noncritical feature with review before touching stock-critical code.

PHẦN 27. DOCUMENTATION VÀ TECHNICAL WRITING

Câu 895. What makes technical documentation useful?

Answer (English): Useful docs answer what the system does, why design choices exist, how to operate/change it, and how to recover from failure. They must be searchable, versioned, and tested against actual behavior.

Câu 896. What should be included in a database schema document?

Answer (English): Document entities/ERD, columns/types, primary/foreign keys, uniqueness/constraints, indexes, ownership, migrations, RLS, and sample integrity queries. Highlight which tables are authoritative versus derived.

Câu 897. How would you document an inventory event model?

Answer (English): Describe event types, versions, fields, signed balance effects, timestamp semantics, ordering, idempotency, correction policy, stock status buckets, and replay/projection process with examples.

Câu 898. How would you document an external API integration?

Answer (English): Document auth/scopes, endpoints/GraphQL queries, rate limits, data ownership, versioning, webhook verification, retry and dedupe semantics, failure modes, and reconciliation/runbooks.

Câu 899. What should an operational runbook include?

Answer (English): Include trigger/symptoms, severity, ownership, prerequisites, safe read-only diagnostics, containment, recovery commands, verification queries, escalation, and communication templates.

Câu 900. How would you document a workaround for a known production issue?

Answer (English): State affected version/environment, exact symptoms, scope, temporary steps, side effects, expiration/owner, and permanent-fix ticket. Warn when the workaround cannot be safely repeated.

Câu 901. What is an architecture decision record (ADR)?

Answer (English): An Architecture Decision Record captures a technical choice, its context, alternatives, rationale, consequences, and status. It preserves why the system was designed a certain way.

Câu 902. When is an ADR useful?

Answer (English): Use an ADR for impactful or hard-to-reverse decisions: inventory ledger, stock locking strategy, field ownership, outbox design, or major runtime migration—not every trivial refactor.

Câu 903. How would you document why the team chose an append-only event ledger?

Answer (English): Explain audit/replay/correction requirements, mutable-quantity alternatives, ledger schema, concurrency/projection cost, replay guarantees, and accepted complexity. Define nonnegotiable business invariants.

Câu 904. How would you document an irreversible database migration?

Answer (English): Document affected data, old/new formats, destructive steps, compatibility plan, backup proof, verification queries, recovery/forward-fix strategy, and exactly who approved deployment.

Câu 905. How do you keep documentation from becoming outdated?

Answer (English): Keep docs next to code, require docs changes in relevant PRs, assign owners, run executable examples/tests in CI where possible, and revisit runbooks after incidents.

Câu 906. What information should be included in a pull request description?

Answer (English): Write purpose, user impact, implementation summary, migrations, risk/rollback, security/invariant effects, testing evidence, screenshots where useful, and related design decisions.

Câu 907. How would you write a technical explanation for warehouse operators?

Answer (English): Use plain language, numbered steps, exact screen labels, expected result, and what to do on error. Avoid exposing SQL internals or asking operators to make ambiguous stock corrections.

Câu 908. How would you communicate a breaking change to other developers?

Answer (English): Describe affected endpoint/schema, migration timeline, compatibility window, required consumer changes, examples, validation steps, and owner/contact. Prefer additive changes and explicit deprecation dates.

Câu 909. How would you document the recovery procedure for a failed synchronization job?

Answer (English): List how to find the job/checkpoint, inspect failed records, fix auth/provider issues, safely resume/replay with the same keys, then reconcile counts and monitor backlog until clear.

Câu 910. How would you describe a complex technical trade-off concisely in writing?

Answer (English): State the goal, two viable options, relevant constraints, a concise cost/risk comparison, recommendation, and reversal condition. Use concrete failure scenarios to make the trade-off understandable.

PHẦN 28. ASYNC SCREENING VÀ TAKE-HOME EXERCISE

Câu 911. Describe your experience with PostgreSQL and relational database design.

Answer (English): I would describe the real schemas, joins, constraints, migrations, indexing, and transaction handling I have personally implemented. Where experience is limited, I would

demonstrate concrete SQL design and tests without claiming production scale.

Câu 912. Describe a production application you built using TypeScript.

Answer (English): I would select an actual TypeScript application, trace a complete user action through frontend, API, validation, database and deployment, and identify code I wrote. I would distinguish deployed results from an unfinished demo.

Câu 913. What is the most complex database schema you have designed or maintained?

Answer (English): I would explain the most complex schema I truly owned, including key relationships, constraints, indexing, and one migration challenge. I would state measured size or complexity only if verifiable.

Câu 914. How have you ensured data consistency in a production system?

Answer (English): I enforce consistency through database keys/checks, atomic transactions, concurrency controls, runtime validation, idempotent commands, and invariant tests. I would pair the principles with an authentic project example if available.

Câu 915. Have you used append-only logs, event sourcing, or audit trails?

Answer (English): I would state whether I have personally shipped event sourcing or only studied/implemented sample ledgers. I can explain immutable events, signed deltas, corrections, deterministic replay, and projection reconciliation without claiming experience I lack.

Câu 916. How do you handle database transactions?

Answer (English): I group dependent writes into a short database transaction, use a single transaction handle through repositories, lock shared stock keys when necessary, and roll back on failure. I retry serialization/deadlock errors with stable operation IDs.

Câu 917. Describe a time you resolved a data integrity problem.

Answer (English): I would explain one real discrepancy, how I traced its origin, applied a narrow correction, and prevented repeat failure. Without such production experience, I would answer by describing a safe investigation approach, not a fictional story.

Câu 918. How do you test database-related business logic?

Answer (English): I test pure rules, real PostgreSQL constraints/functions, rollback, two concurrent writers, permissions, idempotency, and migrations. The authoritative outcome is checked independently from the service response.

Câu 919. What experience do you have integrating external APIs?

Answer (English): I would state APIs I actually integrated and cover authentication, rate limits, schema mapping, retries, deduplication, logs, and data reconciliation. I would not claim to have shipped a Shopify integration if I have not.

Câu 920. How do you handle duplicate webhook events?

Answer (English): I verify signatures, persist a unique provider event ID in an inbox, and process with an idempotent business operation key. Concurrent duplicates acknowledge success but cannot apply the movement twice.

Câu 921. What would you prioritize when taking over an unfamiliar production system?

Answer (English): Map the schema and data owners, read incident history, identify inventory invariants and permissions, reproduce the main workflows, and establish observability. Ship a small tested change before major redesign.

Câu 922. How do you use AI tools in your engineering workflow?

Answer (English): AI is useful for repository navigation, test ideas, code drafts, and docs; I independently verify types, runtime behavior, SQL, permissions and business invariants. I would disclose which assistants I truly use.

Câu 923. Share an example of a feature you delivered end-to-end.

Answer (English): I would describe a genuine end-to-end feature with acceptance criteria, my code contributions, tests, release, and measurable impact. The answer must reflect my real responsibility, not the whole team's.

Câu 924. What would you improve in a project you have recently built?

Answer (English): I would identify a real architecture/UX/test improvement justified by current evidence and describe a bounded migration path. I would not invent deficiencies of a project the interviewer cannot inspect.

Câu 925. Link to a code sample that best demonstrates your engineering ability and explain why you selected it.

Answer (English): I would provide an actual accessible repository or approved redacted code sample, identify an important module, and explain design trade-offs/tests. If code is private, I would seek permission or present a newly written equivalent rather than expose confidential work.

Câu 926. Design a PostgreSQL schema for products, SKUs, bins, warehouses, orders, and inventory events.

Answer (English): Use the complete SQL schema in Appendix A: normalized catalog, warehouse/bin, orders, immutable events, idempotency and stock-key locks. Enforce FKs/uniques/checks and indexes; test migrations before applying to a real system.

Câu 927. Write SQL to derive the inventory balance for each SKU and bin.

Answer (English): `SELECT sku_id, bin_id, SUM(delta) AS on_hand FROM inventory_events GROUP BY sku_id,bin_id;` gives the signed balance under the defined unit/status semantics. The appendix includes practical SQL and zero-balance considerations.

Câu 928. Write a query to identify duplicate external product IDs.

Answer (English): `SELECT platform,account_id,external_variant_id,COUNT(*) FROM channel_mappings GROUP BY 1,2,3 HAVING COUNT(*)>1;` finds duplicate external keys; add a scoped unique constraint after cleaning verified duplicates.

Câu 929. Write a query to find inventory events that may have created an unexpected balance.

Answer (English): Compute running balances partitioned by `(sku_id,bin_id)` ordered by event sequence, and filter transitions into invalid negative territory or around reported discrepancy time. A negative running balance is a clue, not proof of one particular root cause.

Câu 930. Implement a TypeScript function that calculates inventory movements from an event list.

Answer (English): Use a typed discriminated event union and an exhaustive `switch` to turn each event into a signed delta, then reduce with `BigInt` for integer units. Validate input before reduction; code appears in Appendix B.

Câu 931. Write a TypeScript function that validates an incoming inventory event.

Answer (English): Validate unknown data using runtime checks/schema: recognized type, valid IDs, integer positive units (or signed adjustment delta), reason, and event-specific fields. Reject malformed values before database mutation; code in Appendix B.

Câu 932. Implement an API endpoint that records a stock adjustment safely.

Answer (English): Expose `POST /inventory/adjustments`, authenticate and authorize scope, validate payload, and execute a single locked DB transaction inserting a uniquely keyed event.

Return conflict/validation errors and the committed balance; Appendix C shows a complete service pattern.

Câu 933. Add validation to reject malformed quantities and unknown SKUs.

Answer (English): Check integer bounds and event-specific rules in runtime validation, then confirm SKU/bin exist and are in the caller's tenant via transactional SQL. Protect with FKs, check constraints, and authorization.

Câu 934. Write a transaction that records a warehouse transfer between two bins.

Answer (English): Lock both stock keys in stable order, validate source availability, insert debit and credit events under one transfer ID and transaction, then commit. Appendix C includes an executable parameterized SQL pattern.

Câu 935. Prevent a duplicate request from applying the same adjustment twice.

Answer (English): Store a unique `(tenant_id,operation_key)` against the command and ledger entries, claim it in the same transaction as the write, and return its stored result on retries. Test duplicate requests across different connections.

Câu 936. Write tests for valid and invalid stock adjustments.

Answer (English): Include tests for receiving/adjusting valid units, zero/decimal/NaN quantities, wrong SKU/bin, insufficient stock, cross-tenant access, and duplicate keys. Assert both response and actual DB events.

Câu 937. Write a test for two concurrent attempts to allocate the same stock.

Answer (English): Start two simultaneous reservations for a balance of one unit and assert only one commits, one conflicts, and total committed reservations equal one. Use real PostgreSQL connections and repeat the race under load.

Câu 938. Design an idempotent webhook handler.

Answer (English): Verify raw-body provider signature, unique inbox key, fast durable ACK, leased processing worker, idempotent upsert/command, retries, and dead-letter reports. Test duplicates, delayed versions, and response loss.

Câu 939. Create a migration that adds a new field without breaking existing application behavior.

Answer (English): First add a nullable column, deploy code handling old/new data, backfill in batches, validate, enforce required constraint, then remove compatibility code later. Test the migration on existing populated data.

Câu 940. Build a small React interface to search products and display their inventory.

Answer (English): Create an accessible React product search using a debounced API query, pagination, scan-friendly search field, and a table showing SKU/available/reserved/by-bin balance. The UI must never directly mutate authoritative quantity.

Câu 941. Add loading, empty, and error states to the interface.

Answer (English): Show loading indicators while fetching, an explicit empty-result message, retryable sanitized error state, and data with a freshness timestamp on success. Guard against older responses replacing newer search results.

Câu 942. Implement pagination for a product list.

Answer (English): Pass `cursor` and bounded page size to an indexed backend keyset query, return `nextCursor`, and reset cursor when filters change. Keep results keyed by stable product/SKU IDs.

Câu 943. Integrate a mock external API and handle a rate-limit response.

Answer (English): Create a controlled provider client test endpoint returning 429 and `Retry-After`, then assert bounded retry with jitter and a durable checkpoint. The production adapter uses the same code; do not actually hammer real Shopify APIs.

Câu 944. Write a runbook describing how to recover from a failed product synchronization.

Answer (English): The runbook specifies outage symptoms, impacted accounts, provider auth/rate checks, queue IDs/checkpoints, safe replay with original keys, reconciliation, and validation that drift/backlog return to zero; see Appendix E.

Câu 945. Review a pull request and identify risks involving transactions, validation, security, and performance.

Answer (English): Review for non-atomic stock writes, missing unique operation IDs, unauthenticated resource access, unsafe SQL interpolation, unbounded queries, no rollback tests, and missing failure modes. Request tests showing actual invariants before approval.

PHẦN 29. HIRING MANAGER VÀ TECHNICAL ARCHITECT — CÂU HỎI ĐÀO SÂU

Câu 946. Why should inventory logic be enforced in the database rather than exclusively in the application?

Answer (English): The UI or one application worker can be bypassed, while several services and scripts share the same database. Database constraints and transactional routines protect every writer from violating integrity, with backend policy as another layer.

Câu 947. What invariants must always remain true in an inventory system?

Answer (English): Every ledger event has a valid SKU/bin/source; immutable history is preserved; no forbidden negative available stock; paired transfers conserve units; fulfilled quantity never exceeds accepted quantity; duplicate commands produce one effect; permissions remain scoped.

Câu 948. How would you enforce those invariants across multiple concurrent transactions?

Answer (English): Use unique/FK/check constraints plus serialized per-stock-key transactions or Serializable with retries, deterministic locks for transfers, and outbox for external effects. Test concurrent operations on real DB sessions.

Câu 949. What should happen if an inventory operation succeeds in the database but the API response is lost?

Answer (English): The commit remains durable even if the network response disappears, so the client sees an uncertain outcome. Query the operation by idempotency key or retry with the same key and return the original committed result.

Câu 950. How can the client safely retry that operation?

Answer (English): Preserve the same logical operation ID across HTTP retries; backend uniqueness ensures one effect, and duplicates return the stored outcome. A new user-intended adjustment needs a new ID.

Câu 951. What is the difference between exactly-once delivery and exactly-once business effect?

Answer (English): Exactly-once delivery attempts to transmit a message once; exactly-once business effect means duplicates or retries cause only one committed logical side effect. Most systems achieve the latter through idempotency and deduplication.

Câu 952. Why is exactly-once processing difficult in distributed systems?

Answer (English): Crashes and lost acknowledgements create uncertainty between sender, broker, DB, and external service. Transactions generally do not span all systems, so at-least-once delivery plus idempotent effects is more practical.

Câu 953. How would you design an operation that writes database events and subsequently sends a message to another service?

Answer (English): In the same DB transaction as local events, write a uniquely keyed outbox row; a worker later sends the message and records delivery state. The consumer also deduplicates its business effect.

Câu 954. What problem does the transactional outbox pattern solve?

Answer (English): Transactional outbox closes the gap between committing database state and durably recording an intended outbound message. Both rows commit or roll back together, preventing untracked local commits.

Câu 955. How does the outbox pattern differ from publishing a message directly after a database commit?

Answer (English): Publishing after commit can fail/crash before delivery and lose the message; publishing before commit can announce an operation that rolls back. An outbox persists the send intent atomically with the business transaction.

Câu 956. How would you detect and recover from a stuck outbox message?

Answer (English): Track `pending/sending/sent` , attempts, lease expiry, and age; alert on overdue rows and retry with the same message ID. Verify downstream acknowledgements and make receivers idempotent.

Câu 957. How would you handle a webhook that arrives before the database transaction for the related product has committed?

Answer (English): Queue the webhook in the inbox, retry missing references with backoff or fetch current provider state, and avoid manufacturing an orphan. Database visibility/commit ordering must be considered in the consumer.

Câu 958. What if two systems continuously overwrite each other's changes?

Answer (English): An update feedback loop forms; stop one writer, define source/field ownership, add origin/change IDs, suppress echoes, then reconcile the final canonical state once.

Câu 959. How would you define field ownership between Shopify and the internal platform?

Answer (English): Create a field-ownership matrix listing each source for SKU identity, inventory, descriptions, pricing, images, and merchandising metadata per channel. Sync only fields owned by the corresponding system.

Câu 960. How would you distinguish a temporary inconsistency from a permanent data error?

Answer (English): Use version timestamps, queued job age, and documented freshness SLO: temporary lag should resolve within a bounded window, while missing events, invariant failures, or unbounded drift are errors.

Câu 961. How would you know that the system is correct, rather than merely appearing to work?

Answer (English): Prove invariants with real database tests, independent ledger replay, reconciliation against physical counts, sampled audit trails, and monitors that alert on divergence. A successful HTTP response alone is not proof.

Câu 962. Which invariants would you test automatically every day?

Answer (English): Check negative forbidden availability, unbalanced transfer pairs, duplicate operation keys, orphaned event references, projection-vs-ledger balance drift, over-fulfilled order lines, stale failed jobs, and cross-tenant data violations.

Câu 963. How would you design a reconciliation report that identifies the exact cause of a discrepancy?

Answer (English): Show per SKU/bin/status count vs derived balance, cutoff, delta, running events, actor/source, suspicious duplicates, outstanding reservations, and a confidence/category for likely cause. Preserve drill-down and approval workflow.

Câu 964. What would you do if a bug affected 10,000 inventory records but you could not immediately determine which records were wrong?

Answer (English): Freeze suspicious writers and affected operations, take a consistent snapshot, identify a provable affected cohort via event IDs/times/release versions, compare independent recomputation, and repair in audited small batches only after verification.

Câu 965. How would you limit the blast radius of an automated correction?

Answer (English): Require narrow manifest, tenant/warehouse allowlists, small batches, maximum delta/rows, approval threshold, dry-run proof, idempotency keys, and checksums after each batch. Auto-stop on unexpected mismatches.

Câu 966. How would you plan a high-risk migration when you cannot afford to lose inventory history?

Answer (English): Prefer expand-and-contract with preserved old columns, immutable event backup, full rehearsal on production-scale snapshot, checksums, canary migration, and a tested forward-fix plan. Avoid deleting history in place.

Câu 967. How would you prove that a migration preserved every relevant business record?

Answer (English): Compare stable primary key sets, row counts by entity, immutable event checksums and aggregates, foreign-key integrity, and independent reconstructed stock before/after. Audit migration logs and sample critical edge cases.

Câu 968. What is backward compatibility in a database migration?

Answer (English): Backward compatibility means old and new application versions can operate correctly against the transitional schema. Additive columns and dual-read/write windows help avoid simultaneous cutovers.

Câu 969. Why is it sometimes necessary to support old and new data formats simultaneously?

Answer (English): Rolling instances, asynchronous workers, or channel clients may still use old contracts during deployment. Supporting both formats briefly allows safe transition without lost events.

Câu 970. How would you design a safe rollback plan if the database transformation itself cannot be undone?

Answer (English): Prepare the ability to stop new writers, revert compatible application code, rebuild derived projections from preserved immutable data, and apply an audited forward correction. Verify recovery on staging before release.

Câu 971. How would you prioritize correctness, performance, maintainability, and delivery speed when they conflict?

Answer (English): Treat correctness and security as constraints, then balance maintainability, performance, and delivery within those constraints. Favor the simplest design meeting measurable SLOs and explain intentional debt.

Câu 972. What technical decision would you avoid making until you had more information about warehouse operations?

Answer (English): I would not finalize reservation/availability formulas before confirming when stock becomes sellable, how picks consume holds, how damaged/returned items are classified, and how warehouse staff handle offline scans.

Câu 973. What assumptions would you explicitly validate before designing inventory allocation?

Answer (English): Validate stock identity granularity, pick/ship timing, reservation expiry, negative stock policy, cross-bin rules, channel order ingestion, and cancellation/return behavior with real operators before writing allocation code.

Câu 974. How would you work with an Engineering Manager on an unfamiliar database architecture?

Answer (English): First map the existing schema and invariants, bring a concise proposal and failing examples to the Engineering Manager, agree on transaction boundaries and review gates, then implement a small tested slice.

Câu 975. When would you ask for an architecture review instead of making the decision independently?

Answer (English): Ask for review on data model/invariants, authorization, risky migrations, irreversible changes, multi-system consistency, or unfamiliar concurrency assumptions. Handle small low-risk reversible choices independently.

PHẦN 30. FINAL LOGISTICS — HỢP ĐỒNG, LƯƠNG VÀ REMOTE WORK

Câu 976. Are you currently available for a full-time contract engagement?

Answer (English): I would state my actual contract availability and any commitments that might conflict. I should not promise full-time hours unless my schedule genuinely supports them.

Câu 977. How soon could you start?

Answer (English): I would provide the earliest realistic start date after notice periods and preparation, including any time needed for contract and equipment setup. Avoid promising a date that depends on unresolved obligations.

Câu 978. Can you commit to approximately 40 hours per week?

Answer (English): I would confirm only the weekly hours I can consistently deliver, clarify expected overlap and on-call duties, and ask how holidays/time off are handled in a contractor agreement.

Câu 979. Are you comfortable working remotely with a US-based team?

Answer (English): I can describe how I handle asynchronous updates, written documentation, video calls, and secure remote access; I would commit to the arrangement only if my actual environment and time zone allow it.

Câu 980. Can you maintain sufficient overlap with US Eastern business hours?

Answer (English): I would calculate actual US Eastern overlap with Vietnam time, including US daylight-saving changes, and state precisely which recurring window I can maintain. Sustainable scheduling is better than an unrealistic promise.

Câu 981. How do you organize your workday when collaborating across time zones?

Answer (English): I plan around a fixed overlap block for coordination, reserve uninterrupted implementation time, and use written decisions, handoffs, and status reports so work continues across time zones.

Câu 982. Are you comfortable communicating primarily in English?

Answer (English): I would describe my actual ability to discuss architecture, explain trade-offs, write PRs, and handle incidents in English. I would not claim native fluency; clear technical communication matters.

Câu 983. What is your expected hourly rate?

Answer (English): I would quote a realistic USD/hour figure or range based on scope, experience, taxes/benefits, schedule constraints, and contract terms. The stated job band is an input to negotiation, not a guarantee.

Câu 984. How did you determine your expected hourly rate?

Answer (English): I would explain that the rate reflects expected responsibility, consistent availability, remote contractor overhead, and market comparisons, while keeping negotiation linked to actual duties and payment terms.

Câu 985. Are you comfortable working as an independent contractor rather than a local employee?

Answer (English): I would confirm contractor suitability only after reviewing the agreement, scope, payment terms, IP/confidentiality provisions, termination rights, tax responsibility, and relevant local law.

Câu 986. Can you legally provide remote software development services from Vietnam under the proposed arrangement?

Answer (English): I would say that I need to confirm legal eligibility and the written contracting arrangement under applicable Vietnamese and cross-border requirements. I would not assert compliance without reviewing the exact facts or consulting a qualified adviser.

Câu 987. What payment methods and invoicing arrangements would you require?

Answer (English): I would ask for payment currency, accepted bank/transfer method, invoice format, billing cycle, payer fees, exchange-rate treatment, and what happens with overdue invoices. Then state the method I can actually use.

Câu 988. Do you understand the responsibilities associated with contractor taxes and compliance in your jurisdiction?

Answer (English): I understand contractors may have reporting/tax obligations, but the exact treatment depends on legal status and agreement. I would confirm obligations with current official guidance or a qualified tax professional.

Câu 989. What equipment and internet reliability do you have for remote work?

Answer (English): I would describe the computer, backup power/internet arrangements, meeting equipment, and reliability I can verify, not specifications inferred from old configurations. I would explain how I handle outages securely.

Câu 990. Are you willing to attend occasional team gatherings or warehouse visits if required?

Answer (English): I would clarify whether travel is mandatory, frequency, notice, visa requirements, time compensation, and reimbursement before committing. I would state any actual constraints transparently.

Câu 991. What would you need from us to succeed during your first 30 days?

Answer (English): I would need repository and environment access, schema/ledger documentation, prioritized outcomes, a domain expert for warehouse processes, production runbooks, and review feedback on a first small change.

Câu 992. Is there anything in the contract arrangement that you would like to clarify before accepting?

Answer (English): I would clarify hours/overlap, responsibilities and support coverage, contractor IP/confidentiality terms, billing and payment, notice/termination, liability, security requirements, and any travel expectation.

Câu 993. Do you have any constraints around availability, notice periods, or competing commitments?

Answer (English): I would disclose real notice periods, school/work obligations, blackout dates, and other projects that affect availability. I would agree to an overlap window only if it remains sustainable.

Câu 994. What would make you decline an otherwise attractive contract?

Answer (English): I would decline if the agreement requires unsafe production changes without review, impossible hours, unclear payment, unacceptable confidentiality/liability terms, or duties fundamentally different from the stated role.

Câu 995. Is there anything else you would like us to know about your experience?

Answer (English): I would summarize one or two demonstrable strengths, openly name a growth area and how I address it, and express willingness to discuss real code/architecture. I would avoid unverifiable claims about results.

PHẦN 31. CÂU HỎI BẠN NÊN HỎI NGƯỢC NHÀ TUYỂN DỤNG

Câu 996. What are the most important outcomes you expect from this role in the first 30, 60, and 90 days?

Answer (English): Ask this to discover concrete 30/60/90-day priorities and how success will be judged; follow up on which deliverables are nonnegotiable and who decides acceptance.

Câu 997. Which parts of the migration from Airtable to PostgreSQL are still incomplete?

Answer (English): Ask which Airtable entities, relations, scripts, and workflows are still migrating, what data reconciliations remain, and whether legacy writers still operate. The answer exposes migration risk and ownership.

Câu 998. What are the biggest inventory data integrity issues the team is currently facing?

Answer (English): Ask for examples of stock discrepancies, root causes, affected warehouses, recurrence frequency, and current resolution steps. That reveals whether the biggest risk is

scanning, concurrency, or integration.

Câu 999. How is inventory currently calculated, and which invariants are enforced by the database?

Answer (English): Ask whether the ledger, projections, reservation logic, and stock-key locking are documented and which invariants are enforced by DB functions/constraints. Request a concrete receiving-to-pick trace.

Câu 1000. Is the event ledger already established, or would I help design and evolve it?

Answer (English): Ask whether immutable event schema and correction/replay semantics are stable, who owns changes, and whether projections have been independently verified. Do not assume a particular implementation from the job listing alone.

Câu 1001. What does the current database schema look like, and is it documented?

Answer (English): Ask for the current ERD, schema/migration repository, ownership boundaries, and the top tables/functions a new engineer must learn. The state of documentation affects safe onboarding.

Câu 1002. How much of the backend business logic currently lives in PostgreSQL functions, and how much lives in TypeScript?

Answer (English): Ask where transaction boundaries and stock invariants live today, how TypeScript/RPC errors propagate, and what rules must not be duplicated between layers. This clarifies architectural responsibility.

Câu 1003. How much of the product management interface uses Retool versus custom React?

Answer (English): Ask which workflows run in Retool versus React, which UI will be extended first, and whether shared backend APIs already exist. The answer helps plan delivery and testing.

Câu 1004. What are the main technical challenges with the existing Shopify integration?

Answer (English): Ask about API versions, quota limits, field ownership, duplicate/out-of-order webhooks, location mappings, and recurring failures. Seek one current incident example.

Câu 1005. How do you currently monitor and reconcile failed webhooks and marketplace synchronizations?

Answer (English): Ask which inbox/outbox or queue metrics and reconciliation reports exist, who monitors dead letters, and how safe replay is performed. An absence of visibility is an onboarding priority.

Câu 1006. What testing and deployment safeguards are required for database changes?

Answer (English): Ask which DB integration/concurrency tests, staging snapshots, migration reviewers, backup verification, and deployment gates are mandatory. Confirm what happens when a migration fails.

Câu 1007. How are engineering decisions reviewed when there are only a few engineers on the team?

Answer (English): Ask how ADRs, PR approvals, database decisions, and urgent incidents are handled with a small team. Identify which decisions can be made independently and which require review.

Câu 1008. What level of support can I expect from the Engineering Manager when taking ownership of unfamiliar systems?

Answer (English): Ask how often architecture syncs happen, where questions belong asynchronously, and who can clarify ledger and warehouse business semantics. That enables independence without risky assumptions.

Câu 1009. How do you measure the quality and business impact of this role?

Answer (English): Ask whether performance is judged by warehouse error reduction, incident stability, shipped workflows, test quality, or throughput. Seek a measurable baseline and feedback cadence.

Câu 1010. How flexible is the required overlap with US Eastern time?

Answer (English): Ask for actual required ET overlap, daylight-saving expectations, emergency availability, and flexibility around holidays. Convert it to a sustainable local schedule before agreeing.

Câu 1011. Since I would be working from Vietnam as a contractor, what are the payment currency, payment schedule, invoicing, and contract termination terms?

Answer (English): Ask exact invoice frequency, USD/VND handling, transfer fees, due dates, payment dispute procedures, contractor status, and notice clauses. Have terms written in the agreement rather than relying on a verbal promise.

Câu 1012. Are there any mandatory in-person meetings or warehouse visits, and are travel expenses covered?

Answer (English): Ask expected frequency and locations of visits, notice, visa/logistics, travel expense coverage, and whether travel time is paid. Clarify whether travel is optional.

Câu 1013. How do you expect engineers to use Claude Code while ensuring that production code remains safe and reviewable?

Answer (English): Ask which AI tools/data policies are approved, review requirements, tests and permissions for generated code, and how secrets/customer data are protected. Emphasize engineer accountability.

Câu 1014. What are the most common reasons engineers struggle in this role?

Answer (English): Ask for real difficulties—domain learning, time zones, operational incidents, database complexity, or autonomy—and what support helps new hires succeed. The answer reveals hidden expectations.

Câu 1015. What would distinguish an excellent candidate from an acceptable candidate for this position?

Answer (English): Ask what concrete technical decision, exercise, or behavior separates a strong hire from an acceptable one. Follow up with how the team measures correctness, ownership, and communication.

Phụ lục A. PostgreSQL schema thực hành

Mục tiêu: Minh họa một thiết kế có ràng buộc thật, có thể chạy trên PostgreSQL hiện đại để thảo luận ở câu 159, 256–330, 746, 926–929. Ví dụ tập trung vào các đơn vị nguyên và ledger signed-delta. Đây là **reference implementation để học/phỏng vấn**: khi áp dụng cho công ty thực tế, cần xác nhận các chính sách reservation, trạng thái tồn kho, lot/serial, thuế và phân quyền với người phụ trách nghiệp vụ. Không có schema giả định nào có thể tự động thay thế schema production đang dùng.

```
-- PostgreSQL. Chạy trên database phát triển riêng, chưa dùng trực tiếp trên productio
CREATE EXTENSION IF NOT EXISTS pgcrypto;

CREATE TABLE tenants (
  id uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  name text NOT NULL
);

CREATE TABLE products (
```

```
  id uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  tenant_id uuid NOT NULL REFERENCES tenants(id),
  title text NOT NULL,
  archived_at timestamptz,
  UNIQUE (tenant_id, id)
);

CREATE TABLE skus (
  id uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  tenant_id uuid NOT NULL REFERENCES tenants(id),
  product_id uuid NOT NULL,
  code text NOT NULL,
  UNIQUE (tenant_id, id),
  UNIQUE (tenant_id, code),
  FOREIGN KEY (tenant_id, product_id) REFERENCES products(tenant_id, id)
);

CREATE TABLE warehouses (
  id uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  tenant_id uuid NOT NULL REFERENCES tenants(id),
  code text NOT NULL,
  UNIQUE (tenant_id, id),
  UNIQUE (tenant_id, code)
);

CREATE TABLE bins (
  id uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  tenant_id uuid NOT NULL REFERENCES tenants(id),
  warehouse_id uuid NOT NULL,
  code text NOT NULL,
  UNIQUE (tenant_id, id),
  UNIQUE (warehouse_id, code),
  FOREIGN KEY (tenant_id, warehouse_id)
    REFERENCES warehouses(tenant_id, id)
);

CREATE TABLE orders (
  id uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  tenant_id uuid NOT NULL REFERENCES tenants(id),
  external_order_id text,
  status text NOT NULL DEFAULT 'new'
    CHECK (status IN ('new','reserved','picking','packed','shipped','cancelled')),
  created_at timestamptz NOT NULL DEFAULT now(),
  UNIQUE (tenant_id, id)
);

CREATE TABLE order_lines (
  id uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  tenant_id uuid NOT NULL REFERENCES tenants(id),
  order_id uuid NOT NULL,
  sku_id uuid NOT NULL,
  ordered_qty bigint NOT NULL CHECK (ordered_qty > 0),
  FOREIGN KEY (tenant_id, order_id) REFERENCES orders(tenant_id, id),
```

```
  FOREIGN KEY (tenant_id, sku_id) REFERENCES skus(tenant_id, id)
);

-- Một hàng 'stock_keys' là tài nguyên khóa ổn định cho SKU/bin/status,
-- kể cả trước khi nó có movement đầu tiên.
CREATE TABLE stock_keys (
  id uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  tenant_id uuid NOT NULL REFERENCES tenants(id),
  sku_id uuid NOT NULL,
  bin_id uuid NOT NULL,
  stock_status text NOT NULL DEFAULT 'sellable'
    CHECK (stock_status IN ('sellable','receiving','quarantined','damaged')),
  UNIQUE (tenant_id, id),
  UNIQUE (tenant_id, sku_id, bin_id, stock_status),
  FOREIGN KEY (tenant_id, sku_id) REFERENCES skus(tenant_id, id),
  FOREIGN KEY (tenant_id, bin_id) REFERENCES bins(tenant_id, id)
);

-- Theo dõi một logical command (idempotency key), không phải HTTP attempt.
CREATE TABLE inventory_operations (
  id uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  tenant_id uuid NOT NULL REFERENCES tenants(id),
  operation_key text NOT NULL,
  request_hash text NOT NULL,
  status text NOT NULL CHECK (status IN ('processing','completed','rejected')),
  result jsonb,
  created_at timestamptz NOT NULL DEFAULT now(),
  UNIQUE (tenant_id, operation_key),
  UNIQUE (tenant_id, id)
);

-- Source of truth: event bất biến. Không ghi đè số tồn bằng UPDATE.
CREATE TABLE inventory_events (
  id bigint GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  tenant_id uuid NOT NULL,
  stock_key_id uuid NOT NULL,
  operation_id uuid NOT NULL,
  leg smallint NOT NULL DEFAULT 1,
  event_type text NOT NULL
    CHECK (event_type IN
       ('RECEIVED','PICKED','RETURNED','ADJUSTED',
        'TRANSFER_OUT','TRANSFER_IN','STATUS_OUT','STATUS_IN')),
  quantity_delta bigint NOT NULL CHECK (quantity_delta <> 0),
  actor_id uuid,
  reason text NOT NULL,
  occurred_at timestamptz NOT NULL DEFAULT now(),
  recorded_at timestamptz NOT NULL DEFAULT now(),
  original_event_id bigint REFERENCES inventory_events(id),
  UNIQUE (tenant_id, operation_id, leg),
  FOREIGN KEY (tenant_id, stock_key_id)
    REFERENCES stock_keys(tenant_id, id),
  FOREIGN KEY (tenant_id, operation_id)
    REFERENCES inventory_operations(tenant_id, id)
```

```
);

-- Một bảng projection cache có thể tồn tại, nhưng không là nguồn chân lý.
CREATE TABLE inventory_balances_projection (
  stock_key_id uuid PRIMARY KEY REFERENCES stock_keys(id),
  projected_qty bigint NOT NULL,
  through_event_id bigint NOT NULL,
  updated_at timestamptz NOT NULL DEFAULT now()
);

CREATE TABLE channel_mappings (
  id uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  tenant_id uuid NOT NULL REFERENCES tenants(id),
  platform text NOT NULL,
  account_id text NOT NULL,
  external_variant_id text NOT NULL,
  sku_id uuid NOT NULL,
  updated_at timestamptz NOT NULL DEFAULT now(),
  UNIQUE (platform, account_id, external_variant_id),
  FOREIGN KEY (tenant_id, sku_id) REFERENCES skus(tenant_id, id)
);

CREATE TABLE webhook_inbox (
  id uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  provider text NOT NULL,
  account_id text NOT NULL,
  event_id text NOT NULL,
  topic text NOT NULL,
  payload jsonb NOT NULL,
  status text NOT NULL DEFAULT 'received'
    CHECK (status IN ('received','processing','retry_pending','completed','dead_letter
  attempts integer NOT NULL DEFAULT 0 CHECK (attempts >= 0),
  received_at timestamptz NOT NULL DEFAULT now(),
  next_attempt_at timestamptz,
  UNIQUE (provider, account_id, event_id)
);

CREATE TABLE outbox (
  id uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  aggregate_id text NOT NULL,
  message_type text NOT NULL,
  payload jsonb NOT NULL,
  status text NOT NULL DEFAULT 'pending'
    CHECK (status IN ('pending','sending','sent','dead_letter')),
  attempts integer NOT NULL DEFAULT 0,
  created_at timestamptz NOT NULL DEFAULT now()
);

CREATE INDEX inventory_events_stock_history_idx
  ON inventory_events (tenant_id, stock_key_id, id);
CREATE INDEX inventory_events_effective_time_idx
  ON inventory_events (tenant_id, occurred_at, id);
CREATE INDEX webhook_inbox_pending_idx
```

```
  ON webhook_inbox (status, next_attempt_at)
  WHERE status IN ('received','retry_pending');
CREATE INDEX order_lines_order_idx ON order_lines (tenant_id, order_id);

-- Chặn chỉnh sửa/xóa ledger ở cấp độ bảng. DB owner/superuser
-- vẫn có khả năng thay đổi trigger: cần bổ sung REVOKE theo môi trường.
CREATE FUNCTION forbid_ledger_edit() RETURNS trigger
LANGUAGE plpgsql AS $$
BEGIN
  RAISE EXCEPTION 'inventory_events is append-only';
END; $$;
CREATE TRIGGER inventory_events_immutable
BEFORE UPDATE OR DELETE ON inventory_events
FOR EACH ROW EXECUTE FUNCTION forbid_ledger_edit();
```

Các câu SQL quan trọng (kết quả phải phù hợp ý nghĩa stock status của nghiệp vụ):

```
-- (927) Tồn theo từng SKU, bin, status; kể cả các key chưa có event.
SELECT k.tenant_id, k.sku_id, k.bin_id, k.stock_status,
       COALESCE(SUM(e.quantity_delta), 0) AS on_hand
FROM stock_keys k
LEFT JOIN inventory_events e ON e.stock_key_id = k.id
WHERE k.tenant_id = $1
GROUP BY k.tenant_id, k.sku_id, k.bin_id, k.stock_status;

-- (927) Tổng tồn theo warehouse và SKU. Không gọi tất cả là 'sellable'.
SELECT b.warehouse_id, k.sku_id, k.stock_status,
       COALESCE(SUM(e.quantity_delta), 0) AS quantity
FROM stock_keys k
JOIN bins b ON b.id = k.bin_id
LEFT JOIN inventory_events e ON e.stock_key_id = k.id
WHERE k.tenant_id = $1
GROUP BY b.warehouse_id, k.sku_id, k.stock_status;

-- (928) Nếu chưa có UNIQUE, tìm duplicated external IDs:
SELECT platform, account_id, external_variant_id, count(*) AS records
FROM channel_mappings
GROUP BY platform, account_id, external_variant_id
HAVING count(*) > 1;

-- (929) Lịch sử chạy cộng dồn giúp phát hiện thời điểm chuyển âm.
SELECT e.id, e.recorded_at, e.event_type, e.quantity_delta,
       SUM(e.quantity_delta) OVER (
         PARTITION BY e.stock_key_id ORDER BY e.id
         ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW
       ) AS running_qty
FROM inventory_events e
WHERE e.tenant_id = $1 AND e.stock_key_id = $2
ORDER BY e.id;

-- (284/304) Chứng minh net quantity của một transfer bằng 0:
```

```
SELECT operation_id, SUM(quantity_delta) AS net_delta, count(*) AS legs
FROM inventory_events
WHERE event_type IN ('TRANSFER_OUT', 'TRANSFER_IN')
GROUP BY operation_id
HAVING SUM(quantity_delta) <> 0 OR count(*) <> 2;
```

Lưu ý quan trọng: `on_hand` không nhất thiết bằng `available_to_promise` (ATP). Trong hệ thống thật, ATP cần trừ reservation/hold và loại trừ các trạng thái không bán được. Tập trung bảo vệ invariant cuối cùng dưới cùng một transaction, không chỉ tính bằng một truy vấn chạy riêng lẻ ở frontend. Cần phân quyền/RLS, quy tắc actor, chống update gián tiếp, chính sách số âm, và kiểm thử cạnh tranh theo thiết kế thực tế.

Phụ lục B. TypeScript events và validation

Ví dụ sau là code TypeScript có kiểm tra giá trị đầu vào ở runtime, không ép `unknown as Event` để lẩn tránh validation. Dùng Node.js + TypeScript + `zod`.

```
import { z } from 'zod';

const positiveUnits = z.number().int().positive().safe();
const signedUnits = z.number().int().refine(v => v !== 0).safe();
const common = {
  skuId: z.string().uuid(),
  binId: z.string().uuid(),
  operationKey: z.string().min(8).max(200),
};

export const InventoryEventInput = z.discriminatedUnion('type', [
  z.object({ ...common, type: z.literal('RECEIVED'), quantity: positiveUnits }),
  z.object({ ...common, type: z.literal('PICKED'), quantity: positiveUnits }),
  z.object({ ...common, type: z.literal('RETURNED'), quantity: positiveUnits }),
  z.object({
    ...common, type: z.literal('ADJUSTED'),
    delta: signedUnits,
    reason: z.string().min(5).max(500),
  }),
]);

export type InventoryEvent = z.infer<typeof InventoryEventInput>;
export type Result<T, E> =
  | { ok: true; value: T }
  | { ok: false; error: E };

export function parseInventoryEvent(raw: unknown): Result<InventoryEvent, string> {
  const parsed = InventoryEventInput.safeParse(raw);
  return parsed.success
    ? { ok: true, value: parsed.data }
    : { ok: false, error: parsed.error.issues.map(i => i.message).join('; ') };
}
```

```
export function deltaOf(event: InventoryEvent): bigint {
  switch (event.type) {
    case 'RECEIVED':
    case 'RETURNED':
      return BigInt(event.quantity);
    case 'PICKED':
      return -BigInt(event.quantity);
    case 'ADJUSTED':
      return BigInt(event.delta);
    default: {
      const unreachable: never = event;
      throw new Error(`Unhandled inventory event: ${unreachable}`);
    }
  }
}

export function balanceOf(events: readonly InventoryEvent[]): bigint {
  return events.reduce((balance, event) => balance + deltaOf(event), 0n);
}

// Kết quả: +20 -5 +2 -1 = +16.
const sample = [
  { type: 'RECEIVED', skuId: '123e4567-e89b-42d3-a456-426614174000',
    binId: '123e4567-e89b-42d3-a456-426614174001', operationKey: 'receive-001', quanti
  { type: 'PICKED', skuId: '123e4567-e89b-42d3-a456-426614174000',
    binId: '123e4567-e89b-42d3-a456-426614174001', operationKey: 'picked-001', quantit
  { type: 'RETURNED', skuId: '123e4567-e89b-42d3-a456-426614174000',
    binId: '123e4567-e89b-42d3-a456-426614174001', operationKey: 'return-001', quantit
  { type: 'ADJUSTED', skuId: '123e4567-e89b-42d3-a456-426614174000',
    binId: '123e4567-e89b-42d3-a456-426614174001', operationKey: 'adjust-001',
    delta: -1, reason: 'cycle count correction' },
].map(value => InventoryEventInput.parse(value));
console.assert(balanceOf(sample) === 16n);
```

Phân biệt compile-time và runtime: `z.infer` suy ra type sau khi schema đã được định nghĩa; `safeParse` thực sự kiểm tra request đến. `BigInt` dùng cho đơn vị nguyên lớn sau khi kiểm tra `safe integer`. Nếu nghiệp vụ dùng khối lượng, mét hoặc đơn vị lẻ, dùng `NUMERIC` trong PostgreSQL và thư viện decimal thích hợp thay vì tự ý dùng số thực nhị phân.

Phụ lục C. Giao dịch stock chống race và idempotency

Với câu 296, 299, 304, 632–635, 932–938 và 946–956, chỉ nói "dùng transaction" là **chưa đủ**. Phải có lock theo tài nguyên chung, kiểm tra sau lock và unique key cho cùng một business operation. Bên dưới là đoạn **service TypeScript sử dụng PostgreSQL thật** (`npm install pg` và `npm install -D @types/pg typescript`) tương thích với schema Phụ lục A. Ở đây tất cả giao dịch được backend gọi sau khi đã **xác thực/kiểm tra quyền cho tenant, warehouse và actor**; không tự động dùng được như endpoint công khai. Reservation và hàng không bán được cần quy tắc ATP bổ sung theo nghiệp vụ của từng công ty.

```ts
import { createHash } from 'node:crypto';
import pg, { type PoolClient } from 'pg';

const pool = new pg.Pool({ connectionString: process.env.DATABASE_URL, max: 8 });

type Adjustment = {
  tenantId: string;
  actorId: string;     // lấy từ session đã xác thực, không từ body client
  skuId: string;
  binId: string;
  delta: bigint;       // số nguyên; âm là trừ kho
  reason: string;
  operationKey: string; // stable key giữa các HTTP retry
};

type StockResult =
  | { ok: true; balance: string; operationId: string }
  | { ok: false; code: 'INSUFFICIENT_STOCK'; balance: string };

function hashCommand(c: Adjustment): string {
  const normalized = JSON.stringify({
    tenantId: c.tenantId, skuId: c.skuId, binId: c.binId,
    delta: c.delta.toString(), reason: c.reason,
  });
  return createHash('sha256').update(normalized).digest('hex');
}

async function ensureLockedStockKey(
  client: PoolClient,
  tenantId: string,
  skuId: string,
  binId: string,
): Promise<string> {
  // Nếu stock key chưa tồn tại, tạo hàng khóa ổn định trong cùng transaction.
  // FK đảm bảo SKU và bin cùng tenant; lỗi invalid ID rollback transaction.
  await client.query(`
    INSERT INTO stock_keys(tenant_id,sku_id,bin_id,stock_status)
    VALUES ($1,$2,$3,'sellable')
    ON CONFLICT (tenant_id,sku_id,bin_id,stock_status) DO NOTHING
  `, [tenantId, skuId, binId]);

  const rows = await client.query<{ id: string }>(`
    SELECT id FROM stock_keys
    WHERE tenant_id=$1 AND sku_id=$2 AND bin_id=$3
      AND stock_status='sellable'
    FOR UPDATE
  `, [tenantId, skuId, binId]);
  if (rows.rowCount !== 1) throw new Error('STOCK_KEY_NOT_FOUND');
  return rows.rows[0].id;
}

async function authoritativeBalance(
```

```
    client: PoolClient,
    tenantId: string,
    stockKeyId: string,
  ): Promise<bigint> {
    const res = await client.query<{ balance: string }>(`
      SELECT COALESCE(SUM(quantity_delta),0)::text AS balance
      FROM inventory_events WHERE tenant_id=$1 AND stock_key_id=$2
    `, [tenantId, stockKeyId]);
    return BigInt(res.rows[0].balance);
  }

  export async function adjustStock(cmd: Adjustment): Promise<StockResult> {
    if (cmd.delta === 0n) throw new Error('NONZERO_DELTA_REQUIRED');
    if (cmd.reason.trim().length < 5) throw new Error('REASON_REQUIRED');
    if (!cmd.operationKey.trim()) throw new Error('IDEMPOTENCY_KEY_REQUIRED');

    // Important: Before calling this function, verify the actor's right to
    // adjust this tenant/warehouse and enforce approval/quantity limits.
    const client = await pool.connect();
    try {
      await client.query('BEGIN');
      const requestHash = hashCommand(cmd);
      await client.query(`
        INSERT INTO inventory_operations
          (tenant_id,operation_key,request_hash,status)
        VALUES ($1,$2,$3,'processing')
        ON CONFLICT (tenant_id,operation_key) DO NOTHING
      `, [cmd.tenantId, cmd.operationKey, requestHash]);

      // Tranh chấp cùng key phải chờ row/unique-conflict transaction trước.
      const op = await client.query<{
        id: string; request_hash: string; status: string; result: StockResult | null;
      }>(`
        SELECT id,request_hash,status,result
        FROM inventory_operations
        WHERE tenant_id=$1 AND operation_key=$2 FOR UPDATE
      `, [cmd.tenantId, cmd.operationKey]);
      const command = op.rows[0];
      if (!command) throw new Error('OPERATION_NOT_FOUND');
      if (command.request_hash !== requestHash) {
        throw new Error('IDEMPOTENCY_KEY_REUSED_FOR_DIFFERENT_COMMAND');
      }
      if (command.status === 'completed' || command.status === 'rejected') {
        await client.query('COMMIT');
        return command.result!; // DB constraint/service contract requires result
      }

      const stockKeyId = await ensureLockedStockKey(
        client, cmd.tenantId, cmd.skuId, cmd.binId,
      );
      const current = await authoritativeBalance(client, cmd.tenantId, stockKeyId);
      const next = current + cmd.delta;
```

```
      if (next < 0n) {
        const result: StockResult = {
          ok: false, code: 'INSUFFICIENT_STOCK', balance: current.toString(),
        };
        await client.query(`
          UPDATE inventory_operations
          SET status='rejected', result=$2::jsonb WHERE id=$1
        `, [command.id, JSON.stringify(result)]);
        await client.query('COMMIT');
        return result;
      }

      await client.query(`
        INSERT INTO inventory_events
          (tenant_id,stock_key_id,operation_id,leg,event_type,
           quantity_delta,actor_id,reason)
        VALUES ($1,$2,$3,1,'ADJUSTED',$4::bigint,$5,$6)
      `, [cmd.tenantId, stockKeyId, command.id, cmd.delta.toString(),
          cmd.actorId, cmd.reason]);

      const result: StockResult = {
        ok: true, balance: next.toString(), operationId: command.id,
      };
      await client.query(`
        UPDATE inventory_operations
        SET status='completed', result=$2::jsonb WHERE id=$1
      `, [command.id, JSON.stringify(result)]);
      await client.query('COMMIT');
      return result;
    } catch (error) {
      await client.query('ROLLBACK');
      throw error;
    } finally {
      client.release();
    }
  }
```

Vì sao đoạn này chống race? Hai request cùng stock key sẽ tranh chấp `FOR UPDATE`, nên request thứ hai chỉ tính lại tồn sau khi request trước commit. Hai lần gửi cùng `operationKey` tranh chấp unique key ở `inventory_operations`, sau đó trả về **cùng kết quả** thay vì insert thêm event. `request_hash` từ chối tái sử dụng một key cho command khác. Trường hợp thiếu stock cũng được lưu là một kết quả bị từ chối để retry không đổi quyết định. Với PostgreSQL transaction mức `READ COMMITTED`, các điểm lock và tái đọc ở trên là điều kiện thiết yếu; thay đổi isolation cần kiểm thử lại.

Cần bổ sung trước khi dùng production: Xác thực JWT/session, authorize warehouse membership, validate UUID/bounds tại runtime, chính sách `sellable − reserved`, phân quyền adjustment/approval, kiểm tra request body size và rate limiting, chính sách retry deadlock, tracing, deployment/RLS, transaction timeout, và kiểm thử với tải thực. Đây là code giải thích transaction/idempotency, không phải một API production hoàn chỉnh.

Chuyển hàng giữa hai bins: SQL trong một transaction (ở ví dụ này `:...` là tham số được backend bind; không đưa dấu `:...` vào PostgreSQL nguyên văn):

```
BEGIN;
-- Bước 1: đã có 2 stock_keys của cùng tenant/SKU/status.
-- Bước 2: khóa 2 keys theo cùng thứ tự ở mọi nơi để giảm deadlock.
SELECT id FROM stock_keys
WHERE tenant_id = $1 AND id = ANY($2::uuid[])
ORDER BY id
FOR UPDATE;
-- Bước 3: kiểm tra authoritative source balance ở trong transaction.
SELECT COALESCE(SUM(quantity_delta),0) AS source_balance
FROM inventory_events
WHERE tenant_id=$1 AND stock_key_id=$3;
-- Backend reject nếu source_balance < units.
-- Bước 4: $5 là id của inventory_operations đã claim idempotently.
INSERT INTO inventory_events
  (tenant_id,stock_key_id,operation_id,leg,event_type,quantity_delta,reason)
VALUES
  ($1,$3,$5,1,'TRANSFER_OUT',-$4::bigint,'bin transfer'),
  ($1,$6,$5,2,'TRANSFER_IN', $4::bigint,'bin transfer');
-- Bước 5: đánh dấu inventory_operations hoàn thành trong cùng transaction.
COMMIT;
```

Nếu có bất kỳ lỗi nào giữa hai `INSERT`, transaction rollback toàn bộ. Không được mở một transaction khác cho bước ghi hàng đích. **Quan trọng:** `$2` phải chứa đúng 2 key, `source_balance` phải được ứng dụng kiểm tra trước insert và actor/tenant phải được authorize; đây là mẫu SQL mô tả các bước được backend điều phối, không phải script tự thực thi với input chưa bind.

HMAC webhook Shopify: xác minh raw body

```
import { createHmac, timingSafeEqual } from 'node:crypto';

export function verifyShopifyWebhook(
  rawBody: Buffer,
  providedBase64Signature: string,
  secret: string,
): boolean {
  const calculated = createHmac('sha256', secret).update(rawBody).digest();
  const supplied = Buffer.from(providedBase64Signature, 'base64');
  return supplied.length === calculated.length &&
    timingSafeEqual(supplied, calculated);
}

// Request handler: read exact raw bytes; verify the header required by
// the provider/version; reject failure; validate JSON structure;
// INSERT webhook_inbox(provider,account_id,event_id,...)
```

```
// ON CONFLICT(provider,account_id,event_id) DO NOTHING;
// acknowledge 2xx only after durable DB commit; process in worker.
```

`HTTP 200` từ GraphQL không đồng nghĩa mutation đã thành công: phải kiểm tra top-level `errors` và `userErrors` theo schema/version của mutation. Webhook có thể duplicate và out-of-order, vì vậy HMAC chỉ xác minh nguồn/nội dung chứ **không** thay thế inbox idempotency hoặc reconciliation.

Phụ lục D. Kiểm thử bằng PostgreSQL thật

Vì các lỗi inventory thường liên quan transaction và race conditions, **unit test giả lập repository không thể chứng minh hệ thống chống overselling**. Đoạn này minh họa một bài integration test chạy trên database phát triển riêng đã được apply schema của Phụ lục A. Không kết nối production. Test dùng `node:test`, `node:assert/strict`, `pg` và `adjustStock` từ Phụ lục C.

```
import test from 'node:test';
import assert from 'node:assert/strict';
import { randomUUID } from 'node:crypto';
import { Pool } from 'pg';
import { adjustStock } from './inventory-service.js';

const pool = new Pool({ connectionString: process.env.TEST_DATABASE_URL });

async function createFixture() {
  // Mỗi test dùng ID riêng để không đụng dữ liệu của test khác.
  const t = await pool.query<{ id: string }>(
    `INSERT INTO tenants(name) VALUES ($1) RETURNING id`,
    [`test-${randomUUID()}`],
  );
  const tenantId = t.rows[0].id;
  const p = await pool.query<{ id: string }>(
    `INSERT INTO products(tenant_id,title) VALUES ($1,'Test product') RETURNING id`,
    [tenantId],
  );
  const s = await pool.query<{ id: string }>(
    `INSERT INTO skus(tenant_id,product_id,code)
     VALUES ($1,$2,$3) RETURNING id`,
    [tenantId, p.rows[0].id, `SKU-${randomUUID()}`],
  );
  const w = await pool.query<{ id: string }>(
    `INSERT INTO warehouses(tenant_id,code)
     VALUES ($1,$2) RETURNING id`,
    [tenantId, `WH-${randomUUID()}`],
  );
  const b = await pool.query<{ id: string }>(
    `INSERT INTO bins(tenant_id,warehouse_id,code)
     VALUES ($1,$2,'BIN-01') RETURNING id`,
    [tenantId, w.rows[0].id],
  );
```

```
    return {
      tenantId, skuId: s.rows[0].id, binId: b.rows[0].id,
      actorId: randomUUID(),
    };
  }

  test('one effect for repeated idempotency key', async () => {
    const ids = await createFixture();
    const command = {
      ...ids, delta: 7n, reason: 'verified receipt count',
      operationKey: `test-adjust-${randomUUID()}`,
    };
    const [first, retry] = await Promise.all([
      adjustStock(command), adjustStock(command),
    ]);
    assert.deepEqual(retry, first);
    assert.equal(first.ok, true);

    const res = await pool.query<{ n: string }>(`
      SELECT count(*)::text AS n FROM inventory_events e
      JOIN inventory_operations op ON op.id=e.operation_id
      WHERE op.tenant_id=$1 AND op.operation_key=$2
    `, [ids.tenantId, command.operationKey]);
    assert.equal(res.rows[0].n, '1');
  });

  test('two concurrent picks cannot consume one available unit twice', async () => {
    const ids = await createFixture();
    await adjustStock({
      ...ids, delta: 1n, reason: 'verified receipt count',
      operationKey: `seed-${randomUUID()}`,
    });
    const [a, b] = await Promise.all([
      adjustStock({ ...ids, delta: -1n, reason: 'pick for order A',
        operationKey: `pick-a-${randomUUID()}` }),
      adjustStock({ ...ids, delta: -1n, reason: 'pick for order B',
        operationKey: `pick-b-${randomUUID()}` }),
    ]);
    assert.equal([a,b].filter(x => x.ok).length, 1);
    assert.equal([a,b].filter(x => !x.ok).length, 1);

    const res = await pool.query<{ balance: string }>(`
      SELECT COALESCE(SUM(e.quantity_delta),0)::text AS balance
      FROM inventory_events e
      JOIN stock_keys k ON k.id=e.stock_key_id
      WHERE k.tenant_id=$1 AND k.sku_id=$2 AND k.bin_id=$3
        AND k.stock_status='sellable'
    `, [ids.tenantId, ids.skuId, ids.binId]);
    assert.equal(res.rows[0].balance, '0');
  });

  test.after(async () => { await pool.end(); });
```

Chú ý khi chạy thử: `adjustStock` trong Phụ lục C đọc `DATABASE_URL`, còn fixture đọc `TEST_DATABASE_URL`: đặt cả hai biến tới **cùng một database test cô lập**. Đây là test giao dịch thực, không dùng mock database. Thực thi trong CI cần provision PostgreSQL và chạy migration trước khi test. Production phải có auth và approval; test trên chỉ kiểm tra invariant của phần lõi transaction, không phải toàn bộ chính sách kinh doanh.

Những test âm tính không được thiếu: invalid `skuId/binId`, thao tác nhầm tenant, cùng idempotency key nhưng body khác, một event sai `quantity_delta`, mất kết nối giữa các bước, deadlock `40P01`, serialization `40001`, điều chỉnh sẽ làm tồn âm, hủy đơn trong lúc pick, và refresh projection sau khi replay. Với Supabase, thêm integration tests thực thi dưới role claims khác nhau để kiểm tra RLS; không chỉ test bằng service-role.

Phụ lục E. Runbook phục hồi sync

Đây là nội dung thực hành cho câu 944 và các câu xử lý sự cố 656–690, 771, 909, 956. Mỗi tổ chức phải thay tên bảng, dashboard và chính sách phê duyệt theo hệ thống thật của mình.

E1. Phát hiện và khoanh vùng

1. Ghi nhận thời điểm bắt đầu, nhà cung cấp/Shopify shop, topic, phạm vi SKU/warehouse bị ảnh hưởng, mức độ tác động tới nhận đơn và fulfillment.
2. Kiểm tra số webhook đang `received/retry_pending/dead_letter`, tuổi bản ghi chờ lâu nhất, lỗi 401/403/429/5xx, phiên bản API, trạng thái credential và worker.
3. Nếu sai lệch có nguy cơ tạo đơn vượt tồn, tạm ngưng luồng **ghi/publish stock** bị ảnh hưởng; không cần ngưng toàn bộ sản phẩm nếu các workflow khác độc lập và an toàn.
4. Chốt cutoff/watermark để việc so sánh trước/sau nhất quán. Bảo toàn event/queue logs; không xóa để "làm sạch dashboard".

```
SELECT provider, account_id, status, count(*) AS events,
       min(received_at) AS oldest
FROM webhook_inbox
GROUP BY provider, account_id, status
ORDER BY provider, account_id, status;

SELECT provider, account_id, event_id, topic, attempts,
       received_at, next_attempt_at
FROM webhook_inbox
WHERE status IN ('retry_pending', 'dead_letter')
ORDER BY received_at ASC
LIMIT 100;
```

E2. Sửa nguyên nhân, khôi phục

1. Khắc phục xác thực/scope, API version, rate limit hoặc schema mapping. Phân biệt lỗi tạm thời và lỗi vĩnh viễn; không retry payload sai định dạng vô hạn.

2. Xác định các event đã xử lý thành công và các event còn thiếu. Kiểm tra idempotency key trước khi chạy lại; ưu tiên **refetch current provider state** cho webhook cập nhật sản phẩm không bảo đảm thứ tự.
3. Khởi động worker lại với concurrency có giới hạn; retry từ checkpoint ổn định và giữ nguyên provider event ID/business operation key. Chuyển bản ghi không thể tự xử lý về exception queue.
4. Chạy incremental reconciliation kể từ watermark gần nhất, có overlap để tránh sót ranh giới thời gian; sau đó chạy full reconciliation cho đối tượng nghi vấn.
5. Không sửa tồn bằng overwrite số lượng dựa trên payload webhook. Nếu phát hiện movement đã ghi sai, xác minh nguyên nhân và thực hiện **compensating event** qua API/RPC được phê duyệt.

E3. Chứng minh phục hồi hoàn tất

- Queue backlog và tuổi pending về dưới ngưỡng SLO đã thống nhất, không còn worker lease treo.
- Webhook đã xử lý không gây double side effect; thử replay một event đã hoàn thành và xác nhận ledger không đổi.
- Số lượng dữ liệu lệch trong báo cáo reconciliation trở về 0 hoặc còn các exception đã có owner rõ ràng.
- Các SKU/bin ảnh hưởng có event replay và physical-count reconciliation phù hợp; không còn negative stock trái chính sách.
- Có báo cáo incident gồm timeline, scope, root cause, mitigation, correction IDs, PR/test ngăn lỗi tái diễn, và thông tin gửi tới warehouse/product stakeholders.

Phụ lục F. Cách trả lời câu quan trọng trong vòng system design 60 phút

Nội dung ngắn gọn để diễn đạt trôi chảy, **không phải** bịa kinh nghiệm đã làm:

“How would you prevent overselling?”

> First, I would clarify whether availability means physical on-hand stock or sellable stock after reservations. I would define a stable stock key for each SKU and location and serialize commands that modify that availability. Inside a short PostgreSQL transaction, I would acquire the key lock, calculate the current authoritative balance, validate the requested quantity, write the ledger/reservation event using a unique business operation ID, and commit. If stock changed, I would return a conflict rather than overwrite data. I would use the same idempotency key for retries and integration-test two concurrent allocations against a real PostgreSQL database. Dashboards and external marketplace stock can be eventually consistent; the local stock decision cannot.

“How would you design the inventory ledger?”

> I would represent each committed movement as an immutable event with a stock key, signed delta, event type and version, operation ID, actor, reason, and both recorded and effective timestamps. Transfers produce a debit and credit in one transaction, while mistakes are corrected with compensating events. Current balances are aggregated or served from a rebuildable projection, never treated as the independent source of truth. I would test replay determinism, transfer conservation, duplicate request handling, and projection-versus-ledger reconciliation.

“How would you design reliable Shopify synchronization?”

> I would define the owner for each field before building the integration. A signed webhook would enter a durable, deduplicated inbox and trigger a worker to reconcile the current provider state. Outbound updates would use a transactional outbox, bounded retries, and stable operation IDs. I would checkpoint catalog pagination and run periodic reconciliation because webhooks can be missed, duplicated, or out of order. I would monitor lag, backlog, permanent failures, and drift, while preventing sync loops through field ownership and origin tracking.

“How would you change a production database safely?”

> I would favor expand-and-contract migrations, preserving old and new application compatibility through rollout. Before deploying, I would inspect locks/table rewrite risks, test the SQL on a representative dataset, verify backup recovery, and define measurable invariants and rollback or forward-fix procedures. I would deploy additive schema first, backfill in batches, switch application reads and writes, and drop legacy fields only after older instances are gone. For inventory history, I would verify event counts and replayed balances before and after the migration.

Hoàn tất phạm vi: 1.015 câu thuộc PHẦN 1–31 của file nguồn, theo đúng đánh số 1–1.015. Phần 32 trong nguồn là hướng dẫn ôn tập, không phải ngân hàng câu hỏi, nên được thay bằng các phụ lục thực hành và bài giải chuyên sâu. Tài liệu này không đại diện cho câu trả lời chính thức của nhà tuyển dụng; câu hỏi trải nghiệm cá nhân chỉ được hoàn thiện thành phát biểu “I have done X” khi X là sự thật có thể kiểm chứng.